

博士学位论文

分布式系统软件部署策略及其可靠性研究

**RESEARCH ON SOFTWARE DEPLOYMENT
STRATEGY AND RELIABILITY IN DISTRIBUTED
SYSTEM**

苏喜红

哈尔滨工业大学

2011 年 12 月

国内图书分类号：TP311.5
国际图书分类号：681.004.05

学校代码：10213
密级：公开

工学博士学位论文

分布式系统软件部署策略及其可靠性研究

博 士 研 究 生：苏喜红

导 师：杨孝宗教授

申 请 学 位：工学博士

学 科：计算机系统结构

所 在 单 位：计算机科学与技术学院

答 辩 日 期：2011 年 12 月

授予学位单位：哈尔滨工业大学

Classified Index: TP311.5

U.D.C: 681.004.05

Dissertation for the Doctoral Degree in Engineering

**RESEARCH ON SOFTWARE DEPLOYMENT
STRATEGY AND RELIABILITY IN DISTRIBUTED
SYSTEM**

Candidate:	Su Xihong
Supervisor:	Prof. Yang Xiaozong
Academic Degree Applied for:	Doctor of Engineering
Speciality:	Computer Architecture
Affiliation:	School of Computer Science and Technology
Date of Defence:	December , 2011
Degree-Conferring-Institution:	Harbin Institute of Technology

摘 要

随着互联网的出现, 计算机软件所面临的环境发生了变化, 由静态、封闭转为动态、开放。为了适应这种变化, 软件系统从集中式逐渐演化为分布式。另外, 从 20 世纪 90 年代开始, 基于组件的软件开发迅速发展并成为主流技术, 基于组件的分布式软件 (component-based distributed software, CBDS) 得到广泛发展, 这类软件广泛应用在应急响应、工业自动化、导航、卫生保健等各个领域。随着分布式软件系统的规模越来越大、功能越来越复杂, 人们对其可靠性、可用性等性质提出了更高的期望和要求。

随着分布式软件和网络的快速发展, 软件部署成为整个软件生命过程中的一个独立阶段。在普适、移动和分布式环境中, 这种环境具有高度动态配置、未知的操作剖面 and 波动的执行环境, 分布在这样环境下的软件的部署对系统性能有重要影响, 并且部署在应急响应系统、导航、民用基础设施等领域的软件要执行关键任务, 要满足严格的可靠性要求。为了使部署后的系统满足一定的可靠性要求, 部署需要考虑多方面的信息, 如待部署主机节点的可靠性、软件组件的可靠性、软件组件的互联性、主机节点的拓扑结构、硬件资源占用 (如 CPU、内存)、软件架构风格、软件组件复制等等。

本文首先分析软件部署方法和软件架构风格 (software architectural styles), 提出一种将基于组件的软件部署和 CS (client-server)、LCS (layered-client-server) 架构风格结合的方法。该方法分析软件部署和架构风格对系统可靠性的影响。给出软件架构风格研究假设, 在 CS 风格中, 软件组件分为服务组件和客户组件; 在 LCS 风格中, 软件组件分为服务组件、中间组件和客户组件。为了分析软件组件在不同主机节点上部署情况对软件组件失效率的影响, 本文提出软件组件的调整前失效概率。为了分析同一架构风格中不同类型的软件组件对系统失效率的影响, 本文提出软件组件的调整后失效概率和软件组件类型的影响因子。

由于系统运行环境内在动态性和不可预测性, 使得影响系统部署的参数也不断发生变化, 没有一种部署架构对整个系统运行期间来说都是最优化的, 因此系统需要重部署来提高可靠性。然而, 找到最大化可靠性的部署架构需要时间 $O(m^n)$ (m 是主机节点数目, n 是软件组件数目), 是一个指数复杂性问题, 需要设计近似算法来重部署系统。影响系统部署因素分为不确定的和确定性, 很难用精确理论来描述所有的影响因素。因此, 本文引入模糊理论来描述这些影响因素, 并设计近似算法来重部署系统, 从而提高系统可靠性。

当系统可靠性低于某一规定值时，通过系统重部署可提高系统可靠性。然而，系统重部署需要某些软件组件从一个主机节点迁移到其它主机节点，某些软件组件提供的服务将暂时不能用，而如果这些软件组件有副本，即使某个软件组件失效，其它的组件副本仍能继续提供服务，因此，通过软件组件复制来提高系统可靠性是另一个研究重点。由于额外的软件组件将消耗系统资源，需要在系统可用资源、服务的重要性和可靠性三者之间找到一个折中策略，Malek 给出一个优化目标函数 OF 。在分析目标优化函数 OF 和贪婪算法的基础上，本文设计了启发式的贪婪复制算法，该算法根据单位带宽的优化值增量 OB 和单位内存的优化值增量 OM 来选择将被复制的软件组件，从而得到更高的优化值。虽然系统仍可以正常工作，但失效的软件组件会将故障传播给与其进行直接交互的其它软件组件。本文也对相关的主机节点失效和软件组件失效引起的故障传播进行了分析。

大多数架构层次可靠性方法都认为软件组件对系统可靠性的评估有重要影响，并且假设系统中的软件组件可靠性是已知的，或者软件组件提供服务的可靠性是已知的。软件组件的失效率受外部因素和内部因素的影响，随着系统运行时间的增加，这些影响因素的值将发生改变，那么软件组件的失效率也将随时间发生改变，因此软件组件的失效率是一个与时间相关的函数。Xie 模型假设组件软件系统为由 n 个组件组成的串行系统，所有组件并行开发，独立测试，组件失效均为系统失效，并假设每个组件的失效过程可建模为 NHPP (non-homogeneous Poisson process)。这里，也采用 Xie 模型的假设，将部署阶段的软件组件失效行为建模为 NHPP 过程。此外 Yacoub 提出的 ALRRA 方法基于如下的假设：组件执行的频率越高，失效的可能性越大。本文将软件组件的失效率设为与软件组件执行频率相关的函数。在此基础上，分析系统运行一段时间内，参数 ρ 、软件组件复制、系统重部署、软件组件间的交互频率矩阵类型对可靠性的影响。用连接器的方式建模集成架构风格，将集成架构风格中不同风格的软件组件用恰当的概率分布函数来建模，分析在同一时间下的软件组件可靠性、不同时间下的软件组件的可靠性和系统可靠性变化情况。

关键词：软件架构；软件部署；系统可靠性；部署算法；软件组件

Abstract

With the emergence of the Internet, computer software is facing the environment change, from static, closed to dynamic, open environment. In order to adapt the change, software system has gradually become from centralization to distribution. In addition, from 1990s, component-based software development has gradually become mainstream technology, component-based distributed software (CBDS) has rapidly developed, and it has been widely used in emergency response system, industrial automation, navigation, health care and so on. As the size and complexity of component-based distributed softwares increasing, the higher sytem reliability and availability are expected.

With the rapid development of distributed software and the network, software deployment became an independent stage of the whole software life process. In pervasive, mobile and distributed enviroments, these environments are characterized by highly dynamic configuration, unknown operational profile and fluctuating execution environment. The software deployment in these environments has important influence on the system performance. The software deployed on the domain such as emergency reflection system, navigation, civilization infrastruccion, performs the critical mission, and needs to satisfy the rigorous reliability requirement. In order to meet certain reliability value by system redeployment, deployment needs to incorporate the influence of many factors, such as reliability of host nodes, component reliability, component interaction, topological structure of host nodes, hardware resources(for example, CPU, memory), software architectural style, component replication and so on.

This paper firstly analyzes software deployment methods and software architectural styles. An approach of combining component-based software deployment with CS(client-server), LCS(layered-client-server) architectural styles is presented. Given some research hypotheses, in CS architectural style, software components can be divided into server component and client component; in LCS architectural style, software components can be divided into server component, middle component and clinent component. In order to analyze the influence of deploying on different host nodes on component failure, this paper provides the faliure probability before adjusting. In order to analyze the influence of different types of software components in the same architecture style on system failure, this paper provides the failure probability after adjusting and influencing factor of software component types.

Because system run-time envrionment with the dynamic and unpredictable

nature, parameters influencing system deployment are changing. Therefore, it is impossible that the specific deployment architecture is still optimal during the whole system execution, and system needs to be redeployed to improve reliability. Determining the deployment architecture that maximizes system reliability is an exponential problem, in general, $O(m^n)$ (m is the number of host nodes, n is the number of software components). In addition, many uncertainty and certainty factors affect system deployment. It is difficult to describe all factors with precision theory. In this paper, fuzzy theory is introduced to describe all influencing factors, and then design approximate algorithms to redeploy system for improving system reliability.

When system reliability decreases to some fixed value, system needs to be redeployed to improve system reliability. However, during deployment process, some software components will migrate from one host node to another host node, and the services provided by these software components can not be used temporarily, and if there are copies of these software components, even if a software component fails, other component replica still provide the service. Therefore, improvement of system reliability by copying software components is another research focus. Since extra software components will consume system resources, it is necessary to find a compromise method in available system resources, service importance and reliability. Malek gives an optimal object function OF. On the basis of OF and greedy algorithm, this paper designs a heuristic greedy replication algorithm. This algorithm uses the increased optimal value of each unit bandwidth OB and the increased optimal value of each unit memory OM to select the software components to replicate, and obtains the higher optimal value. Although system is still function, the failure of some software components can propagate the fault to other directly interacted software components. This paper also analyzes the fault propagation by the failures of software components and host nodes.

Most of reliability estimation methods at architecture level consider that component reliability plays an important role in system reliability estimation, but still assume that reliability of individual software component is known, or the reliability of the service is known. Failure rates of software components are influenced by external and internal factors. With the increasement of system run-time, the values of these factors will change, the failure rates of software components will also change, so the failure rate of software component is a time-related function. Xie model assumes that component-based system consists of n components in serial, all components are parallelly developed, independently tested. Failure of each component can result in system failure. Failure behavior of each component is modeled by NHPP. In this paper, on the basis of Xie assumption, failure behaviors of software components at deployment stage are modeled by NHPP.

In addition, Yacoub proposed the ALRRA method based on following assumption: the higher execution frequency of software components, the greater the likelihood of failure. In this paper, failure rates of software components are supposed to be a function related to execution frequency of software components. Then the influences of parameters ρ , software component replication, system redeployment, the type of interaction frequency matrix on reliability are analyzed. The integrated architectural style is modeled by connectors, software components of different styles are modeled by appropriate probability distribution function. Analyze the component reliability at the same time, and the change of component reliability and system reliability at different time.

Keywords: software architecture, software deployment, system reliability, deployment algorithm, software component

目 录

摘 要	I
ABSTRACT	III
第 1 章 绪 论	1
1.1 课题背景	1
1.2 相关概念	5
1.2.1 软件架构	6
1.2.2 软件架构风格	7
1.2.3 软件组件	8
1.2.4 可靠性	10
1.3 基于组件的分布式软件可靠性研究现状与进展	13
1.4 当前研究工作存在的问题	15
1.5 本文的主要研究内容	16
第 2 章 基于组件部署与架构风格结合的可靠性研究	19
2.1 引言	19
2.2 软件部署方法和架构风格	20
2.2.1 软件部署方法	20
2.2.2 架构风格	21
2.3 调整失效概率和影响因子	24
2.4 基于组件的部署和CS风格结合的可靠性分析	26
2.5 基于组件的部署和LCS风格结合的可靠性分析	28
2.6 实验及结果分析	31
2.6.1 CS风格实验及结果分析	31
2.6.2 LCS风格实验及结果分析	35
2.7 本章小结	38
第 3 章 考虑模糊关系的组件部署算法	39
3.1 引言	39

3.2 相关概念和表示方法	39
3.3 软件组件间和主机节点间模糊矩阵及求解	42
3.3.1 软件组件间和主机节点间的模糊关系	43
3.3.2 软件组件间和主机节点间模糊矩阵求解	44
3.4 模糊贪婪部署算法	45
3.4.1 模糊贪婪部署算法的描述	45
3.4.2 模糊贪婪部署算法的正确性和复杂度分析	49
3.4.3 实验及结果分析	50
3.5 本章小结	53
第4章 基于相关性的组件重部署算法	54
4.1 引言	54
4.2 软件组件间的依赖关系和相关性	54
4.3 已有部署算法	56
4.4 m优先选择算法	57
4.4.1 m优先选择算法描述	57
4.4.2 m优先选择算法的正确性和复杂度分析	60
4.4.3 m优先选择算法的实例说明	62
4.4.4 实验及结果分析	66
4.5 本章小结	67
第5章 组件复制及故障传播的可靠性研究	69
5.1 引言	69
5.2 复制技术	70
5.3 启发式的贪婪复制算法	71
5.3.1 结构描述和目标函数	71
5.3.2 启发式的贪婪复制算法描述	73
5.3.3 启发式的贪婪复制算法的正确性和复杂度分析	75
5.3.4 实验及结果分析	77
5.4 故障传播对服务可靠性影响的分析	83
5.4.1 实验输入	83
5.4.2 相关软件组件失效引起的故障传播分析	84
5.4.3 相关主机节点失效引起的故障传播分析	86

5.5 本章小结	88
第 6 章 组件时间相关失效率的软件部署可靠性研究	89
6.1 引言	89
6.2 NHPP类软件组件模型	89
6.2.1 NHPP类模型	89
6.2.2 软件组件建模	91
6.2.3 实验及结果分析	92
6.3 集成架构风格的可靠性分析	98
6.3.1 可靠性中常用的概率分布	98
6.3.2 集成架构风格的建模	100
6.3.3 实验及结果分析	102
6.4 本章小结	107
结 论	108
参考文献	110
攻读博士学位期间发表的论文及其它成果	123
哈尔滨工业大学学位论文原创性声明及使用授权说明	125
致 谢	127
个人简历	128

Contents

Abstract (In Chinese)	I
Abstract (In English)	III
Chapter 1 Introduction	1
1.1 Background of the subject.....	1
1.2 Related concepts.....	5
1.2.1 Software architecture.....	6
1.2.2 Software architectural style.....	7
1.2.3 Software component.....	8
1.2.4 Reliability.....	10
1.3 State of the art of reliability of component-based distributed software.....	13
1.4 Existent problems of present research.....	15
1.5 Main contents of this dissertation.....	16
Chapter 2 Reliability study of combining component-based software deployment with architectural styles	19
2.1 Introduction.....	19
2.2 Software deployment method and architectural style.....	20
2.2.1 Software deployment method.....	20
2.2.2 Architectural style.....	21
2.3 Adjusting failure probability and influencing factor.....	24
2.4 Reliability analysis of combining component-based software deployment with CS style.....	26
2.5 Reliability analysis of combining component-based software deployment with LCS style.....	28
2.6 Experiments and results analysis.....	31
2.6.1 Experiments and results analysis of CS style.....	31
2.6.2 Experiments and results analysis of LCS style.....	35
2.7 Brief summary.....	38
Chapter 3 Component deployment algorithm considering fuzzy relation	39
3.1 Introduction.....	39
3.2 Related concept and representation method.....	39
3.3 Fuzzy relation matrix and solving method of software components and host nodes.....	42
3.3.1 Fuzzy relation of software components and host nodes.....	43
3.3.2 Solving method of fuzzy relation matrix of software components and host nodes.....	43

nodes.....	44
3.4 Fuzzy greedy deployment algorithm.....	45
3.4.1 Description of fuzzy greedy deployment algorithm.....	45
3.4.2 Analysis of correctness and complexity of fuzzy greedy deployment algorithm.....	49
3.4.3 Experiments and results analysis.....	50
3.5 Brief summary.....	53
Chapter 4 Component redeployment algorithm based on dependence.....	54
4.1 Introduction.....	54
4.2 Dependent relation and dependence among software components.....	54
4.3 Existed deployment algorithms.....	56
4.4 m preferred selection algorithm.....	57
4.4.1 Description of m preferred selection algorithm.....	57
4.4.2 Analysis of correctness and complexity of m preferred selection algorithm.....	60
4.4.3 Instance illustration of m preferred selection algorithm.....	62
4.4.4 Experiments and results analysis.....	66
4.5 Brief summary.....	67
Chapter 5 Reliability study of component replication and fault propagation...69	
5.1 Introduction.....	69
5.2 Replication technology.....	70
5.3 Heuristic greedy replication algorithm.....	71
5.3.1 Structure description and object function.....	71
5.3.2 Description of heuristic replication algorithm.....	73
5.3.3 Analysis of correctness and complexity of heuristic replication algorithm.....	75
5.3.4 Experiments and results analysis.....	77
5.4 Analysis of fault propagation influencing on service reliability.....	83
5.4.1 Experimental inputs.....	83
5.4.2 Analysis of fault propagation by software component failure.....	84
5.4.3 Analysis of fault propagation by host node failure.....	86
5.5 Brief summary.....	88
Chapter 6 Reliability study of software deployment considering time-related component failure probability.....89	
6.1 Introduction.....	89
6.2 NHPP software component model.....	89
6.2.1 NHPP model.....	89
6.2.2 Software component model.....	91
6.2.3 Experiments and results analysis.....	92

6.3 Reliability analysis of integrated architectural style.....	98
6.3.1 Probability distribution in reliability.....	98
6.3.2 Model of integrated architectural style.....	100
6.3.3 Experiments and results analysis.....	102
6.4 Brief summary.....	107
Conclusions.....	108
References.....	110
Papers published in the period of Ph.D. education.....	123
Statement of copyright and Letter of authorization.....	125
Acknowledgements.....	127
Resume.....	128

第1章 绪 论

1.1 课题背景

随着互联网的出现, 计算机软件所面临的环境发生了变化, 由静态、封闭转为动态、开放。其中, “动态”指变化频繁, “开放”指环境及其变化可能超出开发阶段的预期。为了适应这种变化, 软件系统从集中式逐渐演化为分布式^[1,2]。另外, 从 20 世纪 90 年代开始, 基于组件的软件开发迅速发展并逐步成为主流技术。传统的分布式软件是由同构的软件构造块组成, 而新一代分布式软件往往由运行在各种不同硬件平台和操作系统环境中的互操作组件组成, 称为基于组件的分布式软件 (component-based distributed software, CBDS)^[3]。这类软件广泛应用在应急响应、工业自动化、导航、卫生保健等各个领域。软件组件促进人类社会迅速发展的同时, 其失效所造成的损失越来越明显。2002 年 5 月份, 美国国家标准与技术研究院公布一项报告^[4], 该报告指出美国每年由于软件失效所造成的经济损失高达 595 亿美元, 这相当于美国国内生产总值的 0.6%。美国国家标准与技术研究院报告的数据主要来源于金融服务、工业自动化、卫生保健、导航、民用基础设施等行业, 而且若分析安全关键系统中的软件, 造成的损失将更加严重, 甚至会演变成灾难性的事件。九十年代中期, 美国 Therac25 型放射治疗仪因软件安全互锁装置失效产生超剂量辐射, 造成两人死亡, 多人受伤的严重后果。实际上, 由于软件错误导致系统失效, 给人类带来巨大财产损失或人员伤亡的事例不胜枚举^[5]。

随着分布式软件和网络快速发展, 软件部署成为整个软件生命过程中的一个独立阶段^[6]。软件部署是包含一系列相关活动的过程, 这些活动包括软件的配置、软件安装到执行环境中、软件的激活。也包括软件安装后期的一些活动, 主要有软件的检测、更新、重配置、自适应、重部署^[7]。在具有高度动态配置、未知的操作剖面 and 波动执行环境 (如移动、普适、分布式环境等) 下, 软件部署直接影响系统性能。而部署到上述领域 (如应急响应等) 中软件要执行关键任务, 这些软件要满足严格的可靠性要求^[8]。另外, 由于硬件速度提高、容量变大、成本降低、移动自组网和便携式计算设备的出现等等, 使得软件系统的规模和复杂性不断地增加。软件工程研究者和实践者通过利用软件架构 (software architecture, 简称 SA, 有的文献翻译为软件体系结构) 的原则来开发软件系统, 已经成功地处理软件系统复杂性的增长。软件架构为构建软件系统提供了设计层次的建模和指导, 它包括组件 (计算元素)、连接器 (交互元素)

和配置（也称为拓扑结构）^[9]。

基于 SA 的软件部署（后面统称为系统部署）是指软件组件在主机节点上的一种分配。在自然灾害、军事危机、搜索和救援的情况下，人员的部署就是系统部署的典型应用^[10]。图 1-1 为军队部署和战场仿真图（troops deployment and battle simulation, 简称 TDS），1 个总部(headquarter)、4 位指挥官(commander)和 36 位士兵(soldier)。总部与指挥官、指挥官与其他指挥官间可相互联系。设备(PDAs)间的通信用来协调士兵的行动。运行在设备上的软件系统要求能提供大量的服务：请求信息、发送信息、观察到的当前战场地图信息、资源管理信息等等。



图 1-1 TDS 实例
Fig.1-1 TDS instance

TDS 的部署架构包括地图组件（map component）、存储组件（repository component）、策略分析组件（strategyAnalyzerAgent component）、部署组件（deploymentAdvisor component）、仿真组件（simulationAgent component）、UI 组件等等。地图组件维持地形、人员、坦克单元和地雷区域等系统资源。这些资源一直保存在存储组件中。策略分析组件、部署组件、仿真组件用来分析：1）敌友两方军队的部署情况；2）根据敌人军队和障碍，给出我方军队的建议部署；3）根据战场的当前形势，渐进地仿真出战争的结果。UI 组件提供应用程序的用户界面。

已有的研究^[10-13]表明软件系统的部署架构对系统的非功能属性有重要影响，例如系统可靠性、可用性、延迟、容错、异构、可信性等。这种影响在具有生命期长、高度异构和不可预测的普适计算环境更为明显^[14]。下面分析一个普适环境下的应急部署系统（emergency deployment system, 简称 EDS）例子^[15]，如图 1-2 和 1-3 所示^[16]。这两个图描述的都是相同的 EDS 系统或者 EDS 子系统，不同的是部分相同组件部署到不同的主机节点上。图 1-2 为 EDS 系统的子集合的第一种部署架构，图 1-3 为 EDS 系统的子集合的第二种部署架构。这两种部署架构都包括 17 个软件组件和 5 个主机节点。连接组件的实线表示交互路径，连接主机节点的点线表示网络连接。

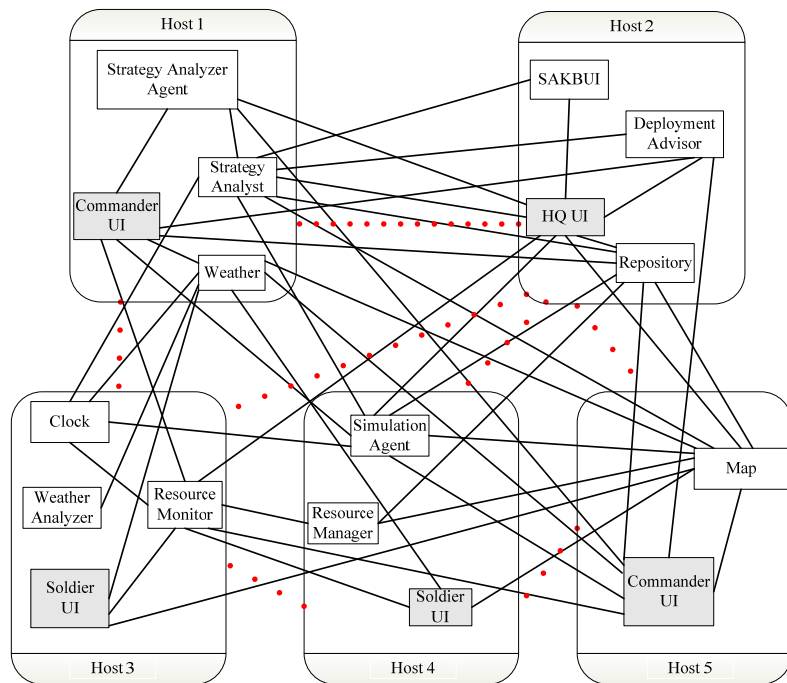


图 1-2 EDS 的第一种部署架构

Fig.1-2 The first deployment architecture of EDS

尽管 EDS（或者 EDS 系统的子集合）的实例对应的两个简图为系统终端用户提供相同的功能（如相同的服务），然而功能完成过程中表现不同的系统效果。例如，图 1-3 中的主机节点 1 和 2 运行的软件组件数目多于图 1-2，意味着主机节点 1 和 2 将消耗更多能量，使电池电量更容易耗光（将影响其它的属性，例如可靠性、可用性等）。而且，主机节点 1 和 2 之间通过网络连接发生的交互次数，在图 1-3 中比图 1-2 中更为频繁，这意味着相关服务的延迟将显著增加，同时系统的耐久性也将进一步降低。

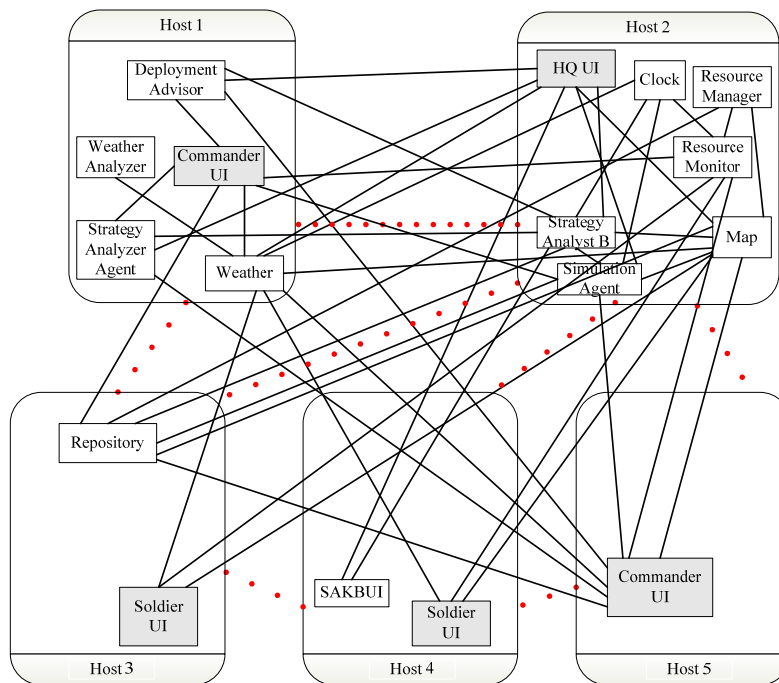


图 1-3 EDS 的第二种部署架构

Fig. 1-3 The second deployment architecture of EDS

这个例子说明确定一个“理想”，甚至只是“较好”的系统部署架构绝非易事。相关原因主要有三个：

第一，对于一个给定系统，其可能的部署架构总数是指数级的，如果系统包括 m 个主机节点和 n 个软件组件，则可能部署架构的数量是 m^n 。这意味着，即使是中等规模的系统，也需要评估大量的选择，从而确定最优部署。

第二，衡量那个部署“更好”的评价方法没有直截了当的答案。例如，对于图 1-2 和图 1-3 所述的两种部署哪一个更好？为了解决评价问题，首先必须定义什么是“好”的部署架构。即使对“好”的部署架构定义达成一致，例如“好”部署的意味着“具有较短的延迟时间，更长的耐久性，更高的可用性，更好的容错，更高的可扩展性等等”，对于某个具体属性的定义，不同的开发者或者不同的组织将可能给出不同的定义，软件工程文献的简略概述也证明了这一点。而且，某些属性的评价指标难以量化，例如，安全性，可信性，异构性等等。

第三，系统可能会受到严重的制约。例如，某些设备可能由于条件不足而无法兼容某些组件（或组件的组合）；由于缺少运行时资源，或者因安全性，某些组件可能需要部署到同一设备上等。在普适环境下，诸如此类的限制可能被特定强调，或者随着系统执行内容的改变而使限制发生改变。因此，即使对于相对较小规模的系统，架构师想要快速给出正确的部署决策也近乎不可能。

由于移动、嵌入式、普适和分布式环境内在的动态性和不可预测性，没有一种特定系统部署架构对系统整个生命周期来说都是最优的。分布在这些环境中的软件逐渐地应用于执行关键任务，并且要满足严格的可靠性要求。然而，系统工作的环境可能比较恶劣，例如有强烈的震动、湿度波动、强电、核辐射的干扰等等因素，PDA 等这样的硬件设备出错的概率必然要增加。在这样的环境下，如何确定“好”的部署将软件组件有效地部署到主机节点上，来满足系统可靠性成为研究热点。本文将具有高系统可靠性的部署架构定义为“好”或者“优化”的部署架构。为了使部署后的系统满足一定的可靠性要求，部署过程要考虑很多方面的因素，如待部署主机节点的可靠性、软件组件的可靠性、软件架构风格、软件组件复制、主机节点的拓扑结构、硬件资源（例如内存）的占用情况等等^{[6]1266}。

本文的工作是从研究普适、移动、嵌入式和分布式环境的移动软件的部署阶段的系统可靠性展开，分析影响部署阶段的系统可靠性的因素，并且给出提高系统可靠性的重部署算法和软件组件的选择性复制算法。本文首先给出不同的软件部署方法和经典的软件架构风格，根据基于组件的软件可靠性评估不足，提出一种将基于组件的软件部署和 CS、LCS 两种软件架构风格结合的方法，该方法分析两者对系统可靠性的影响，并提出概念软件组件调整前失效概率、软件组件调整后失效概率和影响因子；将影响系统部署的因素分为确定性和不确定性，引入模糊性理论将影响因素转换为软件组件间的模糊关系和主机节点间的模糊关系，在分析自适应贪婪算法缺点的基础上，设计出一个模糊贪婪部署算法来重部署系统，从而提高系统可靠性；根据软件组件间的交互频率和交互的重要性，提出软件组件间相关性的概念，在分析 A*算法的优缺点基础上，设计出 m 优先选择算法重部署系统，从而提高系统可靠性；研究软件组件复制对系统可靠性的影响，当系统可用资源有限时，在分析贪婪算法选择将被复制的软件组件的优缺点和可靠性目标优化函数 OF 的基础上，设计启发式的贪婪复制算法来选择将被复制的软件组件，从而得到更高的目标优化值；对相关的软件组件失效和主机节点失效引起的故障传播进行分析；将部署阶段的软件组件的失效行为建模成 NHPP 过程，分析在一段时间内相关参数和软件组件间的交互频率矩阵对系统可靠性的影响；比较重部署方法和软件组件选择性复制方法在提高可靠性上的差别，给出两种方法的适用条件；用连接器的方式来建模集成架构风格，分析集成架构风格的可靠性。

1.2 相关概念

1.2.1 软件架构

(1) 软件架构的定义

软件架构是一个非常重要的概念，然而对软件架构尚没有明确的统一定义，许多软件架构研究者提出了自己的概念和定义，下面列举几个具有代表性的定义：

Perry 和 Wolf 给出软件架构模型为：Software architecture={Elements, Form, Rationale}。也就是，软件架构是有特定形式的架构元素（elements）的集合。架构元素有三种类型：处理元素、数据元素和连接元素。处理元素将数据进行转换，数据元素包括被转换的和被使用的两种信息元素，连接元素用来连接架构的不同部分。架构形式（form）包括加权的属性和关系。“加权”有两个含义：①属性或关系的重要性；②选择的必要性。属性用来约束架构元素选择。关系用来约束不同架构元素之间的交互方式和组织方式。基本原理（rationale）用来描述架构风格、元素和形式的选择方式^[17]。

Fielding 在 Perry 和 Wolf 定义的基础上，给出了一个更详细的软件架构定义：软件架构是根据一些架构元素的配置进行定义，这些架构元素包括组件（component）、连接器（connector）和数据，将架构元素的关系进行某种约束来得到期望的架构属性。其中，组件表示处理元素，连接器表示连接元素^[18]。

Garlan 和 Shaw 关于软件架构的定义为，软件架构为构建软件系统提供了设计层次的建模和指导，它包括组件（计算元素）、连接器（交互元素）和配置（也称为拓扑结构）^[19]。

在 CFRP 模型中，软件架构的模型为：SA={elements, interfaces, connections, connection semantics}。软件系统包括两种类型的元素（elements）：处理元素和数据元素。所有元素都有一个接口（interface）。系统的拓扑结构由多个元素的互连（connections）构成。互连语义（connection semantics）可以为静态互连语义，例如描述数据元素的互连，也可以描述动态连接信息的转换协议，例如管道、过程调用等等^[20,21]。

IEEE 610.12-1990 软件工程的标准词汇中给出的定义为：Architecture = {component, connector, environment, principle}。架构是以组件、组件之间的关系、组件与周围环境的关系为内容的某个系统的基本组织结构，和指导这些内容设计与演化的原理^{[22][21]1228-1237}。

上述软件架构的定义，从不同的角度来理解和分析软件架构，这些定义包含一些共同的实体，例如组件、组件间的交互关系、连接元素或者连接器等。本文采用 Garlan 和 Shaw 给出的软件架构定义。

（2）软件架构的研究现状

20 世纪 90 年代，人们对软件架构进行了广泛的研究，例如卡内基·梅隆大学的 Shaw 和 Garlan、贝尔实验室的 Perry、南加州大学的 Boehm、斯坦福大学的 Luckham 等人都进行了软件架构的研究和相应的学科建设^{[23]18-24}。早期软件架构的研究主要关注软件生命周期的设计阶段，如何利用软件架构分析和解决前期软件系统的设计问题，主要的研究点包括架构描述语言、架构结构的验证、架构结构的分析和架构结构的评估方法等等。此后，由于越来越多的不同背景的研究者的参与，软件架构研究不再局限于软件设计阶段，开始扩展到软件生命周期的各个阶段，例如在系统设计前，将架构引入到需求分析中；在系统设计后期，研究如何利用软件架构来支持系统实现、系统组装、系统部署；在系统开发阶段之后，如何利用软件架构来支持系统的维护、系统的演化和复用等等^{[6]1258}。

1.2.2 软件架构风格

（1）软件架构风格的定义

Garlan 和 Shaw^[18,24]、Garlan 等人^[18,25]用各类组件之间的交互模式对风格进行定义。一种架构风格决定了在这个风格实例中可以使用的连接器与组件的词汇表，以及一组描述如何将它们组合到一起的约束^[18,26]。在这个定义的基础上，Fielding 给出一个非常相似的架构风格定义，架构风格是限制架构元素的功能和角色、以及可以存在的架构元素之间关系的一组架构约束^{[18]13}。

此外，在一些情况下，架构风格的描述也叫做架构模式，架构模式是对设计模式的扩展，描述了架构基本的组织化方案，可以作为具体的软件架构的模板。在实际应用中，风格和模式常常混用。从目的上看，风格和模式都是为了把设计和决策记录下来；从使用上看，两者也大都使用了类似的技术来记录和阐明设计决策。一般而言，在软件架构的研究领域，风格和模式不进行区分，统称为架构风格^{[6]1262}。

（2）软件架构风格的混合

架构风格强调的是软件架构的不同方面，同一个软件系统的架构可以同时体现多种风格，如一个用 J2EE 开发的应用程序，可能既是“三层 C/S 风格”，又是“面向对象风格”。如果系统使用 Spring 框架实现的，那么可能还包含“模型-视图-控制器风格”。当一个架构采用多种风格的时候，称为软件架构风格的混合。

当一个软件架构混合使用了多个风格的时候，可能会带来软件架构风格失配的问题。这个问题是指多个软件架构风格之间有可能包含了相互冲突的规

则，它们彼此是无法共存的。当这样的软件架构风格共同作用时，它们能确定的架构集合为空^{[27]31-41}。

(3) 软件架构风格的集成方法

在架构设计过程中，很多情况下需要多种不同架构风格的组合使用。通常可以采用如下的几种方式^{[27]41}。

① 通过层次关系。即使用某种架构风格的组件可以拥有内部结构，这个内部结构采用另外一种架构风格；

② 通过连接器。即允许一个组件使用不同架构风格的连接器，这样这个组件可以和其他不同风格的组件通过连接器来进行交互；

③ 使用完全不同的架构风格来细化架构描述的一个层次。

这里所说的多个风格的集成同前面的风格的混合不同。风格的混合是将多个风格都作用到整个软件架构上。而风格的集成是分别把风格应用到软件的不同组成部分上，再把这些部分通过上面几种方式组合起来。所以，风格的混合有可能导致风格失配问题，而风格的集成不会导致这样的问题。

1.2.3 软件组件

(1) 软件组件的定义

软件组件(software component, 有的文献译为软构件)是一个相当重要的概念。但是，到目前为止仍没有统一明确的组件定义，这里列举几个比较有影响力的组件定义。

2000 年，卡内基·梅隆大学的软件工程研究所发表一份技术报告，该报告指出组件为一个不透明的功能实体，可被第三方组装，并且符合一个组件模型^[28]。

国际上第一部软件组件专著中指出一个组件是一个组装单元，组件具有约定式规范的接口和明确的依赖环境。组件能够被第三方组装，并且可被独立地部署^[29]。

Fielding 将组件定义为，组件是内部状态和软件指令的一个抽象单元，通过组件接口提供数据转换^{[18]11}。

Brown 将组件描述为“组件是一个非平凡的、几乎独立的和可替换的系统组成部分，它在一个定义完善的体系结构语境下完成一个清晰的功能，组件统一于一且提供了一组接口的物理实现”^[30]。

OMG 对组件的定义更通俗、详细一些：“一个物理的、可替代的部分，它包括具体实现，并提供一组接口。一个组件表示系统实现的一个不可见物理单

元，包括软件代码（源代码、二进制码或者可执行代码）或者等价物（如脚本）或者命令文件等”^[31]。

杨芙清院士认为“组件是软件的构成元素，具有一定的功能和结构，并符合一定的标准，可以完成一个或多个特定的服务。组件隐藏了具体的实现，通过接口对外提供服务”。

《计算机百科全书》中将组件定义为，软件组件是软件系统中具有相对独立功能、可被明确辨识、接口由契约指定、和语言具有明显的依赖关系、可被独立地部署、可被组装的软件实体。

这里采用《计算机百科全书》里关于组件的定义，其中，“具有相对独立功能，可被明确辨识”描述了组件在技术上的内容；“接口由契约指定”的含意是，契约描述了如何使用和管理组件的信息，包括提供给客户程序的信息、提供给组件运行环境的信息，以及可能的、提供给组件库管理系统的静态管理信息等，而其主要内容就是组件的各种接口；“可独立部署、可组装”确保了对组件的使用不依赖组件的开发者所使用的工具和知识，换言之，可以由多个独立的来源获得组件以及组装软件系统^{[27]10}。

（2）软件组件的分类

从移动性的角度，软件组件可分为不可移动（untransferable）和可移动（transferable）。不可移动组件是指在系统运行期间无法移动的组件。可移动组件指系统运行期间，组件可以从一个主机节点经过网络迁移至另一主机节点^{[32]29}。

在软件系统的开发过程中，软件组件可分为两种类型：一类为在实际系统运行时，不与其它组件发生直接关系，并且功能上比较通用的软件组件；另一类为在实际系统运行时，与其它组件存在直接关系的软件组件，并且是从软件系统中抽象出来的功能上比较专用的软件组件^{[33]6}。

（3）软件组件模型

组件模型是对现实组件世界的抽象，是对组件本质特征的描述。组件模型最重要的贡献之一就是把应用开发和部署相分离，主要解决如何开发组件以及如何使用组件构造软件系统的问题，是基于组件的软件开发的核心，是组件的本质特征及组件间关系的抽象描述^[34]。目前，很难建立统一的组件模型来满足不同方面的需求，已有的大量组件模型分为如下三类^{[27]10-11}。

与组件部署/实现相关的模型：该模型能够帮助人们决定如何用某种程序设计语言或者可执行单元的形式来实现组件，被称为基础设施模型。组件最终必须被实现成某种直接可用的形式，因此，这类模型具有非常重要的地位。该模型也规定了组件的开发者 and 使用者都必须遵循的规定和标准。比如，组件的

定位方式、组件的管理方式、组件的部署方式、组件的组装方式、组件的访问方式等等。

与组件规约/组装相关的模型：这类模型主要描述组件的功能规约，也就是组件对外提供何种功能、组件需要外界为它提供何种功能、组件被用于何种语境、以及组件被如何定制等。该模型描述组件的功能和行为规约，并且通过配置这些规约来刻画系统。

与组件分类/描述相关的模型：此模型以综合的方式描述组件，用于管理大量的静态组件，使得组件易于被用户所理解，易于在库中被高效地分类、存储和检索。

1.2.4 可靠性

(1) 可靠性的基本概念

可靠性(reliability, 有的文献翻译为可靠度)是指在给定的时间内、给定的条件下, 系统或者组件完成规定功能的能力。这里的组件可以是硬件或者软件, 组件也可被细分为更小的组件^{[22]18-63}。

错误(error)是指, ① 不正确的结果; ② 产生不正确结果的人的行为; ③ 不正确的步骤、过程或者数据定义; ④ 可计算的、可观察或可测量的值, 与真实的、特定的或者理论值的差别^{[22]31}。

缺陷(defect), 一个产品的异常, 例如 ① 在生命周期的早期发现的遗漏或不完善; ② 软件中包含的故障^[35-36]。

故障(fault)是指, ① 硬件设备或者组件中的一个缺陷; ② 计算机程序中不正确的步骤、过程或者数据定义。这个概念主要用在容错研究中^{[22]32}。

失效(failure)是指, 对于特定的任务, 使系统或者组件不能完成规定功能的事件^{[22]32}。

(2) 可靠性模型的分类

可靠性模型的种类繁多, 用途各异, 在不同的应用背景下, 能有不同的分类。这里从不同的角度对可靠性模型进行如下分类^{[37]31}, 如图 1-4 所示。之后对一部分可靠性模型进行简要介绍^{[38]6}。

黑盒方法将整个系统看作一个单一的个体并且不考虑其内部结构。白盒的方式根据软件系统的体系结构对各个组件的可靠性进行综合计算, 以得到系统的可靠性^{[39]453}。

单故障可靠性模型指系统和系统功能单元的故障模式仅有一种的情况, 或

者为了使问题简化,假设其仅有一种;而对于多故障可靠性模型,描述系统和系统功能单元的故障模式至少为一种^{[37]31}。

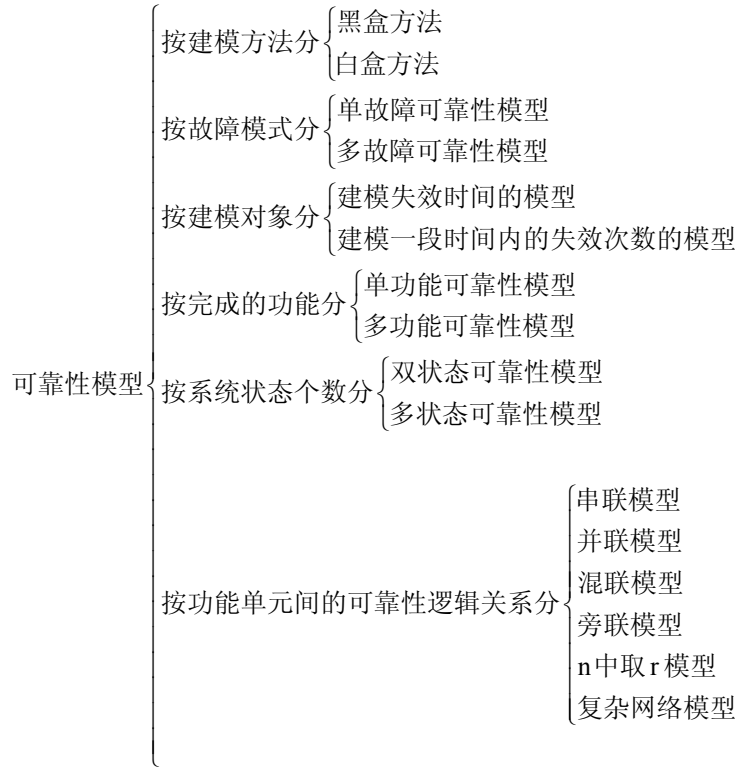


图 1-4 可靠性模型分类

Fig. 1-4 Classification of reliability models

单功能可靠性模型是指,在规定的任务剖面下,系统只能完成一个功能;多功能可靠性模型,是指在规定的任务剖面下,系统能够完成一个以上功能^{[37]31}。

在串联模型中,系统的任意一个部件的失效都将引起系统失效;在并联模型中,系统由 n 个部件并联组成,只有当 n 个部件全部失效时,系统才会失效^{[40]38-84};在混联模型中,既包括串联部分,又包括并联部分^{[41]48}。在旁联模型中,系统有 m 个部件,仅有一个部件进行工作,如果工作部件出现故障,通过故障监测,并用转换装置接到备份部件继续工作,当所有部件都故障的时候,系统才发生故障;在 n 中取 r 模型中,系统由 n 个部件构成,当 n 中至少有 r ($1 \leq r \leq n$) 个部件正常工作的时候,系统才能够正常工作^{[42]25-26}。

(3) 移动软件系统的可靠性评估和提高所面临的挑战

评估和提高移动系统的可靠性是一个具有挑战性的多层面问题。当前研究领域主要面临 5 个关键的挑战^[43]。

上下文对可靠性的影响。在移动的、分布式的和普适的系统中，软件失效的原因有很多，例如内部的软件设计和具体实施缺陷、外部的硬件和网络问题。对于前者，编码缺陷引起的软件失效很大程度依赖于软件外部的运行特性，这些外部系统特性被称为上下文^[44]。例如，某软件缺陷只有当网络吞吐量减少到一定的阈值时才可能会导致失效，其原因可能是通信缓冲区的溢出问题。存在多种多样的上下文属性可能影响软件系统的可靠性，例如硬件平台的具体变化（电池电量耗尽，内存可用量的变化），或是网络状态波动（网络中断，带宽的变化），或者意料之外的使用情况、操作时间影响等。

动态性对可靠性的影响。为了应对上下文环境的变化，移动软件系统往往被设计成具有一定适应性。软件为了适应系统的动态配置而改变其行为，这又会影响移动软件的可靠性。例如，考虑一个如图 1-5 a) 中的某假设的软件系

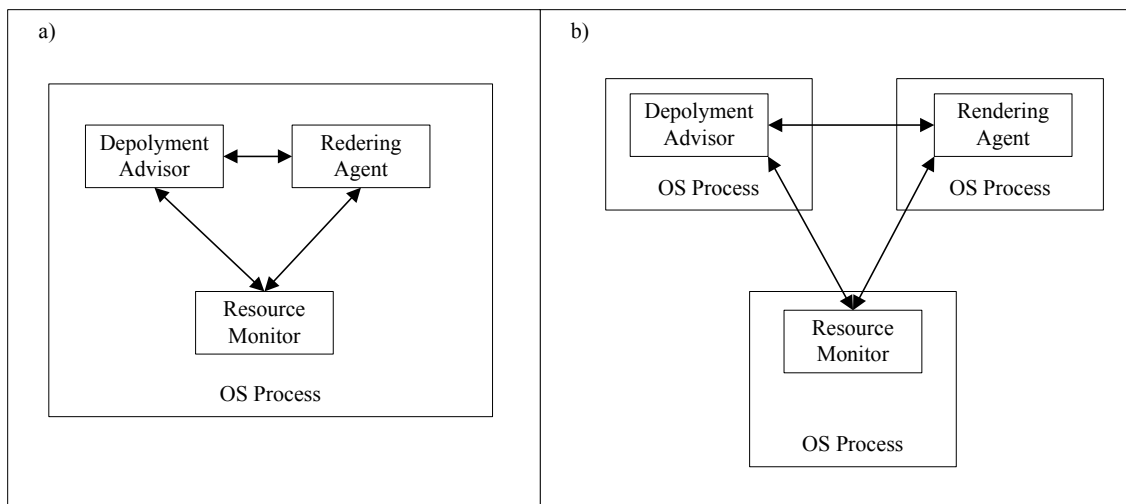


图 1-5 假设系统的两种配置

Fig. 1-5 Two configurations for a hypothetical system

a) 更有效但是不可靠性的配置

b) 效率低但是更可靠性的配置

a) A more efficient but less reliable configuration

b) A less efficient but more reliable configuration

统，其包含三个运行于同一操作系统进程（例如，JVM 进程）中的软件组件。这样的配置可能是高效的，因为这些软件组件能够在同一进程中共享资源（如全局变量）。另一方面，如果该软件组件中的任意一个出现故障，整个系统很容易失效，失效组件的崩溃过程中可以很容易地将故障传播到其它组件。图 1-5 b) 显示了一个可替代情况的系统配置，同样存在的问题是其效率较低，但对失效具有弹性。分配软件组件到操作系统的进程就是一个至关重要的实例，直接影响一个软件系统的可靠性。因此，在移动计算领域，架构重构频繁

发生,评估对其可靠性的影响并确定最可靠的(即最优)配置的能力是极其重要的。

预测可靠性的难度。确定某软件系统最优可靠性配置的能力依赖于能否准确地估计软件系统在日后运作的可靠性。大多数现有的可靠性分析方法^[45-47]主要依靠软件监测和假设软件系统过去的可靠性也会反映出其未来的可靠性。虽然这种假设对于传统的桌面应用这种相对稳定的系统可能是合理的,它却不适用于移动系统的领域。例如,事实上即使移动软件系统在过去数小时没有崩溃发生,并不一定意味着它在接下来的几个小时也不会崩溃。这是由于这类特别系统不断变化的上下文和架构造成的。

细粒度的可靠性预测。为了能够估计某架构配置的可靠性,就需要能评估其组成部分的可靠性。例如,在图 1-5 中描述的情况,只有当有能力先评估其软件组件的可靠性,才可以确定最优的系统配置。如果组件是非常可靠的,图 1-5 a) 是可取的更有效的配置。但如果组件是很不可靠的,图 1-5 b) 中对失效更具弹性的配置就是更优的选择。

可扩展的在线分析。大多数传统的基于随机过程(例如,马尔可夫链、贝叶斯网络)的可靠性评估方法都具有很高的计算代价^[45-47]。系统组件级别而不是系统整体级别的细粒度分析,将进一步加剧了计算代价的问题。在移动计算领域,分析的可扩展性非常重要,需要在系统运行时在线进行分析。

1.3 基于组件的分布式软件可靠性研究现状与进展

分布式软件动态配置早在 70 年代就被提出,随着时间的推移,动态配置技术经历了从早期面向过程的软件,到面向对象的软件,再到基于组件的软件的发展变迁,每一次的变迁都使得软件耦合度不断下降,软件连接方式更加灵活,软件构造特点更加明晰^[48]。

在基于组件的软件可靠性模型中,主要有三类模型:基于状态的模型、基于路径的模型和可加模型。基于状态的方法把应用的架构模型映射成状态空间模型。状态空间模型可以表示为离散时间马尔可夫链(DTMC)^[49,50],其中用不可约的 DTMC 建模连续运行的组件软件应用,用吸收的 DTMC 建模终止应用。如果已知组件的执行时间分布,还可以用连续时间马尔可夫链(CTMC)^[51]或半马尔可夫过程(SMP)^[52]建模应用的体系结构。在基于状态的模型,组件的失效率可以为常量^[53-54],也可以依赖于时间的变量^[55]。基于状态的模型有两种求解方法:组合方法和等级方法。组合方法能计算得到应用可靠性的准确值,等级方法计算得到的是应用可靠性估计的近似值^[56]。基于路径的方法通过计算软件所有可能执行路径的可靠性,然后加以综合来得到整个软件的

可靠性。路径的枚举可以通过算法实现^[57]，也可以通过实验方式实现^[58]，或者通过仿真的方法实现^[59-60]。每条路径的可靠性等于路径上所有组件可靠性的乘积。组件软件应用中所有路径的可靠性按照它们的执行频率加权平均就可以得到整个应用的可靠性评估。如果程序中存在循环而使应用具有无数条执行路径时，使用基于路径的方法进行可靠性估计只能得到近似值。可加模型^[61-65]假设用 NHPP 类软件可靠性增长模型建模组件失效行为，系统的失效强度等于组件失效强度的总和。与基于状态和基于路径的模型不同，可加模型针对的是组件软件的测试和运行阶段进行可靠性分析，得到的可靠性度量不是一个静态的单一的估计值，而是一系列随时间变化的预测值。但可加模型没有考虑系统执行过程中组件的使用情况。

在基于组件的软件可靠性模型基础上，Pant^[66]提出一个预测分布式软件软件可靠性模型。该模型用软件可靠性增长模型评估系统测试期间移除的故障数，并用与测试时间相关的函数描述分布式软件的失效密度。Luo^[67]提出一个基于场景的分布式软件系统可靠性模型，用序列图描述组件间的交互，利用通信图得到场景中的交互参数。Kumar^[68]以分布式程序为基础建立相应的模型，并给出一个评估分布式程序可靠性的算法。Wang^[69]根据组件的可靠性、操作剖面和软件架构提出了一个架构层次的软件可靠性评估模型，该模型分析了调用和返回风格、容错风格、并行风格的软件可靠性。在 Kramer^[70]和 Wermrlinger^[71]研究的基础上，Cao^[72]提出一种通过动态重配置来支持基于组件的分布式软件的容错的架构层方法。通过容错来满足分布式软件的规定的可靠性要求。有些分布式软件可靠性评估方法考虑了实时性，在 Drummond^[73]、Kamenoff^[74]、Sanders^[75]和 Weideman^[76]等人对分布式软件的实时性评估基础上，Jin^[77]强调分布式软件可靠性的重要性，并提出一种分析具有可靠性约束的分布式软件实时性的方法。韩^[78]将精简和广度优先两种方法有机结合，指出一种具有实时特征的分布式软件可靠性评估方法。

随着基于组件的分布式软件的飞速发展，研究者对软件的部署阶段重视程度越来越高，软件部署将会影响可靠性、服务延迟和可用性等系统性能^[16]。Andersson^[79]提出一种普适环境下的软件部署模型 JDrums(java distributed run-time updating management system)，该模型支持基于组件的软件开发、软件动态安装与激活、动态更新系统等。Vo^[80]提出软件部署框架 OTA-PSD，该框架支持软件部署的推模式和拉模式，但该方法没有对系统性能进行分析。在结构和性能范型的基础上，Hughes^[81]提出可变粒度的静态建模框架，用来分析软件属性和影响系统性能的部署。Medvidovic^[16]举例讨论部署架构对系统延迟的影响。Mikic-Rakic^[13]分别用精确算法、随机算法和 Avala 算法来确定高可用

性的系统部署架构，Avala 算法将软件组件和主机节点标志成不同的级别，根据标志的级别将软件组件部署到主机节点上。Mikic-Rakic^[82]提供工具 DeSi 用于基于 SA 的可视化部署，并评估部署后的系统可用性。DeSi 提供了规约、操作、可视化、评估分布式系统部署阶段架构的能力。Popescu^[83]根据目标优化函数 OF，用三个启发式算法和贪婪算法来选择将被复制的软件组件，从而提高系统可靠性。Malek^[84]提出一套通用的多目标优化算法实现软件部署，算法考虑多种影响服务质量的因素，但是需要借助其他工具的支持。Boone^[85-86]利用 UML 模型对系统建模，以提高系统可靠性为目标，提出使用混合整数规划解决组件部署优化问题的策略。

1.4 当前研究工作存在的问题

(1) 已有的基于组件的软件可靠性评估方法^[45-47,53,57,87]，适合传统的桌面软件系统，这类软件在执行过程中相对稳定和静止的^[45-47]。第一，这些软件可靠性评估方法在分析软件组件失效原因时，没有考虑软件组件运行的硬件平台对软件组件失效的影响。第二，在移动、普适和分布式环境中，系统具有高度动态配置、未知的操作剖面 and 波动的执行环境，分布在这些环境中的软件通常具有移动性和自适应性，并且软件执行内容和执行过程在动态地变化，因此，传统的软件可靠性评估方法不适合用来评估动态环境下的移动软件的可靠性，需要新的原则、模型和工具^[43]。第三，已有的可靠性评估方法，主要分析设计阶段和测试阶段的可靠性。对于系统部署阶段的研究，主要有评估系统可用性^[6,13,82-84,88]、延迟^[11]、远程通信带宽^[10]、能量消耗^[89]和通过软件组件复制来提高得到的可靠性优化值^[83]，对部署阶段的系统可靠性研究很少。

(2) Andersson^[79]提出一种普适环境下的软件部署模型 JDrums，该模型支持基于组件的软件开发、软件动态安装与激活、动态更新系统等。Vo^[80]提出软件部署框架 OTA-PSD，该框架支持软件部署的推模式和拉模式，但该方法没有考虑架构风格和性能分析。Wang^[69]分析了调用和返回风格、容错风格、并行风格的软件可靠性，但该方法没有考虑软件部署的影响。也就是，在已有的基于组件的可靠性研究中，缺乏将软件部署方式和架构风格有机地结合起来，分析部署方法、不同软件架构风格和同一软件架构风格中不同的软件组件类型对可靠性的影响。

(3) 在移动、普适和分布式环境中，系统具有高度地动态配置和波动的执行环境。这种内在的动态性和不可预测性，使得没有一种特定的部署架构对整个系统的执行过程来说都是最优的，因此需要系统重部署来提高系统性能。文献[13]利用精确算法、随机算法和 Avala 算法来找到提高系统可用性的部署

架构。然而，精确算法的复杂度高，如果系统有 n 个软件组件和 m 个主机节点，则时间复杂度为 $O(m^n)$ ；随机算法确定的部署架构的可用性值可能很高，也可能很低，不够稳定；Avala 算法将软件组件和主机节点标志成不同的级别，根据标志的级别将软件组件部署到主机节点上，但 Avala 算法经常找到的是较高可用性的部署架构，而不是最优的。并且，缺乏提高系统可靠性的重部署算法。

(4) 当系统可靠性值低于某一规定值时，可通过重部署来提高系统可靠性，也可通过软件组件复制来提高系统可靠性。由于软件组件复制需要消耗额外的系统资源，为了能够在系统可用资源和可靠性之间找到较好的折中策略，Malek 设计了目标优化函数 OF，文献[83]根据目标优化函数 OF，利用贪婪算法来选择出将要被复制的软件组件集合。但贪婪算法常常确定的是局部优化解，也就是找到的是近似最优的将被复制的软件组件集合。

(5) 文献[39]分析了软件组件可靠性和软件组件运行次数在软件可靠性中的重要性，然而，影响系统部署或重部署的因素有很多，如软件架构风格、软件组件的失效过程的建模、软件组件间的交互频率、主机节点的可靠性、软件组件副本等等。目前，缺少对部署阶段的可靠性研究中其它影响因素的分析，以及影响因素之间关系的分析，例如软件组件间交互频率矩阵与软件架构风格间的关系。

1.5 本文的主要研究内容

本文主要研究分布在移动、普适和分布式环境下的移动软件的部署阶段的可靠性提高方法和影响因素。分析不同的软件部署方法和经典的架构风格，提出一种将基于组件的软件部署与 CS 和 LCS 架构风格的有机地结合的方法；针对不同部署架构对可靠性的影响，设计了模糊贪婪部署算法和 m 优先选择部署算法；根据系统可用资源，设计了启发式的贪婪复制算法来选择将被复制的软件组件；部署阶段的软件组件的失效行为建模成 NHPP 过程，分析在一段时间内相关参数、交互频率矩阵对可靠性影响，并分析集成架构风格的可靠性。本文的实验环境是 Intel 的 Core2 Quad Q9400 处理器、主频 2.66GHz、3GB DDR2、Windows XP 系统下的 Java SDK 1.5，并用 Matlab 和 Excel 仿真制图。本文的组织结构图如图 1-6 所示。

全文共分六章。

第 1 章介绍论文背景及相关研究。首先介绍软件架构、软件架构风格、软件组件和可靠性相关概念，给出软件架构的研究现状、架构风格的混合、架构风格集成的方法、软件组件的分类、软件组件模型、可靠性模型分类和移动软件系统可靠性评估和提高所面临的挑战。其次，介绍基于组件的分布式软件可

可靠性研究现状与进展。再次，给出当前研究存在的主要问题。最后概述本文的主要研究内容。

第2章介绍软件部署方法和软件架构风格。提出一种将基于组件的软件部署和CS、LCS两种软件架构风格结合起来的方法。该方法分析部署方法和软件架构风格对可靠性的影响，提出软件组件的调整前失效概率、调整后失效概率和影响因子的概念，并给出影响因子的求解方法。将基于组件的软件部署方法和CS架构风格有机地结合起来，分析并给出求解服务组件的调整后失效概率和客户组件的调整后失效概率的方法。将基于组件的软件部署方法和LCS架构风格有机地结合起来，分析并给出求解LCS架构风格下的服务组件的调整后失效概率、中间组件的调整后失效概率和客户组件的调整后失效概率的方法。之后，给出包含4个主机节点和14个软件组件的实验，分析不同的软件架构风格对系统失效的影响；分析软件组件在主机节点上不同部署对系统失效的影响；分析同一种软件架构中不同类型的软件组件失效对系统失效的影响。

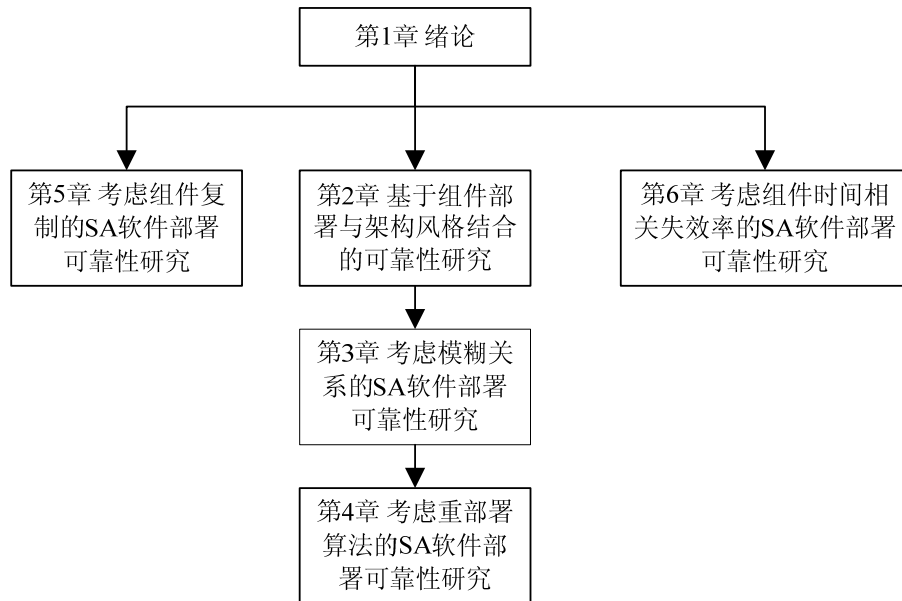


图 1-6 本论文的组织

Fig.1-6 Organization of this dissertation

第3章分析影响系统部署的多种因素，并将其分为确定性因素和不确定性因素。对于不确定性因素，很难用精确性理论来描述，因此引入模糊理论来描述影响部署的因素。将影响因素进一步细分，分成主机节点间的影响因素和软件组件间的影响因素，引入软件组件间模糊关系和主机节点间模糊关系的概念，并给出求解软件组件间模糊矩阵和主机节点间模糊矩阵的方法。根据模糊矩阵，设计模糊贪婪部署算法将软件组件有效地部署到主机节点。对模糊贪婪

部署算法进行正确性和时间复杂度分析。最后，给出 4 个主机节点和 14 个软件组件的实验及结果分析。

第 4 章首先根据软件组件间的交互次数以及交互的重要性，给出软件组件间相关性的概念。之后，分析已有的部署算法包括精确算法、随机算法和 Avala 算法的优点和缺点。精确算法虽然能找到最优的部署架构，但需要的计算时间太昂贵，时间复杂度为 $O(m^n)$ 。在分析 A*算法的优缺点的基础上，设计了 m 优先选择算法，对 m 优先选择算法进行正确性和时间复杂度分析，并给出 3 个主机和 6 个软件组件的 m 优先选择算法的详细执行过程。之后，给出 4 个不同软件组件数和主机节点数的实验及实验结果分析。

第 5 章 介绍不同的复制技术及其相关比较。给出基于组件的软件部署的结构描述，在分析贪婪算法利用目标函数 OF 选择将被复制的软件组件的优点的基础上，设计了启发式的贪婪复制算法。该算法根据 OB 和 OM 选择出将被复制的软件组件。给出包含 12 个软件组件和 4 个主机节点的实验，在系统可用资源相同的情况下，比较启发式的贪婪复制算法和贪婪算法实际消耗的系统资源和得到可靠性优化值。软件组件复制能提高系统可靠性，如果软件组件有副本，即使它失效，那么失效的软件组件的副本仍能继续提供服务，系统仍能正常工作。但是，失效的软件组件会将故障传播给与其直接交互的其它软件组件。对软件组件失效和主机节点失效引起的故障传播进行分析。

第 6 章 分析软件组件的失效率是时间相关的函数，并根据 Xie 模型的假设，将部署阶段的软件组件的失效行为建模成 NHPP 过程。根据 Yacoub 提出 ALRRA 方法的假设，将软件组件的失效率设为与软件组件执行频率相关的函数。之后，给出 12 个软件组件和 4 个主机节点的实验，分析在一段时间内，相关参数、软件组件复制对可靠性的影响；比较软件组件复制和系统重部署在提高系统可靠性方面的差别，并给出两种方法的适用条件；分析软件组件间的交互频率矩阵的类型和架构风格的关系，并给出不同类型的软件组件间的交互频率矩阵对系统可靠性的影响。采用连接器的方式对集成架构风格进行建模，并分析软件组件在不同时间单元的执行情况和软件组件的失效原因，将软件组件的可靠性建模成分段函数。之后，给出实例分析，包括 5 个主机节点、17 个软件组件和 3 种软件架构风格。将不同类型的软件组件用合适的概率分布函数来建模。分析特定时间下的软件组件可靠性。分析在一段时间内，软件组件和集成架构风格的可靠性变化情况。

第2章 基于组件部署与架构风格结合的可靠性研究

2.1 引言

在软件可靠性评估中，基于状态的方法利用控制流程图表示软件系统的内部结构，基于路径的方法计算所有可能执行路径的可靠性，可加模型是假设用NHPP过程来建模软件组件的失效行为，系统的失效强度等于组件的失效强度之和。

首先，这些方法都没有考虑软件组件运行的主机节点对软件组件失效率的影响。对于软件组件失效的原因，可以分两种情况来讨论。第一种情况，软件组件所部署的主机节点完全可靠，软件组件可以为其它主机节点上的软件组件提供服务，或者向其它主机节点上的软件组件请求服务。软件组件失效的可能原因有软件组件内部执行代码出错、软件组件间的状态转移过程中出错等等。基于状态、基于路径和可加模型的方法考虑的是这种情况下的软件组件失效。第二种情况为如果主机节点失效，那么部署在失效的主机节点上的软件组件将无法为其它主机节点上的软件组件提供服务，或者向其发送服务请求。即部署在失效主机节点上的软件组件在这样给定的时间内和给定的条件下，不能完成规定的功能，软件组件也失效。而上述三类软件可靠性评估方法没有考虑这种情况下的软件组件失效。

其次，系统中包括许多软件组件和主机节点，这些主机节点的可靠性不可能完全相同，那么就涉及如何将这软件组件有效地部署到主机节点上。因为主机节点的可靠性对软件组件的失效有重要影响，那么不同的部署结果也将影响软件组件的失效。

再次，在可加模型中，假设系统的失效强度等于组件的失效强度之和。考虑这样的情况，假设系统有3个软件组件 C_1 、 C_2 和 C_3 。 C_1 和 C_2 要向 C_3 发送服务请求， C_3 给 C_1 和 C_2 发送响应信息，并且 C_2 是 C_1 的组件副本。如果 C_1 或者 C_2 失效，系统仍可正常工作。如果 C_3 失效，则整个系统失效。这里 C_1 和 C_2 属于同一类型的软件组件， C_3 属于另一类型的软件组件。很容易看出，不同类型的软件组件的失效对系统失效的影响是不同的，但在可加模型中却没有考虑软件组件类型对系统失效的影响。而软件架构风格正是对软件组件间这种约束关系的一种描述。

Andersson^[79]提出一种普适环境的软件部署模型 JDrums，该模型支持基于组件的软件开发、软件动态安装与激活、动态更新系统等。Vo^[80]提出软件部署

框架 OTA-PSD, 该框架支持软件部署的推模式和拉模式, 但该方法没有对系统性能进行分析。Wang^[69]根据组件的可靠性、操作剖面 and 软件架构提出了一个架构层次的软件可靠性评估模型, 该模型分析了调用和返回风格、容错风格、并行风格的软件可靠性。但上述方法缺乏将软件部署方法和架构风格有机地结合起来分析可靠性。

针对以上的问题和相关方法的不足, 本章提出一种将软件部署方法和架构风格有机地结合起来的方法, 该方法分析两者对系统可靠性的影响。组件部署方面的研究正受到越来越多的关注, 组件的部署十分重要, 并且 CS 风格是基于网络应用的架构风格中最常见的, 因此本章重点研究基于组件的软件部署方法与两层 CS、三层 LCS 两种架构风格。

2.2 软件部署方法和架构风格

2.2.1 软件部署方法

(1) 即时软件部署

即时软件部署 (just-in-time software deployment, 简称 JTSD) 是在用户访问软件组件的最近时刻进行软件安装与激活。例如 Java Web Star Technology^[90]和 Smart Deployment Infrastructure^[80,91]。这种部署方法节约设备内存, 容易适应用户偏好、设备容量和用户所需的内容。由于软件不是存储在本地设备上, 可不必在移动设备上更新软件^[92]。然而, 如果频繁请求该软件, 将造成较长的时间消耗和较高网络流量。

(2) 基于组件的软件部署

组件技术的出现提高了软件开发的效率、灵活性、代码复用率和软件质量, 降低了软件维护的难度^{[85]30}。Fjellheim、Taconet 和 Kwan 在移动设备上开发了基于组件的软件部署 (component-based software deployment, 简称 CBSWD) 架构^[91,93-95]。这种架构采用一种基于当前内容的功能自适应技术来实现。这种功能自适应技术能改变软件完成功能的执行方式。基于组件的软件部署方法支持对用户内容和设备能力的自适应^{[80]634}。

(3) 基于中间件的软件部署

组件化中间件技术通过更高层次的抽象定义弥补了面向对象中间件技术的一些缺陷, 通过组件模型定义了组件的虚拟边界、发布方法以及组件与其它组件或者客户端进行交互的方式。工业界主要使用下列三种组件模型进行组件化中间件的开发: J2EE/EJB、CORBA/CCM 和 COM/DCOM^{[96]3-9}。

基于中间件的软件部署 (middle-based software deployment, 简称 MBSD)

方法都采用中间件的架构。这种架构通常包括两个模块，一个服务器端模块用以处理用户请求，寻找匹配的软件并且将其发送到相应的设备，另一个是客户端模块，负责下载、安装以及更新软件。

（4）上下文相关软件部署

上下文信息是指位置、动作、历史记录等信息，反映与对象相关的一些属性信息。上下文感知是系统自动地对上下文信息、上下文信息的变化以及上下文信息的历史进行感知和应用，并根据其调整自身的行为^{[96]2}，适合普适环境下上下文感知的组件适配与组件部署。上下文相关的软件部署（**context-based software deployment**，简称 **CTBSD**）利用内容来决定部署到移动设备上的功能或者组件^[97,98]。

（5）拉模式软件部署

在拉模式的软件分发方式中，远程主机按照一定的时间间隔向部署服务器询问是否有某个软件的更新版本，如果有软件的更新版本，请求部署服务器把该软件包传给自己，并且根据部署服务器的指令执行相应操作^{[99]2-3}。拉模式软件部署（**pull model of software deployment**，简称 **PLMSD**）允许用户自己识别所需软件，适合用户指定方式的软件部署^[100]。

（6）推模式软件部署

推模式的分发方式为部署服务器将软件包推给各个远程主机^{[99]3}。推模式软件部署（**push model of software deployment**，简称 **PHMSD**）也称为提供者发动的软件部署，由提供者初始化一个部署过程，也就是在用户没有发现新软件或者软件更新时，新软件或者软件的更新已经被推到了移动设备上。这个模型广泛应用于台式机上的软件部署^[101]。

2.2.2 架构风格

当前，已存在若干被广泛接受的架构风格。下文将介绍几种经典的架构风格，如数据流风格、分层风格、复制风格、点对点风格、移动代码风格等。

（1）数据流风格

数据流风格主要包括管道和过滤器风格（**pipe and filter**，简称 **PF**）、统一管道和过滤器风格（**uniform pipe and filter**，简称 **UPF**）。在 **PF** 风格中，每个过滤器必须是一个独立实体，过滤器之间无需共享状态。过滤器不需要知道与其输入管道和输出管道所连接的其它过滤器的存在。过滤器从输入端读取数据流，然后在输出端产生数据流。过滤器对输入管道的输入数据流进行限制，并保证输出管道的输出数据流有合适的内容，但它并不知道其输入管道、输出管道上的其它过滤器的实现细节^{[102][23]50-51}。

在 PF 风格的基础上,若所有过滤器都具有相同的接口,则形成了 UPF 风格。这种风格的主要实例出现在 Unix 操作系统中,其中过滤器进程具有由一个字符输入数据流(`stdin`)和两个字符输出数据流(`stdout` 和 `stderr`)组成的接口。通过限定使用这个接口,就能够随意排列组合独立开发的过滤器,从而形成新的应用。这也简化了理解一个特定的过滤器如何工作的任务^{[18]26}。

(2) 分层风格

分层风格主要有 CS 风格和 LCS 风格。CS 风格的服务器端负责数据管理。CS 风格的客户端与用户完成交互任务。此外,服务器组件提供了一组服务,并且监听对这些服务的请求。客户端组件可将请求通过连接器发送给服务器,期望执行一个服务。服务器可以拒绝执行这个请求,也可以执行这个请求并将响应发送回客户端^{[102][23]56-58}。

LCS 风格在 CS 风格的基础上添加了代理组件和网关组件,一个代理组件作为一个或多个客户端组件的共享服务器,它接收请求并进行可能的转换后将其转发给服务器。一个网关组件在客户端或代理看来好像一个常规的服务器,但是事实上它将请求进行可能的转换后转发给了它的“内部层”服务器。这些额外的中间件添加了很多个层,用来为系统添加诸如负载均衡和安全性检查这样的功能。基于 LCS 风格的架构常常被称为两层、三层或者多层架构。

LCS 风格也可以作为在大规模分布式系统中管理标识的一种解决方案,在这样的系统中,了解所有服务器的完整信息是代价昂贵的。相反,服务器被组织为多个层次,这样那些很少被用到的服务器可以由中间组件来处理,而不是直接由每个客户端组件来处理^{[18]28}。

(3) 复制风格

复制风格主要有复制仓库(`replicated repository`,简称 PR)和缓存风格。基于复制仓库风格的系统通过多个进程提供相同的服务,从而来改善数据的可访问性和可伸缩性。主要的例子包括诸如 XMS 这样的分布式文件系统和 CVS 这样的远程版本控制系统。

缓存风格为复制仓库风格的一种变体,通过复制个别请求的结果,从而使后面的请求重用这个结果。这种形式的复制最常出现在潜在的数据集远远超出单个客户端的容量的情况下,例如在 WWW^[18,103]中,或者在不必完全访问仓库的地方。可以执行延迟复制,当复制一个请求的尚未缓存的响应数据时,根据引用的局部性和兴趣的趋同性,将有用的数据项复制到缓存中以备稍后重用。还可以执行主动复制,基于预测到的请求来预先获取可缓存的数据项^{[18]26}。

与复制仓库风格相比,缓存风格对于用户可察觉性能的改善较少,因为没有命中缓存的请求更多,只有近期被访问过的数据才能被离线操作使用。另一

方面，缓存风格实现起来要容易得多，不需要复制仓库风格那么多的处理和存储，而且由于只有当前数据被请求时才会传输数据，因此缓存风格更加高效。

（4）点对点风格

点对点风格主要有基于事件的集成风格（event-based integration，简称 EBI）、C2 风格和分布式对象风格。EBI 风格通过除去了解连接器接口标识的必要性，降低了组件之间的耦合。此风格是一个组件能够发布一个或者多个事件。在事件发布后，系统中的其它组件能够注册某些感兴趣的事件类型，由系统本身来调用所有已注册的组件。

EBI 系统的基本形式由一个事件总线组成，所有的组件通过这个总线侦听它们所感兴趣的事件。当然，这会立即导致有关通知的数量、增加由通知引发其他组件的广播从而导致的事件风暴、在通知传送系统中的单点故障等方面的可伸缩性问题。这些问题能够通过以损害简单性为代价，使用分层系统和事件过滤来加以改善^{[18]32}。

C2 架构风格直接支持大粒度的重用，并且通过加强底层独立性，支持系统组件的灵活组合。它通过将 EBI 风格和 LCS 风格相结合来达到这些目标。异步通知消息向下传送，异步请求消息向上传送，这是组件之间通信的唯一方式。这加强了对高层依赖的松散耦合（服务请求可以被忽略），并且与底层实现了零耦合（不知道使用了通知），从而改善了对整个系统的控制，又没有丧失 EBI 的大多数优点^{[18]32}。

分布式对象（distributed objects，简称 DO）风格是将系统组织为结对进行交互的组件的集合。通常，一个对象的状态对于所有其他对象而言，是完全隐藏和受保护的。检查或修改对象状态的唯一方法是对该对象的一个公共的、可访问的操作发起请求或调用。这样就为每个对象创建了一个良好定义的接口，在对象的操作实现和它的状态信息保持私有的同时，公开操作对象的规格，这样做改善了可进化性。

一个操作可以调用可能位于另一些对象之上的操作。这些操作也同样可以调用其他对象之上的操作，以此类推。一个相关联的调用链被称为一个动作。状态分布位于对象之间，这对使状态尽可能保持最新而言是有利的，但不利之处是难以获得系统活动的总视图（缺乏可见性）^[18,104]。

（5）移动代码风格

移动代码风格（mobile code styles，简称 MCS）主要包括虚拟机、远程求值、按需代码和移动代理。移动代码风格使用移动性来动态地改变在处理过程与数据源或结果目的地之间的距离。Fuggetta^[18,105]全面检查过这些风格。为了考虑不同组件的位置，在架构层面引入一种站点抽象，作为主动配置的一部分。

位置概念的引入,使得在设计的层面对组件之间的交互开销进行建模成为可能。特别是,当与包含网络通信的交互开销作比较时,共享同一位置的组件之间的交互开销成本被认为是可以忽略不计的。通过改变自己的位置,一个组件可以改善交互质量,减少交互开销从而提高效率和用户可察觉的性能^{[18]29}。

虚拟机(virtual machine, 简称 VM)风格^[18,24]是所有移动代码风格的基础,该风格为代码的执行提供了一个满足安全性和可靠性的受控环境。虚拟机风格本身并不是基于网络的风格,但是它通常在 CS 风格中与一个组件结合在一起使用。

远程求值(remote evaluation, 简称 REV)风格^[18,105]来源于 CS 风格和 VM 风格,一个客户端组件知道如何来执行一个服务,但缺少执行此服务所必需的资源(CPU 周期、数据源等等),这些资源恰好位于一个远程站点上。客户端首先将如何执行服务的代码发给远程站点上的服务器组件,然后,服务器组件执行代码并将执行结果发回客户端。REV 风格假设将要被执行的代码是处在一种受保护的环境中,这样除了那些正在被使用的资源外,它不会影响到相同服务器的其他客户端。

在按需代码(code on demand, 简称 COD)风格中,客户端组件向一个远程服务器发送关于如何处理资源代码的请求,远程服务器将如何处理资源的代码发给客户端组件,客户端组件接收并在本地执行处理资源代码^{[18]31}。

在移动代理(mobile agent, 简称 MA)风格^[18,105]中,将完整的计算组件、组件的状态、必需的代码和执行任务需要的数据共同移动到远程站点。MA 风格的移动性是同时以 REV 和 COD 风格的两种方式工作的。

2.3 调整失效概率和影响因子

组件通常由第三方开发,具有清晰的接口描述,组件只能通过接口实现组件与其它组件的交互。组件接口分为需求接口和提供接口。其中,需求接口规定组件为实现自己的功能需要从其它组件得到的服务,提供接口规定组件为其它组件提供的服务^[106]。除了常见的功能接口描述以外,组件接口还表明该组件使用场合和可靠性等执行指标的描述^{[58]27-32},由组件的规范说明可得到软件组件的原始失效概率。

已有的基于组件的软件可靠性评估中,主要考虑软件组件的执行路径、状态、软件组件间的转移概率等,没有考虑软件组件所部署到的主机节点的运行情况。软件组件部署到主机节点后,在主机节点正常工作的情况下,软件组件才能正常工作;如果主机节点不能正常工作,或者主机节点经常失效,那么部署在此主机节点上的软件组件也不能提供服务,或者只能间断性地提供服务。

为了描述部署的不同主机节点对软件组件失效的影响，这里引入概念软件组件的调整前失效概率。对于软件组件 C_i ，如果 C_i 部署到主机节点 H_j 上，则 C_i 的调整前失效概率 fc_i 可由公式 (2-1) 求得。其中， ph_j 表示主机节点 H_j 的失效概率， pc_i 表示软件组件 C_i 的原始失效概率。

$$fc_i = ph_j + (1 - ph_j) \times pc_i \quad (2-1)$$

软件架构风格定义了软件组件的角色、功能以及软件组件间的约束关系。在单一的软件架构风格、混合的软件架构风格或者集成的架构风格中，软件组件将被分为不同的类型。前面已经分析过，不同类型的软件组件失效对系统失效率的影响是不同的。为了描述系统的软件架构风格中不同软件组件类型对系统失效率的影响，引入概念软件组件的调整后失效概率。对于软件组件 C_i ，如果系统中的软件组件分为 k 种类型，那么 C_i 的调整后失效概率 fc'_i 可由公式 (2-2) 求得。

$$fc'_i = \begin{cases} a_1 \times fc_i, & \text{如果 } C_i \text{ 属于第1种类型的软件组件} \\ a_2 \times fc_i, & \text{如果 } C_i \text{ 属于第2种类型的软件组件} \\ \dots\dots\dots \\ a_k \times fc_i, & \text{如果 } C_i \text{ 属于第 } k \text{ 种类型的软件组件} \end{cases} \quad (2-2)$$

在公式 (2-2) 中， $a_i (1 \leq i \leq k)$ 称为第 i 种软件组件类型的影响因子。影响因子的确定要经过以下 5 个步骤：

- ① 将软件组件间的交互图转换为有向图，其中节点表示软件组件，边描述对应的软件组件是否有交互关系；
- ② 边上有两个权值 x 和 y ， x 表示对应的软件组件的交互次数， y 表示每次交互的重要性；
- ③ 对于每个节点，计算所有出边的重要性之和；
- ④ 同一类型的软件组件对应同一类型的节点，求出同一类型的所有节点重要性的均值；
- ⑤ 不同类型节点的重要性的均值的比值，也就是不同类型的软件组件的影响因子的比值。

通过以上的 5 个步骤，可求出不同类型的软件组件的影响因子的比值。系统失效概率为小于 1 的正实数，因为影响因子描述的是软件组件失效对系统失效产生影响的程度，所以影响因子是小于 1 的正实数。系统架构师凭经验给出影响因子的初始值，也可在软件测试过程中或者系统运行过程中，来调整影响

因子的初始值，使它更适合当前的运行环境。

系统可靠性可由软件组件的调整后概率求得。如果系统由 N 个组件和 M 个主机节点构成，设定系统可靠性降低的主要原因是组件和主机节点的失效。系统可靠性可由式(2-3)求得。 R_{system} 表示系统可靠性。 P_{system} 表示系统失效率。 $ph_{1,M}$ 主机节点 $H_1, \dots, H_M$ 同时失效的概率。 fc'_i 组件 C_i 的调整后失效概率。

$$R_{system} = 1 - P_{system} = 1 - \bigcup_{j=1}^M ph_j - \left(\prod_{j=1}^M (1 - ph_j) \right) \times \left(\bigcup_{i=1}^N fc'_i \right) \quad (2-3)$$

$$\bigcup_{j=1}^M ph_j = \sum_{j=1}^M ph_j - \sum_{1 \leq i < j \leq M} ph_{i,j} + \dots + (-1)^{M-1} ph_{1,M} \quad (2-4)$$

2.4 基于组件的部署和CS风格结合的可靠性分析

组件部署方面的研究正受到越来越多的关注，基于组件的软件部署非常重要。CS 风格在基于网络应用的架构风格中最为常见，本节将基于组件的软件部署和 CS 架构风格有机地结合起来，分析两者对系统可靠性的影响。系统包含 M 个主机节点 $H = \{H_1, H_2, \dots, H_M\}$ 和 N 个软件组件 $C = \{C_1, C_2, \dots, C_N\}$ ，每个软件组件都向其它软件组件提供不同的服务，或者向其它软件组件请求服务。软件组件和主机节点间的部署关系可以矩阵 HC 描述，如果软件组件 C_j 部署在主机节点 H_i 上， $hc_{i,j}$ 的值为 1，否则为 0。

$$HC = \begin{matrix} & \begin{matrix} C_1 & C_2 & \dots & C_N \end{matrix} \\ \begin{matrix} H_1 \\ H_2 \\ \vdots \\ H_M \end{matrix} & \begin{bmatrix} hc_{1,1} & hc_{1,2} & \dots & hc_{1,N} \\ hc_{2,1} & hc_{2,2} & \dots & hc_{2,N} \\ \vdots & \vdots & \vdots & \vdots \\ hc_{M,1} & hc_{M,2} & \dots & hc_{M,N} \end{bmatrix} \end{matrix}$$

在 CS 风格下，作以下三个研究假设：

- ① 软件组件有两种类型：客户组件和服务组件；
- ② 客户组件向服务组件发送请求或者接收响应；
- ③ 服务组件接收请求或发送响应给客户组件。

实例系统包含 N 个软件组件 $C = \{C_1, \dots, C_N\}$ ， SC 表示服务组件的集合， SC_2 表示客户组件的集合。如果 $SC = \{C_1, \dots, C_k\}$ ，则其余软件组件构成客户组件集合，也就是 $SC_2 = C - SC$ 。不同类型软件组件失效对系统失效的影响是不同的，如式(2-5)和(2-6)所示。

$$fc'_i = a \times fc_i \quad (2-5)$$

$$a = \begin{cases} a_1, & \text{如果 } C_i \in SC \\ a_3, & \text{如果 } C_i \in SC_2 \end{cases} \quad (2-6)$$

fc_i 表示软件组件 C_i 的调整前失效概率, fc'_i 表示软件组件 C_i 的调整后失效概率, 可由公式(2-5)求得。 a, a_1 和 a_3 为影响因子, 取值为[0,1]间的任意实数, 其中 a_1 为服务组件类型的影响因子, a_3 为客户组件类型的影响因子。下面求出服务组件和客户组件的调整后失效概率。

(1) 服务组件的调整后失效概率

如果服务组件 C_s 部署在主机节点 H_j 上, 那么 C_s 的调整前失效概率 fc_s 可由式(2-7)得到, C_s 的调整后失效概率 fc'_s 可由式(2-8)得到。 ph_j 表示主机节点 H_j 的失效概率, pc_s 表示软件组件 C_s 的原始失效概率。

$$fc_s = 1 - (1 - ph_j) \times (1 - pc_s) \quad (2-7)$$

$$fc'_s = a_1 \times fc_s \quad (2-8)$$

(2) 客户组件的调整后失效概率

客户组件 C_i 需要向多个服务组件发送请求, 例如软件组件 $C_l, \dots, C_h$ 。这些软件组件已经部署到若干的主机节点上, 例如 $H_p, \dots, H_q$ 。 $CSCH$ 表示这些主机节点的集合, $CSCH = \{H_p, \dots, H_q\}$, $|CSCH| = k$ 。也就是, 这些服务组件 $C_l, \dots, C_h$ 被分为 k 个软件组件子集合, $CSCR_1, \dots, CSCR_k$ 。假设集合 $CSCR_1$ 中的所有软件组件部署到主机节点 H_p , $CSCR_k$ 中的软件组件部署到 H_q , 以此类推, 则每个子集合的软件组件将部署到一个主机节点上。

① 如果客户组件 C_i 部署到了 $H_l, H_l \in CSCH$, 那么软件组件 C_i 的调整前失效概率 fc_i 可由式(2-9)求得。 S_j 描述了某个软件组件子集合中所有软件组件都正常工作的概率。

$$fc_i = 1 - (1 - pc_i) \times \prod_{j=1}^k S_j \quad (2-9)$$

$$\begin{aligned} S_1 &= (1 - ph_p) \times \prod_{\forall C_l \in CSCR_1} (1 - pc_l) \\ &\vdots \\ S_k &= (1 - ph_q) \times \prod_{\forall C_j \in CSCR_k} (1 - pc_j) \end{aligned} \quad (2-10)$$

② 如果客户组件 C_i 部署到了 $H_l, H_l \notin CSCH$, 那么软件组件 C_i 的调整前失效概率 fc_i 可由式(2-11)求得。

$$fc_i = 1 - (1 - ph_l) \times (1 - pc_i) \times \prod_{j=1}^k S_j \quad (2-11)$$

因为 C_i 是客户组件，所以软件组件 C_i 的调整后失效概率 fc'_i 可由式(2-12)求得。

$$fc'_i = a_3 \times fc_i \quad (2-12)$$

2.5 基于组件的部署和LCS风格结合的可靠性分析

LCS 风格的架构常常被称为两层、三层或多层架构。本节以基于组件的软件部署和三层的 LCS 架构风格为例，分析两者对系统可靠性的影响。系统包含 M 个主机节点 $H = \{H_1, H_2, \dots, H_M\}$ 和 N 个软件组件 $C = \{C_1, C_2, \dots, C_N\}$ ，每个软件组件都向其它软件组件提供不同的服务，或者向其它软件组件请求服务。

在 LCS 风格下，作以下五个研究假设：

- ① 软件组件有三种类型：客户组件、中间组件和服务组件；
- ② 客户组件向中间组件发送请求或接收响应；
- ③ 中间组件接收请求或发送响应给客户组件；
- ④ 中间组件向服务组件发送请求或接收响应；
- ⑤ 服务组件接收请求或发送响应给中间组件。

若实例系统包含 N 个软件组件， $C = \{C_1, \dots, C_N\}$ ， SC 表示服务组件的集合， SC_1 表示中间组件的集合， SC_2 表示客户组件的集合。这四个集合满足如下关系： $C = SC \cup SC_1 \cup SC_2$ ， $SC \cap SC_1 = \emptyset$ ， $SC \cap SC_2 = \emptyset$ ， $SC_1 \cap SC_2 = \emptyset$ 。不同类型组件的失效对系统失效的影响是不同的，如式(2-13)和(2-14)所示。

$$fc'_i = a \times fc_i \quad (2-13)$$

$$a = \begin{cases} a_1, & \text{如果 } C_i \in SC \\ a_2, & \text{如果 } C_i \in SC_1 \\ a_3, & \text{如果 } C_i \in SC_2 \end{cases} \quad (2-14)$$

fc_i 表示软件组件 C_i 的调整前失效概率， fc'_i 表示软件组件 C_i 调整后失效概率。 a, a_1, a_2 和 a_3 为影响因子，取值为 $[0,1]$ 间的任意实数，其中 a_1 为服务组件类型的影响因子， a_2 为中间组件类型的影响因子， a_3 为客户组件类型的影响因子。下面求出服务组件、中间组件和客户组件的调整后失效概率。

(1) 服务组件的调整后失效概率

如果服务组件 C_s 部署在主机节点 H_j 上，那么服务组件 C_s 的调整前失效概率 fc_s 可由公式(2-15)求得，其中 pc_s 表示服务组件 C_s 的原始失效概率。服务组件 C_s

的调整后失效概率 fc'_s 可由式(2-16)求得。

$$fc_s = 1 - (1 - ph_j) \times (1 - pc_s) \quad (2-15)$$

$$fc'_s = a_1 \times fc_s \quad (2-16)$$

(2) 中间组件的调整后失效概率

中间组件 C_m 需要向多个服务组件发送请求, 例如 $C_x, \dots, C_y$ 。这些服务组件已经部署到若干个主机节点上, 例如 $H_a, \dots, H_b$ 。 $LCSCH$ 表示这些主机节点的集合, $LCSCH = \{H_a, \dots, H_b\}$, $|LCSCH| = k_s$ 。也就是服务组件 $C_x, \dots, C_y$ 被分为 k_s 个软件组件子集合, $LCSCR_1, \dots, LCSCR_{k_s}$ 。类似地, 假设 $LCSCR_1$ 中的软件组件部署到主机节点 H_a 上, $LCSCR_{k_s}$ 中的软件组件部署到主机节点 H_b 上, 以此类推。中间组件 C_m 的调整前失效概率 fc_m 和调整前失效概率 fc'_m 的计算过程如下:

① 如果中间组件 C_m 部署到主机节点 H_d 上, $H_d \in LCSCH$, fc_m 可由式(2-17)求得, pc_m 表示软件组件 C_m 的原始失效概率。 S_j 描述了某个软件组件子集合中的所有软件组件都正常工作的概率。

$$fc_m = 1 - (1 - pc_m) \times \prod_{j=1}^{k_s} S_j \quad (2-17)$$

$$\begin{aligned} S_1 &= (1 - ph_a) \times \prod_{\forall C_i \in LCSCR_1} (1 - pc_i) \\ &\vdots \\ S_{k_s} &= (1 - ph_b) \times \prod_{\forall C_j \in LCSCR_{k_s}} (1 - pc_j) \end{aligned} \quad (2-18)$$

② 如果中间组件 C_m 部署在主机节点 H_d 上, $H_d \notin LCSCH$ 。 fc_m 可由式(2-19)求得。

$$fc_m = 1 - (1 - ph_d) \times (1 - pc_m) \times \prod_{j=1}^{k_s} S_j \quad (2-19)$$

因为 C_m 为中间组件, C_m 的调整后失效概率 fc'_m 可由式(2-20)求得。

$$fc'_m = a_2 \times fc_m \quad (2-20)$$

(3) 客户组件的调整后失效概率

客户组件 C_c 向多个中间组件发送请求, 例如 $C_e, \dots, C_f$ 。这些中间组件已经部署在若干个主机节点上, 例如 $H_g, \dots, H_l$ 。 $LCSCHM$ 表示这些主机节点的集合, $LCSCHM = \{H_g, \dots, H_l\}$, $|LCSCHM| = k_M$ 。也就是, 这些中间组件 $C_e, \dots, C_f$ 被分成 k_M 个软件组件子集合, $LCSCRM_1, \dots, LCSCRM_{k_M}$ 。假设 $LCSCRM_1$ 中的软件组件部署到主机节点 H_g 上, $LCSCRM_{k_M}$ 中的软件组件部署到主机节点 H_l 上, 以此类推。

$\exists LCSCRM_i (i=1,2,\dots,k_M), \exists C_m \in LCSCRM_i$, 中间组件 C_m 需要向多个服务组件发送请求, 例如 $C_g, \dots, C_r$ 。这些服务组件已经部署到若干个主机节点上, 例如 $H_m, \dots, H_n$ 。 $LCSCHS$ 表示这些主机节点的集合, $LCSCHS = \{H_m, \dots, H_n\}, |LCSCHS| = k_s$ 。也就是, 这些服务组件 $C_g, \dots, C_r$ 被分成 k_s 个软件组件子集合: $LCSCRS_1, \dots, LCSCRS_{k_s}$ 。假设 $LCSCRS_1$ 中的软件组件部署到主机节点 H_m 上, $LCSCRS_{k_s}$ 中的软件组件部署到主机节点 H_n 上, 以此类推。

假设中间组件 C_m 部署在主机节点 H_{r_2} , 客户组件 C_c 部署在主机节点 H_{r_1} 。
 $HH = LCSCRM \cup LCSCHS$, 根据以下四种情况求得软件组件 C_c 的调整前失效概率 fc_c :

① 如果 $H_{r_1} \in HH, H_{r_2} \in HH$

$$fc_c = 1 - (1 - pc_c) \times (1 - pc_m) \times temp \quad (2-21)$$

② 如果 $H_{r_1} \notin HH, H_{r_2} \in HH$

$$fc_c = 1 - (1 - pc_c) \times (1 - pc_m) \times (1 - ph_{r_1}) \times temp \quad (2-22)$$

③ 如果 $H_{r_1} \in HH, H_{r_2} \notin HH$

$$fc_c = 1 - (1 - pc_c) \times (1 - pc_m) \times (1 - ph_{r_2}) \times temp \quad (2-23)$$

④ 如果 $H_{r_1} \notin HH, H_{r_2} \notin HH$

$$fc_c = 1 - (1 - pc_c) \times (1 - pc_m) \times (1 - ph_{r_1}) \times (1 - ph_{r_2}) \times temp \quad (2-24)$$

其中, $temp$ 是临时变量, 如式 (2-25) 所示。 $S_{m,1}, \dots, S_{m,k_s}$ 描述了中间组件 C_m 的发送请求的每个服务组件子集合中的所有软件组件都正常工作的概率。如果集合 $LCSCRM_m$ 中的软件组件部署在主机节点 H_l , ph'_m 表示主机节点 H_l 的失效概率。

$$temp = \prod_{m=1}^{k_M} (((1 - ph'_m) \times \prod_{\forall C_i \in LCSCRM_m, C_i \neq C_m} (1 - pc_i)) \times \prod_{j=1}^{k_s} S_{m,j}) \quad (2-25)$$

$$\begin{aligned} S_{m,1} &= (1 - ph_m) \times \left(\prod_{\forall C_i \in LCSCRS_1} (1 - pc_i) \right) \\ &\vdots \end{aligned} \quad (2-26)$$

$$S_{m,k_s} = (1 - ph_n) \times \left(\prod_{\forall C_i \in LCSCRS_{k_s}} (1 - pc_i) \right)$$

因为组件 C_c 为客户组件, 客户组件 C_c 的调整后失效概率 fc'_c 可由式 (2-27) 求得。

$$fc'_c = a_3 \times fc_c \quad (2-27)$$

2.6.1 CS风格实验及结果分析

(1) 输入

系统输入包括 4 个主机节点的失效率、14 个软件组件的原始失效概率、CS 架构风格的软件组件交互图、随机产生的三种部署架构。CS 架构风格的软件组件间的交互图如图 2-1 所示。

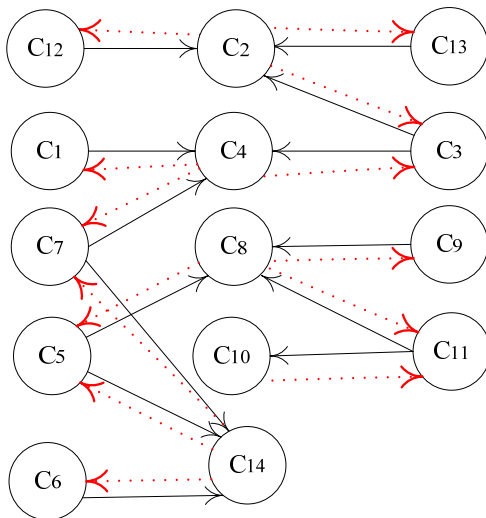


Fig. 2-1 Interaction graph of software components in CS style

- 31 -

4 个主机节点的失效率为 $[0, 0.002]$ 间的任意实数, ph_i 表示为主机节点 H_i 的失效率。 $ph_1 = 0.0013$, $ph_2 = 0.0005$, $ph_3 = 0.0009$, $ph_4 = 0.0002$ 。14 个软件组件的原始失效概率为 $[0, 0.003]$ 间的任意实数, pc_i 表示为软件组件 C_i 的原始失效概率。 $pc_1 = 0.0004$, $pc_2 = 0.0013$, $pc_3 = 0.0007$, $pc_4 = 0.0003$, $pc_5 = 0.0011$, $pc_6 = 0.0002$, $pc_7 = 0.0006$, $pc_8 = 0.0012$, $pc_9 = 0.0008$, $pc_{10} = 0.0013$, $pc_{11} = 0.0021$, $pc_{12} = 0.0012$, $pc_{13} = 0.0001$, $pc_{14} = 0.0002$ 。

随机产生三种部署架构, 描述如何将 14 个软件组件部署到 4 个主机节点上。矩阵 HC_1 、 HC_2 和 HC_3 分别描述第一种部署架构、第二种部署架构和第三种部署架构。

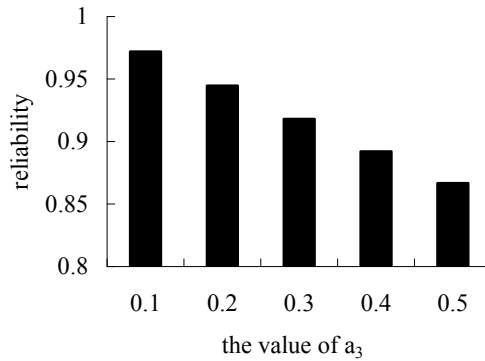
$$HC_1 = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \end{bmatrix} \quad HC_2 = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$HC_3 = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

从矩阵 HC_1 可以看出第一种部署架构是, 软件组件 C_1 、 C_2 、 C_3 、 C_4 、 C_{12} 和 C_{13} 部署到主机节点 H_1 上, 软件组件 C_5 、 C_6 和 C_{14} 部署到主机节点 H_2 上, 软件组件 C_7 、 C_8 和 C_9 部署到主机节点 H_3 上, 软件组件 C_{10} 和 C_{11} 部署到主机节点 H_4 上; 从矩阵 HC_2 可以看出第二种部署架构是, 软件组件 C_4 、 C_8 和 C_{13} 部署到主机节点 H_1 上, 软件组件 C_1 、 C_2 、 C_9 、 C_{10} 和 C_{12} 部署到主机节点 H_2 上, 软件组件 C_3 、 C_7 、 C_{11} 和 C_{14} 部署到主机节点 H_3 上, 软件组件 C_5 和 C_6 部署到主机节点 H_4 上; 从矩阵 HC_3 可以看出第三种部署架构是, 软件组件 C_5 部署到主机节点 H_1 上, 软件组件 C_4 、 C_8 、 C_{11} 、 C_{12} 、 C_{13} 和 C_{14} 部署到主机节点 H_2 上, 软件组件 C_1 、 C_2 、 C_3 、 C_7 、 C_9 和 C_{10} 部署到主机节点 H_3 , 软件组件 C_6 部署到主机节点 H_4 上。

(2) 影响因子 a_3 对可靠性的影响

在本实验中, 软件组件分为两种类型: 服务组件和客户组件。服务组件类型的影响因子设为 a_1 , 客户组件类型的影响因子设为 a_3 。根据 CS 风格的软件组件间交互图, 通过 2.3 节的求解影响因子的 5 个步骤求出 a_1 和 a_3 的比值。为了使计算简明, 将权值 x 和 y 均设置为 1, 得到 $a_1 : a_3 = 9 : 5$ 。这里根据第一种部署架构将 14 个软件组件部署到 4 个主机节点上。当 a_3 的初始值不同时, 可靠性的变化情况如图 2-2 所示。

图 2-2 随着 a_3 值改变的可靠性变化情况Fig.2-2 Reliability with different values of a_3

在图 2-2 中, a_3 值确定在 CS 风格中客户组件的失效对系统失效的影响, a_3 值增加表明客户组件的平均失效对系统失效的影响逐渐增加, 由于 a_1 与 a_3 的比值固定, 也表明服务组件的平均失效对系统失效的影响在增加。随着 a_3 值的增加, 得到的可靠性值降低, 而且降低幅度较大, 由 $a_3 = 0.1$ 时的 0.972 降至 $a_3 = 0.5$ 时的 0.8667, 这充分说明影响因子 a_3 的作用是非常明显的。

(3) 软件组件类型对可靠性的影响

本实验采用第一种部署架构。服务组件类型的影响因子为 a_1 , 客户组件类型的影响因子为 a_3 。根据 2.3 节的影响因子求解过程可知, 一般情况下, 权值 x 和 y 取值不同, 得到 a_1 与 a_3 的比值也不同。为了使计算简明, 这里讨论 5 种情况的 a_1 与 a_3 的关系, 这 5 种情况下的可靠性对比如图 2-3 所示。

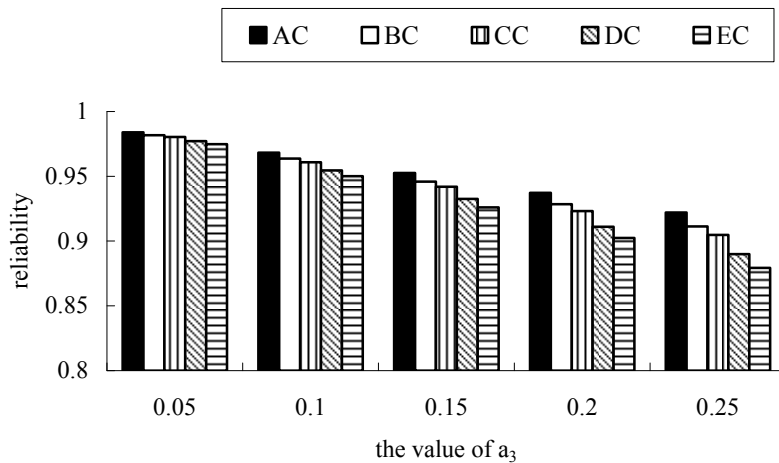


图 2-3 5 种情况的可靠性对比图

Fig.2-3 Comparison of reliability in five different cases

在图 2-3 中, AC 表示 $a_1 = 1.2 \times a_3$ 情况下的可靠性, BC 表示 $a_1 = 1.5 \times a_3$ 下的可靠性, CC 表示 $a_1 = 1.8 \times a_3$ 情况下的可靠性, DC 表示 $a_1 = 2.5 \times a_3$ 情况下的可靠性, EC 表示 $a_1 = 3 \times a_3$ 情况下的可靠性。在 CS 架构风格下, 服务组件和客户组件间交互次数、交互的重要性、客户组件的数目、服务组件的数目等等不同的情况, 使 a_1 与 a_3 有不同的对比关系。对于同一 a_3 值, 当 a_1 与 a_3 的比值逐渐增大, 服务组件的失效影响对系统失效的影响越来越大时, 系统越容易失效, 得到的可靠性越低。对于不同的 a_3 值, 随着 a_3 值的增加, 客户组件的失效对系统失效的影响越来越大, 并且随着 a_1 与 a_3 比值的增加, 服务组件的失效对系统的影响逐渐增大, 系统更容易失效, 得到的可靠性更低。因此, 为了提高系统可靠性, 应降低不同类型的软件组件的影响因子, 降低其对系统失效的影响。

(4) 部署架构对可靠性的影响

前面两个实验分析了同一软件架构风格 CS 中, 不同的软件组件类型对可靠性的影响。本实验分析 14 个软件组件部署到 4 个不同的主机节点的 3 种部署架构对系统可靠性的影响。服务组件类型的影响因子 a_1 , 客户组件类型的影响因子为 a_3 , $a_1 : a_3 = 9 : 5$ 。3 种部署架构对系统可靠性的影响如图 2-4 所示。

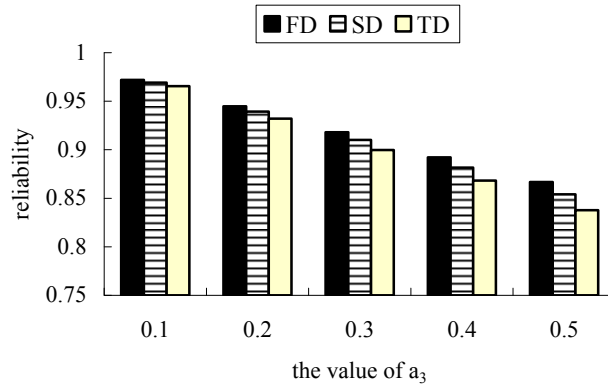


图 2-4 3 种部署方式的可靠性对比图

Fig.2-4 Comparison of reliability in three deployments

在图 2-4 中, FD 表示第一种部署架构的可靠性, SD 表示第二种部署架构的可靠性, TD 表示第三种部署架构的可靠性。对于固定的 a_3 , 也就是 CS 风格中软件组件类型对系统失效影响确定的情况, 不同的部署方式对系统失效的影响也是不同的。按照第一种部署架构将 14 个软件组件部署到 4 个主机节点得到的可靠性最高, 而按照第三种部署架构得到的可靠性最低。并且, 随着 a_3 值的增加, 三种部署架构得到可靠性值的差值也在增加。因此, 不同的部署架构对

软件组件的失效率和系统失效率的影响是明显的，值得进一步研究如何将软件组件更有效地部署到主机节点上，从而得到较高的软件组件可靠性和系统可靠性。

2.6.2 LCS风格实验及结果分析

在本实验中，主要分析基于组件的软件部署和 LCS 架构风格有机结合对系统失效率的影响。软件组件集合 C 为 $C = \{C_1, C_2, \dots, C_{14}\}$ ，集合 C 中的所有组件被分为 3 个软件组件子集合：服务组件集合 SC ，中间组件集合 SC_1 和客户组件集合 SC_2 ，主机节点集合 $H = \{H_1, H_2, H_3, H_4\}$ 。

(1) 输入

系统输入包括 4 个主机节点的失效率、14 个软件组件的原始失效概率、LCS 架构风格的软件组件交互图、随机产生的 1 种部署架构。4 个主机节点的失效率为 $[0, 0.002]$ 间的任意实数， ph_i 表示为主机节点 H_i 的失效率。 $ph_1 = 0.0013$ ， $ph_2 = 0.0005$ ， $ph_3 = 0.0009$ ， $ph_4 = 0.0002$ 。14 个软件组件的原始失效概率为 $[0, 0.003]$ 间的任意实数， pc_i 表示为软件组件 C_i 的失效概率。 $pc_1 = 0.0004$ ， $pc_2 = 0.0013$ ， $pc_3 = 0.0007$ ， $pc_4 = 0.0003$ ， $pc_5 = 0.0011$ ， $pc_6 = 0.0002$ ， $pc_7 = 0.0006$ ， $pc_8 = 0.0012$ ， $pc_9 = 0.0008$ ， $pc_{10} = 0.0013$ ， $pc_{11} = 0.0021$ ， $pc_{12} = 0.0012$ ， $pc_{13} = 0.0001$ ， $pc_{14} = 0.0002$ 。

LCS 风格的软件组件间的交互图，如图 2-5 所示。

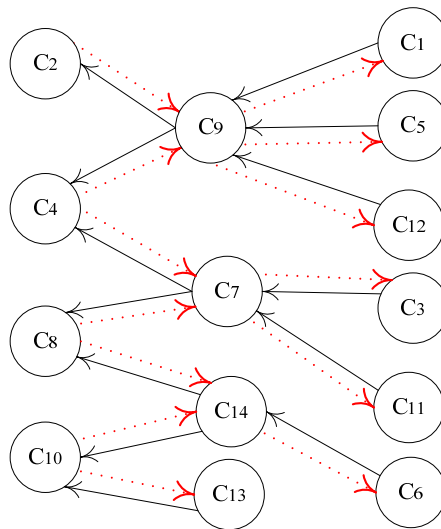


图 2-5 LCS 风格的组件交互图

Fig. 2-5 Interaction graph of software components in LCS style

从图 2-5 可以看出，服务组件集合 $SC = \{C_2, C_4, C_8, C_{10}\}$ ，客户组件集合

$SC_2 = \{C_1, C_3, C_5, C_6, C_{11}, C_{12}, C_{13}\}$ 和中间组件集合 $SC_1 = \{C_7, C_9, C_{14}\}$ 。客户组件 C_1 向中间组件 C_9 发送请求，用实线表示，中间组件 C_9 向服务组件 C_2 发送请求，也用实线表示，中间组件 C_9 接收来自服务组件 C_2 的响应，用点线表示。

矩阵 HC_4 描述在 LCS 风格下，将 14 个软件组件部署到 4 个主机节点上，称为第四种部署架构。从矩阵 HC_4 可以看出，软件组件 C_1 、 C_2 、 C_3 、 C_9 和 C_{12} 部署到主机节点 H_1 上，软件组件 C_4 、 C_5 、 C_7 和 C_{11} 部署到主机节点 H_2 上，软件组件 C_6 、 C_8 和 C_{14} 部署到主机节点 H_3 上，软件组件 C_{10} 和 C_{13} 部署到主机节点 H_4 上。

$$HC_4 = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \end{bmatrix}$$

(2) 软件组件类型对可靠性的影响

在本实验中，用第四种部署架构将软件组件部署到主机节点上。软件组件分为 3 种类型：服务组件、中间组件和客户组件。这里，将服务组件类型的影响因子设为 a_1 ，中间组件类型的影响因子设为 a_2 ，客户组件类型的影响因子设为 a_3 。由 a_1 、 a_2 和 a_3 的比值求解，与不同类型的软件组件间的交互次数、交互的重要性、客户组件的数目、中间组件的数目和服务组件的数目等等有关。根据 LCS 风格的软件组件间交互图，通过 2.3 节的求解影响因子的 5 个步骤求出 a_1 、 a_2 和 a_3 的比值。为了使计算简明，将权值 x 和 y 均设置为 1，得到 $a_1 : a_2 : a_3 = 7 : 16 : 4$ 。另外，为了更清楚地描述 3 种类型的软件组件失效对系统失效的影响，再讨论 3 种 a_1 、 a_2 和 a_3 的不同比值关系对系统失效的影响。

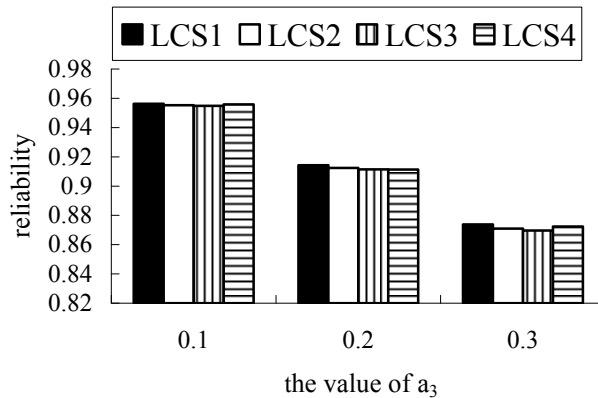


图 2-6 4 种情况下的可靠性对比图

Fig.2-6 Comparison of reliability in four cases

如图 2-6 描述了 4 种情况下的可靠性对比关系。LCS1 表示第一种情况下 $a_1:a_2:a_3=7:16:4$ 的可靠性，LCS2 表示第二种情况下 $a_1:a_2:a_3=3:2:1$ 的可靠性，LCS3 表示第三种情况下 $a_1:a_2:a_3=3:3:1$ 的可靠性，LCS4 表示第四种情况下 $a_1:a_2:a_3=3:2:2$ 的可靠性。从图 2-6 很容易看出，随着 a_3 值的增加，也就是客户组件的失效对系统失效影响的增加，这 4 种情况下得到的可靠性都在降低。对于特定的 a_3 ，虽然在 LCS2 和 LCS4 的情况中，服务组件和中间组件的影响因子的比值相同，并且在 LCS2 和 LCS3 的情况中，服务组件和客户组件的影响因子的比值相同，但得到的系统可靠性仍体现出了一定的差别。系统中所具有的软件组件类型和软件组件间的相互制约关系，对系统可靠性有重要影响。

(3) 不同架构风格和部署架构对可靠性的影响

在本实验中，比较在 CS 和 LCS 两种架构风格中，部署架构和影响因子对可靠性的影响，如图 2-7 所示。

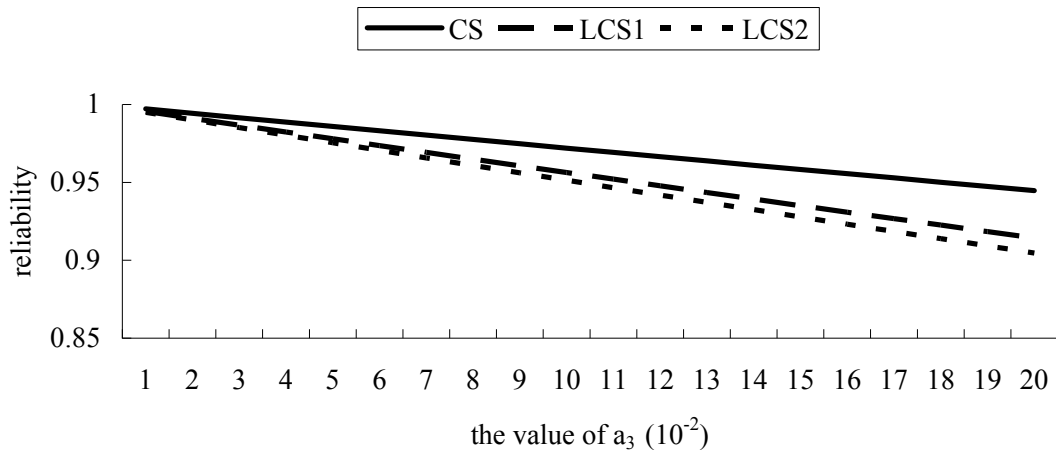


图 2-7 CS 和 LCS 架构风格的可靠性对比图

Fig.2-7 Comparison of reliability of CS and LCS styles

在图 2-7 中，CS 表示在 CS 风格下，采用第一种部署架构和 $a_1:a_3=9:5$ 情况下的可靠性，LCS1 表示在 LCS 风格下，用第四种部署架构和 $a_1:a_2:a_3=7:16:4$ 情况下的可靠性，LCS2 表示在 LCS 风格下，用第一种部署架构和 $a_1:a_2:a_3=4:4:1$ 情况下的可靠性。从 CS 和 LCS2 比较可以看出，采用相同的软件组件和主机节点，并且软件组件在主机节点上的部署架构相同，但由于系统采用的软件架构风格不同，使得到的可靠性差异很大，表现软件架构风格对可靠性的影响是明显的。从 CS 和 LCS1 比较可以看出，尽管使用相同的软件组件和主机节点，但两者采用的软件架构风格和部署方法都不同，使得到的可靠性差距仍然较大。因此，软件架构风格和部署架构对可靠性分析有重要影响。

2.7 本章小结

本章首先介绍常见的软件部署方法和经典的软件架构风格，分析部署的原理和软件架构风格的构建原理。其次，提出了一种把基于组件的软件部署和 CS 风格、LCS 风格进行有机地结合的方法，该方法分析两者对系统失效率的影响。根据软件组件在主机节点上的不同部署情况对软件组件失效率的影响，提出概念软件组件的调整前失效概率。根据软件架构风格中不同类型的软件组件对系统失效率的影响，提出概念软件组件的调整后失效概率和影响因子，并给出求解影响因子的方法。最后，给定相同 4 个主机节点和 14 个软件组件的实验，实验结果表明，在基于组件的软件部署中，不同的软件架构风格、同一软件架构风格中不同类型的软件组件和部署架构对系统可靠性有重要影响。

第3章 考虑模糊关系的组件部署算法

3.1 引言

由第2章的分析可知,软件组件在主机节点上的不同部署架构对软件组件的失效率和系统失效率有重要影响。本章将分析影响部署的因素和如何将软件组件有效地部署到合理的主机节点上。首先,影响部署的因素很多,例如系统工程师的知识、软件架构师的知识、软件组件的大小、软件组件运行需要的内存大小等等。其次,对于主机节点上的可用内存,在系统运行前或者运行期间,都可准确知道每个主机节点上的可用内存,这种类型的因素可精确地描述出来。但是,像系统工程师的知识或者软件架构师的知识,这类影响因素具有模糊性和未确知性,很难将其进行精确化描述。再次,考虑这些影响因素是为了确定软件组件如何在主机节点上进行部署,也就是确定软件组件间的关系和主机节点间的关系。对于软件组件间的关系,例如,部署过程中要求软件组件 C_i 和 C_j 部署到同一个主机节点上,或者要求 C_i 和 C_j 不能部署到相同主机节点上。由于这种关系受确定性和不确定性因素的共同影响,本章引入模糊理论,提出软件组件间的模糊关系和主机节点间的模糊关系,并根据模糊矩阵,设计模糊贪婪部署算法将软件组件有效地部署到主机节点上。

3.2 相关概念和表示方法

在现实世界中,模糊现象或者模糊概念无处不在,例如“老人”、“矮个子”、“劣质产品”、“健康者”、“晴天”等等,可以说人脑中能形成的概念都具有模糊性。1965年,美国南加州大学的Zadeh教授在论文^[107]中第一次提出了模糊集合的概念,成功地应用数学方法来描述模糊现象,从此开创了模糊数学和其应用的新纪元。模糊数学一经产生,便表现出其旺盛的生命力。1980年,丹麦哥本哈根的史密斯水泥公司第一次使用模糊系统实现对生产设备的控制。1983年,日本第一部模糊控制试剂投放装置在秋田市的应用领域得到实现。1988年,日立公司创新地在日本仙台市的地铁实现了大规模模糊控制。随后,索尼、松下等多家电子设计公司用模糊技术产生了数百种“智能”产品。相关领域跨越摄像机、电视机、洗衣机、微波炉到计算机存储装置、机车变速器和机械手。时至今日,模糊数学在实际中的应用几乎覆盖了国民经济的各个相关领域及主要部门。模糊数学广泛地应用在农业生产、气象预告、医疗、环境、军事情报、质量管理、人工智能、故障诊断、系统评价、聚类分析、控制理论、专家系统、

图像识别、决策优化等诸多方面^[108-109]。

(1) 相关概念

定义 3-1 所谓给定论域 U 上的一个模糊子集 $\underline{A}$ ，就是给定由论域 U 到区间 $[0,1]$ 的一个映射

$$\mu_{\underline{A}} : U \rightarrow [0,1] \quad (3-1)$$

$$u \mapsto \mu_{\underline{A}}(u) \in [0,1] \quad (3-2)$$

这个映射 $\mu_{\underline{A}}$ 将任一 $u \in U$ 对应着一个确定的值 $\mu_{\underline{A}}(u) \in [0,1]$ ，值 $\mu_{\underline{A}}(u)$ 叫做 u 对模糊子集 $\underline{A}$ 的隶属度，映射 $\mu_{\underline{A}}$ 叫做模糊子集 $\underline{A}$ 的隶属函数 ($\underline{A}$ 的隶属函数也可以记为 $\mu_{\underline{A}}(u)$ 或简记为 $\underline{A}(u)$)^{[110]13}。

定义 3-2 所谓从 X 到 Y 的一个模糊关系 $\underline{R}$ 是指存在映射

$$\underline{R} : X \times Y \rightarrow [0,1] \quad (3-3)$$

$\underline{R}$ 的隶属函数为 $\mu_{\underline{R}}(x,y)$ ，函数值 $\mu_{\underline{R}}(x,y)$ 代表有序数对 (x,y) 的相关程度^{[111]50}。

定义 3-3 如果对于任意的 i, j 都有 $r_{ij} \in [0,1]$ ，则称矩阵 $\underline{R} = (r_{ij})_{m \times n}$ 为模糊矩阵^{[110]49}。

$$\underline{R} = \begin{bmatrix} r_{11} & r_{12} & \cdots & r_{1n} \\ r_{21} & r_{22} & \cdots & r_{2n} \\ \vdots & \vdots & & \vdots \\ r_{m1} & r_{m2} & \cdots & r_{mn} \end{bmatrix}$$

设 $\underline{A} = (a_{ij})_{m \times n}$ ， $\underline{B} = (b_{ij})_{m \times n}$ ，则有^{[111]49}

$$\textcircled{1} \quad \underline{A} = \underline{B} \Leftrightarrow a_{ij} = b_{ij}; \quad (3-4)$$

$$\textcircled{2} \quad \underline{A} \leq \underline{B} \Leftrightarrow a_{ij} \leq b_{ij}; \quad (3-5)$$

$$\textcircled{3} \quad \underline{A} \cup \underline{B} = (a_{ij} \vee b_{ij})_{m \times n}; \quad (3-6)$$

$$\textcircled{4} \quad \underline{A} \cap \underline{B} = (a_{ij} \wedge b_{ij})_{m \times n} \quad (3-7)$$

模糊矩阵的合成运算，设 $\underline{A} = (a_{ij})_{m \times s}$ ， $\underline{B} = (b_{ij})_{s \times n}$ ，则称模糊矩阵

$$\underline{A} \circ \underline{B} = (c_{ij})_{m \times n} \quad (3-8)$$

为 $\underline{A}$ 与 $\underline{B}$ 的合成，其中， $c_{ij} = \bigvee_{k=1}^s (a_{ik} \wedge b_{kj})$ ^{[88]50}。

(2) 模糊子集表示方法

表示论域 U 上的一个模糊子集，原则上只需将每个元素 $u \in U$ 赋予该元素对模糊子集 $\underline{A}$ 的隶属度 $\mu_{\underline{A}}(u)$ ，然后将它们用一定形式构造在一起即可。下面是常用的几种表示方法^{[110]14}。

论域 U 是有限集, $U = \{u_1, u_2, \dots, u_n\}$, U 上任一模糊集 $\underline{A}$, 其隶属函数为 $\{\underline{A}(u_i)\} (i=1, 2, \dots, n)$ 。

① Zadeh 表示法

$$\underline{A} = \frac{\underline{A}(u_1)}{u_1} + \frac{\underline{A}(u_2)}{u_2} + \dots + \frac{\underline{A}(u_n)}{u_n} \quad (3-9)$$

其中, $\frac{\underline{A}(u_i)}{u_i}$ 不是分数, “+” 也不表示求和, 只有符号意义, 它表示点 u_i 对模糊集 $\underline{A}$ 的隶属度是 $\underline{A}(u_i)$ 。

② 序偶表示法

$$\underline{A} = \{(u_1, \underline{A}(u_1)), (u_2, \underline{A}(u_2)), \dots, (u_n, \underline{A}(u_n))\} \quad (3-10)$$

③ 向量表示法

$$\underline{A} = (\underline{A}(u_1), \underline{A}(u_2), \dots, \underline{A}(u_n)) \quad (3-11)$$

若论域 U 为可列集 $U = \{u_1, u_2, \dots, u_n, \dots\}$, 则 U 上的模糊子集为

$$\underline{A} = \sum_{i=1}^{\infty} \frac{\underline{A}(u_i)}{u_i} \quad (3-12)$$

(3) 模糊分布

如果模糊集合的定义域为实数域上, 那么模糊集合的隶属函数被称为模糊分布。模糊分布可分为偏小型、中间型和偏大型。广泛应用的模糊分布主要包括矩形分布、梯形分布、抛物线型分布、正态分布、 Γ 型分布、柯西分布等。这里列举 k 次抛物线分布、矩阵分布和正态分布, 当 $k=1$ 时, 抛物线分布退化为梯形分布^[112]。

① k 次抛物线分布的表达式

$$\text{偏小型:} \quad \underline{A}(x) = \begin{cases} 1, & x < a \\ \left(\frac{b-x}{b-a}\right)^k, & a \leq x \leq b \\ 0, & x > b \end{cases} \quad (3-13)$$

$$\text{中间型:} \quad \underline{A}(x) = \begin{cases} 0, & x < a \\ \left(\frac{x-a}{b-a}\right)^k, & a \leq x < b \\ 1, & b \leq x < c \\ \left(\frac{d-x}{d-c}\right)^k, & c \leq x < d \\ 0, & x \geq d \end{cases} \quad (3-14)$$

$$\text{偏大型: } \tilde{A} = \begin{cases} 0, & x < a \\ \left(\frac{x-a}{b-a}\right)^k, & a \leq x \leq b \\ 1, & x > b \end{cases} \quad (3-15)$$

② 矩形分布的表达式

$$\text{偏小型: } \tilde{A}(x) = \begin{cases} 1, & x \leq a \\ 0, & x > a \end{cases} \quad (3-16)$$

$$\text{中间型: } \tilde{A}(x) = \begin{cases} 0, & x < a \\ 1, & a \leq x \leq b \\ 0, & x > b \end{cases} \quad (3-17)$$

$$\text{偏大型: } \tilde{A}(x) = \begin{cases} 0, & x < a \\ 1, & x \geq a \end{cases} \quad (3-18)$$

③ 正态分布的表达式

$$\text{偏小型: } \tilde{A}(x) = \begin{cases} 1, & x \leq a \\ e^{-\left(\frac{x-a}{\sigma}\right)^2}, & x > a \end{cases} \quad (3-19)$$

$$\text{中间型: } \tilde{A} = e^{-\left(\frac{x-a}{\sigma}\right)^2} \quad (3-20)$$

$$\text{偏大型: } \tilde{A} = \begin{cases} 0, & x \leq a \\ 1 - e^{-\left(\frac{x-a}{\sigma}\right)^2}, & x > a \end{cases} \quad (3-21)$$

3.3 软件组件间和主机节点间模糊矩阵及求解

影响部署的因素很多，按软件的不同阶段来分，包括设计阶段和运行阶段的影响因素。设计阶段的因素包括系统工程师的知识、需求文档中可用的数据、功能相似环境的执行日志等等。运行阶段的因素包括软件架构师的知识、内部属性（例如，失效频率、异常和服务请求）、系统的外部属性（例如，网络波动、可充电电池的电量、主机节点的位置改变、软件组件的位置改变）等等。从构成系统的主要实体来分，主要包括影响软件组件可靠性的因素和影响主机节点可靠性的因素。影响软件组件可靠性的因素主要包括软件组件间的逻辑移动、网络物理链接的可靠性、软件组件的大小、软件组件提供的服务、软件组件间交互频率、软件组件提供服务的重要性、软件组件运行需要的内存大小、软件组件间的位置约束等等。影响主机节点可靠性的因素主要包括电量消耗、硬件设备的移动性、硬件平台的异构性、电磁干扰（尤其对无线网络）、硬件平台

各部件的可靠性等。这些因素有确定性的因素，也有不确定性因素，很难将所有因素都精确化描述。为了在部署过程中充分考虑这些因素的影响，引入模糊理论，将多维的影响因素转换为软件组件间的模糊关系和主机节点间的模糊关系。

3.3.1 软件组件间和主机节点间的模糊关系

定义 3-4 软件组件间的模糊关系：设软件组件集合 $C = \{C_1, C_2, \dots, C_n\}$ ， $|C|=n$ ，称 $C \times C$ 的一个模糊子集 $\underline{R} \in F(C \times C)$ 为 C 到 C 上的模糊关系，记为 $C \xrightarrow{\underline{R}} C$ ，其隶属函数为

$$\mu_{\underline{R}} : C \times C \rightarrow [0, 1] \quad (3-22)$$

$$(c, c) \mapsto \mu_{\underline{R}}(c, c) = \underline{R}(c, c) \quad (3-23)$$

并称隶属度 $\underline{R}(c, c)$ 为 (c, c) 关于模糊关系 $\underline{R}$ 的相关程度。

为了描述系统中任意两个软件组件间的模糊程度，引入概念软件组件间的模糊矩阵。

定义 3-5 软件组件间的模糊矩阵：设 $\underline{R} \in F(C \times C)$ ，若对于 $i=1, 2, \dots, n$ ； $j=1, 2, \dots, n$ ，都有 $r_{ij} = \underline{R}(c_i, c_j) \in [0, 1]$ ，则称

$$\underline{R} = \begin{bmatrix} r_{11} & r_{12} & \cdots & r_{1n} \\ r_{21} & r_{22} & \cdots & r_{2n} \\ \vdots & \vdots & & \vdots \\ r_{n1} & r_{n2} & \cdots & r_{nn} \end{bmatrix}$$

为 $n \times n$ 阶软件组件间的模糊矩阵。

设系统中包含 m 个主机节点 $H = \{H_1, H_2, \dots, H_m\}$ ，对于 $\forall H_i, H_j \in H$ ，这两个主机节点间的网络带宽、网络物理链接的可靠性、主机节点上可用的内存和周围环境中的电磁干扰等，都将影响这两个主机节点部署软件组件的效果。例如，如果主机节点 H_i 和 H_j 的网络物理链接的可靠性很高，有两个软件组件 C_i 和 C_j ，部署过程要求 C_i 和 C_j 或者部署到相同主机节点上，或者部署到网络物理链接可靠性高的相邻主机节点上，但是，如果主机节点 H_i 没有足够可用的内存来部署软件组件 C_i 或者 C_j ，并且主机节点 H_j 上可用的内存只能部署 C_i 或者 C_j ，而不能同时部署 C_i 和 C_j ，那么软件组件 C_i 和 C_j 就不能部署到主机节点 H_i 和 H_j 上。也就是，主机节点间的关系将影响部署过程。影响主机节点间关系的因素很多，具有确定性和不确定性，因此引入模糊理论来描述主机节点间的关系。

定义 3-6 主机节点间的模糊关系：设主机节点集合 $H = \{H_1, H_2, \dots, H_m\}$ ， $|H| = m$ ，称 $H \times H$ 的一个模糊子集 $\tilde{Q} \in F(H \times H)$ 为 H 到 H 上的模糊关系，记为 $H \xrightarrow{\tilde{Q}} H$ ，其隶属函数为

$$\mu_{\tilde{Q}} : H \times H \rightarrow [0, 1] \quad (3-24)$$

$$(h, h) \mapsto \mu_{\tilde{Q}}(h, h) = \tilde{Q}(h, h) \quad (3-25)$$

并称隶属度 $\tilde{Q}(h, h)$ 为 (h, h) 关于模糊关系 $\tilde{Q}$ 的相关程度。

为了描述系统中任意两个主机节点间的模糊程度，引入概念主机节点间的模糊矩阵。

定义 3-7 主机节点间的模糊矩阵：设 $\tilde{Q} \in F(H \times H)$ ，若对于 $i = 1, 2, \dots, m$ ； $j = 1, 2, \dots, m$ ，都有 $q_{ij} = \tilde{Q}(h_i, h_j) \in [0, 1]$ ，则称

$$\tilde{Q} = \begin{bmatrix} q_{11} & q_{12} & \cdots & q_{1m} \\ q_{21} & q_{22} & \cdots & q_{2m} \\ \vdots & \vdots & & \vdots \\ q_{m1} & q_{m2} & \cdots & q_{mm} \end{bmatrix}$$

为 $m \times m$ 阶主机节点间的模糊矩阵。

3.3.2 软件组件间和主机节点间模糊矩阵求解

影响软件组件间关系的因素有很多，包括不确定性因素和确定性因素。为了综合考虑这些影响因素对软件组件间关系的影响，提出软件组件间模糊矩阵，其求解过程如下：

- ① 列举出要考虑的影响软件组件间模糊关系的因素，设有 m 种；
- ② 对于每种影响因素，给出其相应的隶属函数；
- ③ 设系统中包含 n 个软件组件，假设第 $i (1 \leq i \leq m)$ 种影响因素对应的模糊子集为 A_i ，求出软件组件 $C_j (1 \leq j \leq n)$ 对应的隶属度为 $\mu_{A_i}(C_j)$ ，将 n 个软件组件的隶属度值组成模糊矩阵 B_i ， $B_i = [\mu_{A_i}(C_1), \mu_{A_i}(C_2), \dots, \mu_{A_i}(C_n)]$ ；
- ④ 对于第 $i (1 \leq i \leq m)$ 种影响因素，求出任意两个软件组件间关于第 i 种影响因素的模糊矩阵 D_i ， $D_i = B_i^T \circ B_i$ ，其中 B_i^T 表示 B_i 的转置矩阵；
- ⑤ 对于模糊矩阵 $D_i (1 \leq i \leq m)$ ，求出软件组件间的模糊矩阵 $R = D_1 \cap D_2 \cap \dots \cap D_m$ 。

类似地，主机节点间的模糊矩阵的求解过程如下：

① 列举出要考虑的影响主机节点间模糊关系的因素，设有 k 种；

② 对于每种影响因素，给出其相应的隶属函数；

③ 设系统中包含 m 个主机节点，假设第 $i(1 \leq i \leq k)$ 种影响因素对应的模糊子集为 E_i ，求出主机节点 $H_j(1 \leq j \leq m)$ 对应的隶属度为 $\mu_{E_i}(H_j)$ ，将 m 个主机节点的隶属度值组成模糊矩阵 G_i ， $G_i = [\mu_{E_i}(H_1), \mu_{E_i}(H_2), \dots, \mu_{E_i}(H_m)]$ ；

④ 对于第 $i(1 \leq i \leq k)$ 种影响因素，求出任意两个主机节点关于第 i 种影响因素的模糊矩阵 P_i ， $P_i = G_i^T \circ G_i$ ，其中 G_i^T 表示 G_i 的转置矩阵；

⑤ 对于模糊矩阵 $P_i(1 \leq i \leq k)$ ，求出主机节点间的模糊矩阵 $Q = P_1 \cap P_2 \cap \dots \cap P_k$ 。

3.4 模糊贪婪部署算法

3.4.1 模糊贪婪部署算法的描述

在主机节点失效和软件组件失效的情况下，软件组件在主机节点上部署极大地影响系统可靠性。有许多因素影响系统部署架构，例如每个主机节点上可用的内存、组件大小、软件架构风格等等。对于给定的环境，确定一个最大化系统可靠性的部署架构是非常困难的。精确算法尝试每种可能的部署架构，并且至少可以找到一个最优的部署架构，但精确算法的时间复杂度是 $O(m^n)$ (m 为主机节点数， n 是软件组件数)，需要的计算成本太昂贵。因此，设计近似算法将软件组件部署到主机节点上非常重要。本章设计模糊贪婪部署算法，设系统有 m 个主机节点和 n 个软件组件，该算法根据软件组件间的模糊矩阵和主机节点间的模糊矩阵，利用贪婪算法的思想将软件组件分成 m 个模糊的软件组件子集合，再利用贪婪算法将 m 个软件子集合中的软件组件部署到相应的 m 个主机节点上。

贪婪算法每次选择在当前看起来“最好”的选择。也就是，它通过一个局部最优期望得到全局最优解决方法^[13]。根据贪婪算法思想，设计模糊贪婪部署算法，如图 3-1 所示。在图 3-1 中，系统包含 n 个软件组件 $C = \{C_1, C_2, \dots, C_n\}$ 和 m 个主机节点 $H = \{H_1, H_2, \dots, H_m\}$ ，ComSub 表示软件组件子集合的数组， MDC 表示软件组件子集合间的模糊矩阵， Q 表示主机节点间的模糊矩阵， $Div(n, m, R)$ 是将 n 个软件组件分成 m 个模糊软件组件子集合的方法，如图 3-2 所示。

Algorithm 模糊贪婪部署算法

Input Influencing factors; //Influencing host nodes and software components;

Output Deployment architecture ;

```

1  {
2      Generate fuzzy matrix  $\tilde{R}$  among software components;
3      Divide  $n$  components into  $m$  software component subsets based on  $\tilde{R}$  ,
4      /* The  $n$  is the number of software components,  $m$  is the number of host nodes;*/
5      ComSub= Div( $n, m, \tilde{R}$ );
6      Calculate fuzzy dependence degree among software component subsets, obtain the
7      fuzzy matrix  $\tilde{MDC}$  ;
9      Deploy  $m$  software components subsets on  $m$  host nodes, Deploy (  $\tilde{MDC}, \tilde{Q}$  ).
10 }
```

图 3-1 模糊贪婪部署算法的伪代码

Fig. 3-1 Pseudocode of fuzzy greedy deployment algorithm

Algorithm Div

Input $\tilde{R}$; //The fuzzy matrix of software components;

Output ComSub [] ; //Software component subsets;

```

1  {   RemainCom=C; // RemainCom represents the set of unallocated components;
2      for ( $i=1; i \leq m; i++$ ) // Initialize  $m$  software component subsets.
3          { Find the maximum value  $r_{ab}$  from  $\tilde{R}$  ;
4              Delete row  $a, b$  and column  $a, b$  from  $\tilde{R}$  ;
5              RemainCom=RemainCom-  $\{C_a\}$  -  $\{C_b\}$  ;
6              /* Software components  $C_a$  and  $C_b$  find the corresponding the subset ComSub[ $i$ ] of
7              software components;*/
9              ComSub[ $i$ ]= $\{C_a, C_b\}$  ;
10         }
11         while ( RemainCom  $\neq \phi$  ) // Check all software components have been allocated.
12             { Choose the maximum value  $r_{ed}$  from  $\tilde{R}$  ,
13                  $C_e \in \text{ComSub}[i]$  ,  $C_d \in \text{RemainCom}$  ;
14                 RemainCom=RemainCom- $\{C_d\}$  ; // Delete  $C_d$  .
15                 Delete row  $d$  and column  $d$  from  $\tilde{R}$  ;
16                 ComSub[ $i$ ]=ComSub[ $i$ ]  $\cup C_d$  ; //ComSub[ $i$ ] contains  $C_d$  ;
17             }
18     }
19 }
```

图 3-2 Div 方法的伪代码

Fig. 3-2 Pseudocode of Div method

Deploy($\tilde{MDC}, \tilde{Q}$) 将 m 个模糊软件组件子集合中的软件组件部署到相应的 m 个主

机节点上的方法，如图 3-3 所示。

在图 3-2 中，需要将 n 个软件组件分成 m 个模糊软件组件子集合， $RemainCom$ 表示没有找到对应的模糊软件组件子集合的软件组件， $ComSub[i]$ 表示第 i 个模糊软件组件子集合。对于这 m 个模糊软件子集合，用公式 (3-26) 来描述任意两个模糊软件子集合的接近关系 $R_i(x_i, x_j)$ [114]，其中， x_i 表示第 i ($1 \leq i \leq m$) 个软件组件子集合中软件组件隶属度的均值， x_j 表示第 j ($1 \leq j \leq m$) 个软件组件子集合中软件组件隶属度的均值。这里将软件组件子集合与其自身的接近关系的接近程度设为 0。

$$\psi_{R_i}(x_i, x_j) = \begin{cases} e^{-|x_i - x_j|}, & x_j > x_i, j \neq i \\ e^{-|(x_i - x_j)/2|}, & x_j \leq x_i, j \neq i \end{cases} \quad (3-26)$$

任意两个模糊软件组件子集合的接近关系的值 $\psi_{R_i}(x_i, x_j)$ ，作为第 i 个模糊软件组件子集合和第 j 个模糊软件组件子集合间的模糊相关程度，从而求出模糊软件子集合间的模糊矩阵 $\tilde{MDC}$ 。根据模糊矩阵 $\tilde{MDC}$ 和 $\tilde{Q}$ ，利用 Deploy 方法将软件组件有效地部署到主机节点上，如图 3-3 所示。

Algorithm Deploy

Input matrix $\tilde{MDC}, \tilde{Q}$;

Output Deployment architecture ; // Deployment architectures after deploying

```

1  {
2       $RH = H, \forall i (1 \leq i \leq m), CH(ComSub[i]) = \emptyset, SumCS = \bigcup_i ComSub[i]$ . //Initialization
3
4      choose the maximum value  $\tilde{mdc}_{a,b}$  from  $\tilde{MDC}$ ,
5      choose the maximum value  $q_{ef}$  from  $\tilde{Q}$ ,
6
7      /* Software components of set  $ComSub[a]$  may be deployed on host node  $H_e$  or  $H_f$  */
8       $CH(ComSub[a]) = CH(ComSub[a]) \cup \{H_e, H_f\}$ ;
9       $CH(ComSub[b]) = CH(ComSub[b]) \cup \{H_e, H_f\}$ ;
10     for  $ComSub[i] \in SumCS$ 
11
12     /*Check whether repeated host node is in set  $CH(ComSub[i])$  or not */
13     Find the duplicate host nodes in  $CH(ComSub[i])$ ,
14     /*Software components of set  $CH(ComSub[i])$  will be deployed on host node  $H_d$ , and
15     delete host node  $H_d$  from set  $RH$ . */
16     for example,  $H_d$ ,  $Del(ComSub[i], H_d)$ ;
17     for  $ComSub[y] \in SumCS$ 
18
19     /* Host node  $H_z$  and host node  $H_d$  are the candidate host nodes. But software components
20     of some set have been deployed on host node  $H_z$ . There is no software component
```

```

21  deployed on host node  $H_q$  */
22      if( ( $\exists H_z \in CH(ComSub[y])$ ) && ( $H_z \notin RH$ ) &&
23          ( $\exists H_q \in CH(ComSub[y])$ ) && ( $H_q \in RH$ ))
24          Del (ComSub[y],  $H_q$ )
25      }
    
```

图 3-3 Deploy 方法的伪代码

Figure 3-3 Pseudocode of Deploy method

在图 3-3 中， H 表示主机节点的集合 $H = \{H_1, H_2, \dots, H_m\}$ ， RH 是余下的主机节点的集合， $CH(ComSub[i])$ 描述 $ComSub[i]$ 可以被部署的候选主机节点的集合， $SumCS$ 是软件组件的集合， $mdc_{a,b}$ 表示软件组件子集合 $ComSub[a]$ 和 $ComSub[b]$ 间的模糊相关度， $q_{e,f}$ 表示主机节点 H_e 和 H_f 间的模糊相关度， MDC 是软件组件子集合间的模糊矩阵， $\tilde{Q}$ 描述主机节点间的模糊矩阵，方法 $Del(ComSub[y], H_q)$ 描述 $ComSub[y]$ 中的软件组件部署到主机节点 H_q 上，如图 3-4 所示，其中， $Deployment[ComSub[i]]$ 表示 $ComSub[i]$ 中的软件组件被部署到主机节点。

```

Algorithm Del
Input  ComSub[x],  $H_y$ ;
Output Deployment[];
1  {
2      SumCS=SumCS-ComSub[x]; //Set ComSub[x] has found the corresponding host node;
3      /*Software components of set ComSub[x] has been deployed on some host node  $H_y$ ;*/
4      Deployment [ComSub[x]]=[ $H_y$ ];
5      Delete row x and column x from  $MDC$ ;
6      Delete row y and column y from  $\tilde{Q}$ ;
7      /*Host node  $H_y$  is deleted from the set of RH. No other software component will be
8      deployed on host node  $H_y$ . /
9      RH=RH-  $\{H_y\}$ ;
10     }
11
12 }
    
```

图 3-4 Del 方法伪代码描述

Fig.3-4 Pseudocode of Del method

3.4.2 模糊贪婪部署算法的正确性和复杂度分析

(1) 模糊贪婪部署算法的正确性分析

模糊贪婪部署算法是正确的，如果模糊贪婪部署算法对于每一个输入都最终停止，而且产生正确的输出。首先分析模糊贪婪部署算法对于每个输入都停止，之后分析该算法对于每个输入都能产生正确的输出。

① 模糊贪婪部署算法对于每一个输入都能最终停止

对于某个软件组件 C_i 找到对应软件组件子集合，如果软件组件 C_i 作为某个软件组件子集合的初始化的软件组件，那么 C_i 执行 1 步便找到对应的软件组件子集合；如果 C_i 不是作为软件组件子集合的初始化软件组件，那么根据 C_i 与 m 个软件组件子集合中软件组件的模糊相关度，确定软件组件 C_i 属于哪个软件组件子集合，这个过程最多执行 m 步， C_i 便可找到对应的软件组件子集合。因此，对于 n 个软件组件中的所有 C_i 最多执行有限的 mn 步找到对应的软件组件子集合。

之后，将 m 个软件组件子集合中的软件组件部署到 m 个主机节点上，每个软件组件子集合中的软件组件只能部署到一个主机节点上，且每个主机节点上只能部署 1 个软件组件子集合中的软件组件。根据排列组合的关系可知，最多经过有限的 $m!$ 步可以将 m 个软件组件子集合中的软件组件部署到这 m 个主机节点上。

因此，对于每一种输入，模糊贪婪部署算法最多经过有限的 $mn + m!$ 步，算法将停止。

② 对于每一个输入算法都能产生正确的结果

这里不正确的结果包括两种情况：某个软件组件没有找到部署的主机节点或者某个主机节点上没有部署任何软件组件。

1) 某个软件组件没有找到部署的主机节点

假设某个软件组件 C_i 没有找到部署的主机节点。根据 Deploy 方法可知， m 个软件组件子集合中的软件组件都会部署到相应的 m 个主机节点上。也就是说，软件组件 C_i 没有找到对应的软件组件子集合，而在 Div 方法中，判断 *RemainCom* 集合为空，Div 方法才停止，若存在某个软件组件 C_i 未找到对应的子集合，Div 方法是不能停止的，因此，所有的软件组件都能部署到相应的主机节点上。

2) 某个主机节点上没有部署任何软件组件

根据 Div 方法知，初始化时， m 个软件组件子集合的每个子集合中有两个初始化的软件组件。初始化后， m 个软件组件子集合的软件组件数目是非减的，

所以每个软件组件子集中至少有两个软件组件。根据 Deploy 方法知, 这 m 个软件组件子集中的软件组件将部署到 m 个不同的主机节点上, 也就是说, 每个主机节点上至少有两个软件组件。因此, 不可能存在某个主机节点没有部署软件组件。

综上所述, 模糊贪婪部署算法对于每个输入, 算法都将最终停止, 并且输出正确的结果, 因此模糊贪婪部署算法是正确的。

(2) 模糊贪婪部署算法的时间复杂度分析

设系统有 n 个软件组件和 m 个主机节点, 这里软件组件数至少和主机节点数一样多 ($n \geq m$), 否则将有某个主机节点上没有部署软件组件。产生 n 个软件组件间的 $n \times n$ 阶模糊关系矩阵 $\underline{R}$ 需要花费时间 $O(n^2)$, 采用 Div 方法将 n 个软件组件分成 m 个软件组件子集需要花费时间 $O(n^2 \log n)$, 采用 Deploy 方法将 m 个软件组件子集中的软件组件部署到 m 个主机节点上需要花费时间 $O(m^2 \log m)$, 因此, 模糊贪婪部署算法需要的总时间复杂度为 $O(n^2 + n^2 \log n + m^2 \log m) = O(n^2 \log n)$ 。

3.4.3 实验及结果分析

实验系统包括 4 个主机节点和 14 个软件组件。实验输入包括软件组件失效率、主机节点失效率、影响系统部署的因素、主机节点间的模糊矩阵、软件组件间的模糊矩阵。利用模糊贪婪部署算法将 14 个软件组件部署到 4 个主机节点上, 并与自适应贪婪算法^[115]得到的部署架构的可靠性和时间复杂度进行比较。

(1) 输入

psh_i 表示软件组件 C_i 的失效率, 为 $[0, 0.02]$ 间的任意实数, 如表 3-1 所示。 ph_i 表示主机节点 H_i 的失效率, 为 $[0, 0.008]$ 间的任意实数, 如表 3-2 所示。

表 3-1 软件组件失效率
Table 3-1 Failure probabilities of software components

参数	psh_1	psh_2	psh_3	psh_4	psh_5	psh_6	psh_7
值	0.0004	0.0011	0.0007	0.0009	0.0013	0.0052	0.0021
参数	psh_8	psh_9	psh_{10}	psh_{11}	psh_{12}	psh_{13}	psh_{14}
值	0.0108	0.0012	0.002	0.0078	0.0019	0.0184	0.013

表 3-2 主机节点的失效率
Table 3-2 Failure probabilities of host nodes

参数	ph_1	ph_2	ph_3	ph_4
----	--------	--------	--------	--------

值	0.0075	0.0038	0.0019	0.0003
---	--------	--------	--------	--------

影响系统部署的因素包括组件大小、每个主机节点的可用内存、系统工程师的知识、每个软件组件所需内存、网络可靠性、软件组件位置约束、软件组件间交互频率、软件架构师的知识、软件架构风格等等。其中，影响主机节点间模糊关系的因素包括主机节点的可用内存、主机节点的可靠性、网络可靠性、软件架构师的知识、系统工程师的知识等，在求解主机节点间的模糊矩阵 $\tilde{Q}$ 时，其影响因素的模糊分布采用 2 次抛物线型的分布。

$$\tilde{Q} = \begin{bmatrix} 0 & 0.1728 & 0.7272 & 0.1 \\ 0.2051 & 0 & 0.17 & 0.6249 \\ 0.5530 & 0.2020 & 0 & 0.2450 \\ 0.0507 & 0.1442 & 0.8051 & 0 \end{bmatrix}$$

影响软件组件间模糊关系的因素包括软件组件的大小、软件组件需要的内存、软件组件间交互频率、软件组件的失效率、软件架构师的知识、系统工程师的知识、软件架构风格等，在求解软件组件间的模糊矩阵 $\tilde{R}$ 时，其影响因素的模糊分布采用 3 次抛物线型分布。

$$\tilde{R} = \begin{bmatrix} 0 & 0.2168 & 0.3097 & 0.4439 & 0.8133 & 0.1363 & 0.1351 & 0.9776 & 0.0550 & 0.3580 & 0.4256 & 0.3167 & 0.1630 & 0.4796 \\ 0.2319 & 0 & 0.2916 & 0.1443 & 0.6271 & 0.7129 & 0.2625 & 0.0272 & 0.1824 & 0.3738 & 0.4762 & 0.3499 & 0.8021 & 0.2113 \\ 0.4856 & 0.0528 & 0 & 0.6376 & 0.9275 & 0.2690 & 0.2875 & 0.0294 & 0.3931 & 0.4877 & 0.6562 & 0.0084 & 0.1990 & 0.5459 \\ 0.6522 & 0.8481 & 0.3743 & 0 & 0.1121 & 0.2207 & 0.0609 & 0.3576 & 0.2154 & 0.0910 & 0.1962 & 0.3969 & 0.2293 & 0.3911 \\ 0.5568 & 0.3518 & 0.8068 & 0.2999 & 0 & 0.6170 & 0.4960 & 0.3134 & 0.2998 & 0.3382 & 0.1628 & 0.2334 & 0.3371 & 0.2411 \\ 0.4787 & 0.7937 & 0.0974 & 0.3516 & 0.3438 & 0 & 0.1863 & 0.3109 & 0.0768 & 0.5149 & 0.6133 & 0.0850 & 0.3683 & 0.0929 \\ 0.6222 & 0.0683 & 0.2584 & 0.2688 & 0.8995 & 0.2816 & 0 & 0.3100 & 0.3525 & 0.2077 & 0.6505 & 0.2376 & 0.5175 & 0.1870 \\ 0.9442 & 0.6544 & 0.1930 & 0.6302 & 0.0034 & 0.0178 & 0.0134 & 0 & 0.7045 & 0.0682 & 0.2886 & 0.4148 & 0.6705 & 0.8445 \\ 0.4927 & 0.1102 & 0.3333 & 0.5159 & 0.3155 & 0.3299 & 0.1233 & 0.7207 & 0 & 0.4250 & 0.0311 & 0.9697 & 0.5206 & 0.1595 \\ 0.6437 & 0.6114 & 0.2744 & 0.2110 & 0.4010 & 0.3796 & 0.1567 & 0.3965 & 0.1557 & 0 & 0.6799 & 0.4138 & 0.6609 & 0.4714 \\ 0.2265 & 0.3066 & 0.3974 & 0.6461 & 0.2945 & 0.9413 & 0.6171 & 0.9992 & 0.0074 & 0.2216 & 0 & 0.4688 & 0.3710 & 0.2017 \\ 0.0133 & 0.1252 & 0.0429 & 0.5365 & 0.3928 & 0.5802 & 0.5893 & 0.1348 & 0.7221 & 0.1611 & 0.2280 & 0 & 0.6416 & 0.6913 \\ 0.4675 & 0.1662 & 0.4393 & 0.1633 & 0.0059 & 0.4496 & 0.2938 & 0.2233 & 0.6685 & 0.6382 & 0.6585 & 0.2632 & 0 & 0.4120 \\ 0.9096 & 0.1311 & 0.8386 & 0.3099 & 0.6820 & 0.6434 & 0.3697 & 0.1326 & 0.5799 & 0.6641 & 0.6711 & 0.4820 & 0.4869 & 0 \end{bmatrix}$$

(2) 实验结果分析

由于系统包含 4 个主机节点，因此，14 个软件组件被分为 4 个软件组件子集合。通过 Div 方法得到软件组件子集合 1 为 $\{C_1, C_2, C_7, C_{11}, C_{13}, C_{14}\}$ ，软件组件子集合 2 为 $\{C_4, C_8, C_{10}\}$ ，软件组件子集合 3 为 $\{C_3, C_5\}$ 和软件组件子集合 4 为

$\{C_6, C_9, C_{12}\}$ 。根据公式 (3-26)，得到 4 个软件组件子集合的接近关系，这里将这种接近程度值看作为 4 个软件组件子集合间的模糊相关度。4 个软件组件子集合的模糊相关度如图 3-5 所示。根据模糊贪婪部署算法的 Deploy 方法将 4 个软件组件子集合中的软件组件部署到 4 个主机节点上，如图 3-6 所示。软件组件 C_3 和 C_5 部署到主机节点 H_1 上，软件组件 C_6 、 C_9 和 C_{12} 部署到主机节点 H_2 上，软件组件 C_4 、 C_8 和 C_{10} 部署到主机节点 H_3 上，软件组件 C_1 、 C_2 、 C_7 、 C_{11} 、 C_{13} 和 C_{14} 部署到主机节点 H_4 上。

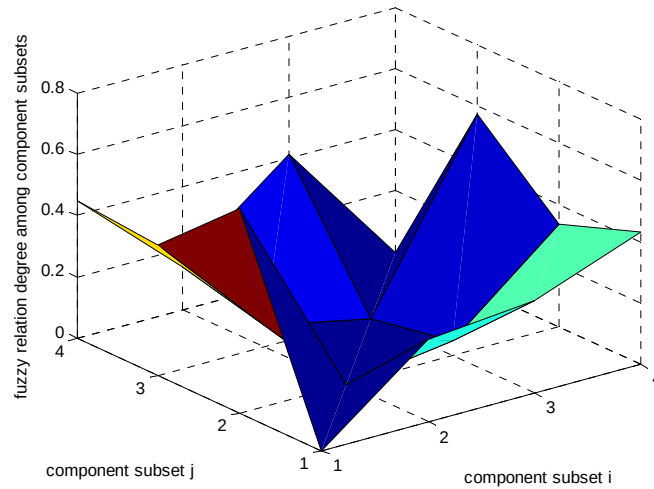


图 3-5 软件组件子集合间的模糊相关度

Fig. 3-5 Fuzzy relation degree among software component subsets

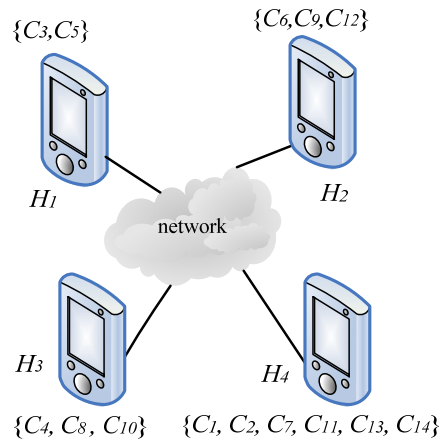


图 3-6 运用模糊贪婪部署算法生成的部署架构

Fig. 3-6 Deployment architecture by fuzzy greedy deployment algorithm

自适应贪婪算法试着选择“最好”的主机节点和“最好”的软件组件。“最好”的主机节点是指与系统中其它主机节点的网络可靠性之和最高，并且余下

的可用内存最大。“最好”的软件组件是指与系统中其它软件组件交互频率之和最高，自适应贪婪算法的复杂度为 $O(n^3)$ (n 为软件组件数目)。模糊贪婪部署算法不仅考虑了网络可靠性、内存容量和软件组件间交互频率的影响，也考虑了组件大小、软件架构风格的影响。模糊贪婪部署算法将多维影响因素转换为软件组件间模糊矩阵和主机节点间的模糊矩阵，并将软件组件部署到主机节点上，算法的复杂度为 $O(n^2 \log n)$ 。两种算法的所得到部署架构和系统可靠性如表 3-3 所示。

表 3-3 两种算法的部署架构和可靠性分析

Table 3-3 Deployment architecture and reliability analysis of two algorithms

算法	H_1	H_2	H_3	H_4	系统可靠性
自适应贪婪 算法	C_1, C_{14}	$C_3, C_4,$ C_5, C_7	C_9, C_{12}	$C_2, C_6, C_8,$ C_{10}, C_{11}, C_{13}	0.9021
模糊贪婪部署 算法	C_3, C_5	C_6, C_9, C_{12}	$C_4, C_8,$ C_{10}	$C_1, C_2, C_7,$ C_{11}, C_{13}, C_{14}	0.9038

从表 3-3 可知，模糊贪婪部署算法比自适应贪婪算法能得更高可靠性的部署架构。因此，与自适应的贪婪算法相比，模糊贪婪部署算法找到的部署架构的可靠性提高 0.2%，时间复杂度从 $O(n^3)$ 降到 $O(n^2 \log n)$ 。

3.5 本章小结

本章首先介绍模糊关系的相关概念、模糊子集的表示方法和常见的模糊分布。之后，引入模糊理论将这多维的影响因素转化为软件组件间的模糊关系与主机节点间的模糊关系。提出软件组件间模糊矩阵和主机节点间模糊矩阵的概念，并给出软件组件间模糊矩阵和主机节点间模糊矩阵的求解方法。根据主机节点间的模糊矩阵和软件组件间的模糊矩阵，设计模糊贪婪部署算法，该算法利用贪婪算法的思想，将系统中的 n 个软件组件分成 m 个软件组件子集合，并将这 m 个子集合中的软件组件部署到 m 个主机节点上。对模糊贪婪部署算法进行正确性和时间复杂度分析。之后，给出包含 4 个主机节点和 14 个软件组件的实验，实验结果表明，与自适应贪婪算法相比，模糊贪婪部署算法不仅能找到更好的部署架构，找到的部署架构的可靠性提高约 0.2%，而且能够降低系统时间复杂度($O(n^3)$ 降到 $O(n^2 \log n)$)，节约很多处理时间。

第 4 章 基于相关性的组件重部署算法

4.1 引言

由于移动、普适和分布式计算环境的内在动态性和不可预测性的特点，主机节点和软件组件可能会不可预见地出现故障，从而导致系统性能降低。如果主机节点失效，那么部署在失效的主机节点上的软件组件不能向其它主机节点上的软件组件提供服务，或者向其它主机节点上的软件组件请求服务，即部署在失效的主机节点上的软件组件也失效。在主机节点失效和软件组件失效的情况下，如何将软件组件部署到主机节点上极大地影响系统可靠性。此外，在系统运行期间，影响系统部署架构的因素可能会发生变化，因此，对于当前的系统运行环境，当前正在执行的部署架构可能不是最好的，需要系统重部署来提高可靠性。如果系统有 n 个软件组件和 m 个主机节点，那么找到最大化系统可靠性的部署架构是指数复杂性问题，需要花费时间 $O(m^n)$ ，需要设计近似算法来重部署系统。Medvidovic^[16]举例说明部署架构对系统延迟的影响，并给出可用性、延迟、能量消耗、内存限制的函数描述，构造一个多维优化问题。然而，多维优化问题的精确评估是非常困难的事情。Mikic-Rakic^[13]设计 Avala 近似算法来重部署系统从而提高系统可用性。Avala 算法将软件组件和主机节点标志成不同的级别，根据标志的级别将软件组件部署到主机节点上。然而，Avala 算法通常找到的是近似优化的部署架构。

4.2 软件组件间的依赖关系和相关性

(1) 软件组件间的依赖关系

研究者们从不同的问题出发，对软件组件间相互依赖关系提出了不同的分类方法。Huang^[116]认为在基于组件方式的软件开发中，软件组件间可存在着数据依赖与控制依赖这两种关系。对于不同组件 C_1 与 C_2 ，如果 C_2 可直接使用 C_1 产生的计算结果，则此两者存在着数据依赖。而如果 C_1 的计算结果会决定是否执行 C_2 ，则此两者存在着控制依赖。

Chen^[117]由依赖关系产生的时机分析，将软件组件之间的依赖关系分为静态依赖与动态依赖，如果在设计过程就已确定下来的依赖可称为静态依赖；而那些仅在运行时刻才发生、并通过调用方式存在的依赖关系可称为动态依赖。

Judith^[118]提出软件组件间的依赖关系应分为结构依赖与行为依赖。结构依

赖包括文本包含依赖和继承依赖。文本包含依赖是指软件组件规范可由多个语句结合的源模型产生。继承依赖是指软件组件规范可从其它源模块继承产生。行为依赖包括时序依赖、状态依赖、因果依赖和输入/输出依赖。时序依赖是指某个软件组件的行为会在另一软件组件的某行为之前或者之后发生。状态依赖是指某个软件组件的行为仅在系统或其子系统处于某一特殊的状态时才会发生。因果依赖是指某个软件组件的某行为发生将意味着另一软件组件的另一行为会发生。输入/输出依赖是指某个软件组件使用另一软件组件所提供的信息，或是向另一个软件组件提供一些信息。

Binkley^[119]将依赖关系主要分为三种，基于引用的依赖、组成结构依赖和数据完整性依赖。基于引用的依赖关系是指某个软件组件的运行成功与否将取决于其对另一个软件组件的产生的声明是否发生了某些改变。组成结构依赖关系是指某个软件组件的运行成功与否将取决于处于其内部组件的组织结构或者相对位置是否发生了某些改变。数据完整性依赖关系是指某个软件组件的运行成功与否将取决于共享数据的完整与否。基于引用的依赖关系大量存在于基于组件的应用系统中。组成结构依赖是描述由于某个软件组件频繁使用其它组件，故而在结构上将它们视为一个整体考虑。数据完整性依赖描述的是两个软件组件因共享数据从而存在的依赖关系^[120]。

(2) 软件组件间的相关性

前面的软件组件间依赖关系主要根据软件故障、数据、状态等来区分。在分布式、移动和普适环境中，大量的软件组件运行在开放的、高度动态的、多变的环境中，系统可增加软件组件和删除软件组件。在系统运行期间，软件组件需要给其它软件组件提供服务，或者向其它软件组件请求服务，部署时将交互频繁的软件组件尽量部署到同一个主机节点上。为了更简明地描述软件组件间的关系，本章将根据软件组件是否有交互关系和交互的重要性引入软件组件间的相关性和软件组件间的相关度矩阵的概念。软件组件间的相关性可用一个图来表示，图 $G = (C, E, W)$ 是一个有向图， C 为软件组件集合， $C = \{C_1, C_2, \dots, C_n\}$ ，并且 $|C| = n$ ， E 表示软件组件间交互的集合， W 表示软件组件间交互的重要性。

定义 4-1 软件组件间的相关性：给定有向图 $G = (C, E, W)$ ，如果 $\exists C_i, C_j \in C$ ， $(C_i, C_j) \in E$ ，则称软件组件 C_i 和软件组件 C_j 是相关的。

定义 4-2 软件组件间的相关度矩阵：给定一个矩阵 $E = (e_{ij})_{n \times n}$ ，其中 $e_{ij} = f_{ij} \times w_{ij}$ ， f_{ij} 表示软件组件 C_i 和 C_j 间的交互次数， w_{ij} 表示软件组件 C_i 和 C_j 间的单次交互的重要性，将矩阵 E 正规化为 $[0, 1]$ 间的实数矩阵 CC ， CC 称为软件组件间的相关度矩阵。

$$E = \begin{matrix} & C_1 & \cdots & C_n \\ \begin{matrix} C_1 \\ \vdots \\ C_n \end{matrix} & \begin{bmatrix} e_{11} & \cdots & e_{1n} \\ \vdots & \cdots & \vdots \\ e_{n1} & \cdots & e_{nn} \end{bmatrix} \end{matrix}$$

4.3 已有部署算法

当系统可靠性低于某一规定值时，可通过重部署来提高系统可靠性。已存在的系统部署算法包括精确算法、随机算法和 Avala 算法。为了清楚地描述这些算法的部署过程，首先给出部署架构的数据结构描述，如图 4-1 所示。

```
struct DeployArchitecture
{int flag; // Check the state of deployment architecture;
 double systemReliability; // System reliability;
 int delComWhichHost; //Subscript of the host node;
 int[] comDeployedHost[n]; //Deployment situation
}
```

图 4-1 部署架构的数据结构

Fig. 4-1 Data structure of deployment architecture

flag 用来标识当前部署架构所处的状态，*flag* 的值可为 1、0 或者-1。如果部署架构已经被重部署，则 *flag* 的值为 0。如果部署架构可以被重部署，但还未进行重部署，则 *flag* 的值为 1。如果部署架构不能被重部署，则 *flag* 的值为-1。*systemReliability* 描述部署架构的系统可靠性。*delComWhichHost* 描述选定且将被部署软件组件来自的主机节点。*n* 表示系统中软件组件的数目。*comDeployedHost* 描述软件组件在主机节点上的分配，若 *comDeployedHost*[2] 的值为 3，表示软件组件 C_2 部署到主机节点 H_3 上。

(1) 精确算法

此算法扩展所有的系统部署架构，计算所有可能部署架构的系统可靠性，从中选择有最大可靠性的部署架构。精确算法至少可以找到一个最优的部署架构，其算法的复杂度为 $O(m^n)$ (m 是主机节点数， n 是软件组件数)。可以看出，即使主机节点数和软件组件数都非常小，计算代价仍很昂贵。

(2) 随机算法

系统初始化时，求出软件组件与其它软件组件的相关度之和，之后将这 n 个软件组件按相关度之和从小到大排序。系统初始化后，随机算法选择具有最低相关度之和的软件组件，随机选择一种 *flag* 为 1 的部署架构进行重部署。对于新产生的部署架构中，随机地选择一个软件组件和随机选择一种 *flag* 为 1 的部

署架构继续重部署，算法重复一定的次数，直到新产生的部署架构的 *flag* 值都为-1，算法结束，余下的 *flag* 值为 0 的部署架构就是得到的“较好”的部署架构。

系统初始化过程，将 n 个软件组件按照相关度之和进行排序，需要花费时间为 $O(n\log n)$ ，从 n 个软件组件随机选择一个软件组件和从 m 种部署架构随机选择一种部署架构进行重部署，这个过程需要花费时间 $O(nm)$ ，这个过程最多重复 n 次便可以选择出一种部署架构，需要花费时间 $O(n^2m)$ ，因此该算法的总时间复杂度为 $O(n\log n + n^2m) = O(n^2m)$ 。该算法每次随机地选择一种部署架构进行重部署，那么选择的部署架构的可靠性可能很高，也可能很低，虽然该算法的计算时间较少，但不能保证每次都得到较高可靠性的部署架构，用随机算法选择“较好”部署架构不够稳定。

(3) Avala 算法

Avala 算法将软件组件和主机节点标志成不同的级别，根据标志的级别将软件组件部署到主机节点上。系统初始化后，Avala 算法选择在当前看来级别“最高”的部署架构^[13]进行重部署，级别“最高”的部署架构需要满足两个条件：

① *flag* 值为 1；② 系统可靠性最大。扩展当前级别“最高”的部署架构，在新产生的部署架构中，继续选择级别“最高”的部署架构进行重部署，算法重复一定的次数，当新产生的部署架构的 *flag* 值都为-1 时，算法停止。则余下的 *flag* 值为 0 的部署架构是最优的。

系统初始化过程，将 n 个软件组件按照相关度之和进行排序，需要花费时间 $O(n\log n)$ ，从 n 个软件组件中选择出具有最小相关度之和的软件组件和从 m 种部署架构选择具有最高可靠性的部署架构进行重部署，这个过程需要花费时间 $O(nm\log n\log m)$ ，这个过程最多重复 n 次便可以选择出一种好的部署架构，需要花费时间为 $O(n^2m\log n\log m)$ ，因此该算法的总时间复杂度为 $O(n\log n + n^2m\log n\log m) = O(n^2m\log n\log m)$ 。

与精确算法相比，Avala 算法不一定保证每次都能好到最好的部署架构，但是计算时间比精确算法要少得多；与随机算法相比，Avala 算法虽然计算时间较多，但稳定地找到较高可靠性的部署架构。

4.4 m 优先选择算法

4.4.1 m 优先选择算法描述

A*算法结合了深度优先搜索和广度优先搜索的优点。根据评估函数，从当

前可用的节点选择最好的进行扩展。如果 A*算法能找到解，那么一定是优化解^[113]。如果将 A*算法直接应用到部署过程中，由于系统中的主机节点和软件组件数目较多，A*算法不断重新部署系统，直到找到一个最高可靠性的部署架构。若系统有 n 个软件组件和 m 个主机节点，在最坏的情况下，A*算法找到最优部署架构所需时间为 $O(m^n)$ ，所需的计算时间太昂贵。为了降低 A*算法在部署过程中的时间复杂度，又能找到比 Avala 算法更稳定更高可靠性的部署架构，将 A*算法改进，设计 m 优先选择算法(m preferred selection, 简称 MPS)。m 优先选择算法不是扩展当前层次的所有部署架构，而是从中首次选择出具有较高可靠性的 $2m$ 种部署架构进行扩展，详细的算法描述如图 4-2 所示。

```

Algorithm  m 优先选择
Input  matrix CC[n][n];
Output optimal deployment architecture ;
1  InitializeDeploy (CC); // CC is the dependence degree matrix of software components;
2  {Select the deployment architecture that has maximum system reliability, DA_M;
3  Choose the next component that will be deployed,
4  if the number of remained software components is more than one,
5  then x=ChooseComponent (delComWhichHost);
6  else break; // The algorithm stop;
7  Deploy the selected component on other host nodes,
8  Reploy(DAs, DA_M, x); // DAs is the set of deployment architectures.
9  Prune the existed deployment architectures, Prune(DAs);
10 Sort the remained deployment architectures, Sort(DAs);
11 If the value of flag of some deployment architecture is 1, goto Step 2;
12 The remained deployment architecture with flag 0 is the optimal one.
13 }
    
```

图 4-2 系统部署的 m 优先选择算法的伪代码

Fig. 4-2 Pseudocode of m preferred selection algorithm of system deployment

在图 4-2 中， m 优先选择算法中包括初始化部署架构的 InitializeDeploy 方法、选择下一次将被部署的软件组件来自哪个主机节点的 ChooseComponent 方法、将选定的软件组件在其它主机节点上进行重部署的 Reploy 方法、剪掉不可能成为最优的部署架构的 Prune 方法和将部署架构按照可靠性顺序进行排序的 Sort 方法。

由于系统重部署过程中，会扩展出很多新的部署架构，系统的计算时间会增加很多。因此， m 优先选择算法在部署过程中，增加了部署架构的剪枝，如图 4-3 所示。

```

Algorithm Prune
Input  DeployArchitecture DAs;
Output DeployArchitecture DAs // Delete some deployment architectures ;
1  { optimal=new DeployArchitecture();
2    for (int i=0; i< DAs.size(); i++) // Check the state of all deployment architectures;
3    {
4      if (DAs.get( i ).flag==-1) // The deployment architecture will not be redeployed;
5        DAs.remove( i );// Delete the deployment architecture;
6      if (DAs.get( i ).flag==0)
7        if (DAs.get( i ).systemReliability > optimal.systemReliability)
8          /* Generate the new optimal deployment architecture*/
9            optimal=DAs.get( i );
10       else DAs.remove( i );
11    }}

```

图 4-3 Prune 方法的伪代码

Fig. 4-3 Pseudocode of Prune method

在图 4-3 中,如果部署架构的 *flag* 值为-1,表明这个部署架构已经被部署过,并且具有较低的可靠性,因此,删除这个部署架构;如果部署架构的 *flag* 值为0,并且可靠性高于已有的“最优”的部署架构,则这个部署架构更换为“最优”的部署架构,否则删除这个部署架构。

```

Algorithm InitializeDeploy
Input  matrix CC[n][n];
Output DeployArchitecture DAs ; // Initialized deployment architectures;
1  {Initialize failure probabilities of  $n$  software components and failure probabilities of  $m$ 
2    host nodes;
3    for each software component  $C_i \in C$ 
4      Calculate the sum dependence degree of  $C_i$  with other software components;
5      Sort the sum dependence degree of software components in ascending order;
6      Select the software component that has the minimum sum of dependence degree every
7      time, and select  $m-1$  software components;
8      Deploy  $m-1$  software components on other  $m-1$  host nodes;
9      Set related information for each deployment architecture;
10 }

```

图 4-4 InitializeDeploy 方法的伪代码

Fig. 4-4 Pseudocode of InitializeDeploy method

图 4-4 为系统部署的初始化过程,每次选择与其它的软件组件的相关度之和最小的软件组件,之后在软件组件的相关度矩阵中删除这个软件组件所在的行或者列。这个过程重复 $m-1$ 次,选择出 $m-1$ 个软件组件。将这 $m-1$ 个软件组

件部署到其它的主机节点上，为每个新产生的部署架构设置相关的信息，例如 *flag* 值、部署架构的可靠性等等。

```

Algorithm  Reploy
Input  DeployArchitecture DAs;
Output DeployArchitecture DAs ; //Newly generated deployment architectures;
1  { DA_M.flag=0; // Mark the deployment architecute has been deployed;
2    DeployArchitecture newDA=new DeployArchitecture();
3    int [ ] cdh=DA_M.comDeployedHost;
4    int [ ] newcdh=new int [n];
5    int a=DA_M.delComWhichHost; // Subscript of some host nodes;
6    for each host node  $H_b$ ( $H_a$  does not equal to  $H_b$ )
7      {
8        Deploy Cx on  $H_b$ ; // Software component Cx;
9        /*Record the deployment state of software components deployed on host nodes*/
10       arraycopy(cdh, 0, newcdh, 0, x-1);
11       arraycopy(cdh, x+1, newcdh, x+1, n-x);
12       newcdh[x]=b; // Cx has been deployed on  $H_b$ ;
13       calculate system reliability of new deployment architecture, Rel;
14       if (DA_M.systemReliability > Rel)
15         newDA.flag=-1;
16       /* Set related information for the newly generated deployment architectures*/
17       else { newDA.flag=1;
18             newDA.delComWhichHost=a;
19             newDA.systemReliability=Rel;
20             newDA.comDeployedHost=newcdh;
21           }}
22     DAs.add(0, newDA);
23   }

```

图 4-5 Reploy 方法的伪代码

Fig.4-5 Pseudocodoe of Reploy method

在图 4-5 中， a 是主机节点的下标，表示将要被重新部署的软件组件 C_x 来自主机节点 H_a 。将软件组件 C_x 部署到其它的主机节点上，扩展出新的部署架构，之后为新扩展出的部署架构设置相关的信息。

4.4.2 m 优先选择算法的正确性和复杂度分析

(1) m 优先选择算法的正确性分析

一个算法是正确的，如果它对于每一个输入都最终停止，而且产生正确的输出。首先分析对于每个输入算法都最终停止，之后分析对于每个输入都能产

生正确的输出。

① 对于每一个输入算法都能最终停止

选定一个将被部署的软件组件 C_i ， C_i 将被部署到其它 $m-1$ 个中某个主机节点上，进行有限的 $m-1$ 步，就能确定 C_i 将被部署到的主机节点。对于有限的 n 个软件组件，每个软件组件的部署过程都进行有限的 $m-1$ 步选择，便能确定部署到合理的主机节点上。也就是说最多经过有限的 $(m-1)^n$ 步，所有的软件组件将部署到相应的主机节点上，算法将停止。

② 对于每一个输入算法都能产生正确的结果

这里不正确的结果包括两种情况：某个软件组件没有找到部署的主机节点和某个主机节点上没有部署软件组件。

1) 某个软件组件没有找到部署的主机节点

根据算法可知，软件组件有两种情况，要么被选择出来重新部署到其它主机节点上；要么留在原来的主机节点上。所以，不可能存在某个软件组件没有找到部署的主机节点。

2) 某个主机节点上没有部署软件组件

在部署初始化时，假设从主机节点 H_i 上选择出软件组件来初始化到其它的 $m-1$ 个主机节点，那么初始化后，每个主机节点至少有一个软件组件。在系统部署过程中，这 $m-1$ 个主机节点上的软件组件数目是非减的。对于主机节点 H_i ，当部署在其上的软件组件数目为 1 时，算法停止。因此，每个主机节点上至少会部署一个软件组件，不存在某个主机节点上没有部署软件组件。

综上所述， m 优先选择算法对于每个输入，算法都将最终停止，并且输出正确的结果，因此 m 优先选择算法是正确的。

(2) m 优先选择算法的复杂度分析

系统初始化过程，将 n 个软件组件按照相关度之和进行排序，需要花费时间 $O(n \log n)$ ，从 n 个软件组件中选择出具有最小相关度之和的软件组件和从 m 种部署架构选择具有最高可靠性的部署架构进行重部署，这个过程需要花费时间 $O(nm \log n \log m)$ ，这个过程最多重复 n 次便可以选择出一种部署架构，需要花费时间 $O(n^2 m \log n \log m)$ 。 m 优先选择算法用广度优先算法首次选择出 $2m$ (m 为主机节点数) 个具有较高可靠性的部署架构，初始部署后选择的 $2m$ 种架构都进行这样的扩展，需要花费时间 $O(n^2 m^2 \log n \log m)$ 。因此， m 优先选择算法时间复杂度为 $O(n \log n + n^2 m^2 \log n \log m) = O(n^2 m^2 \log n \log m)$ 。

4.4.3 m 优先选择算法的实例说明

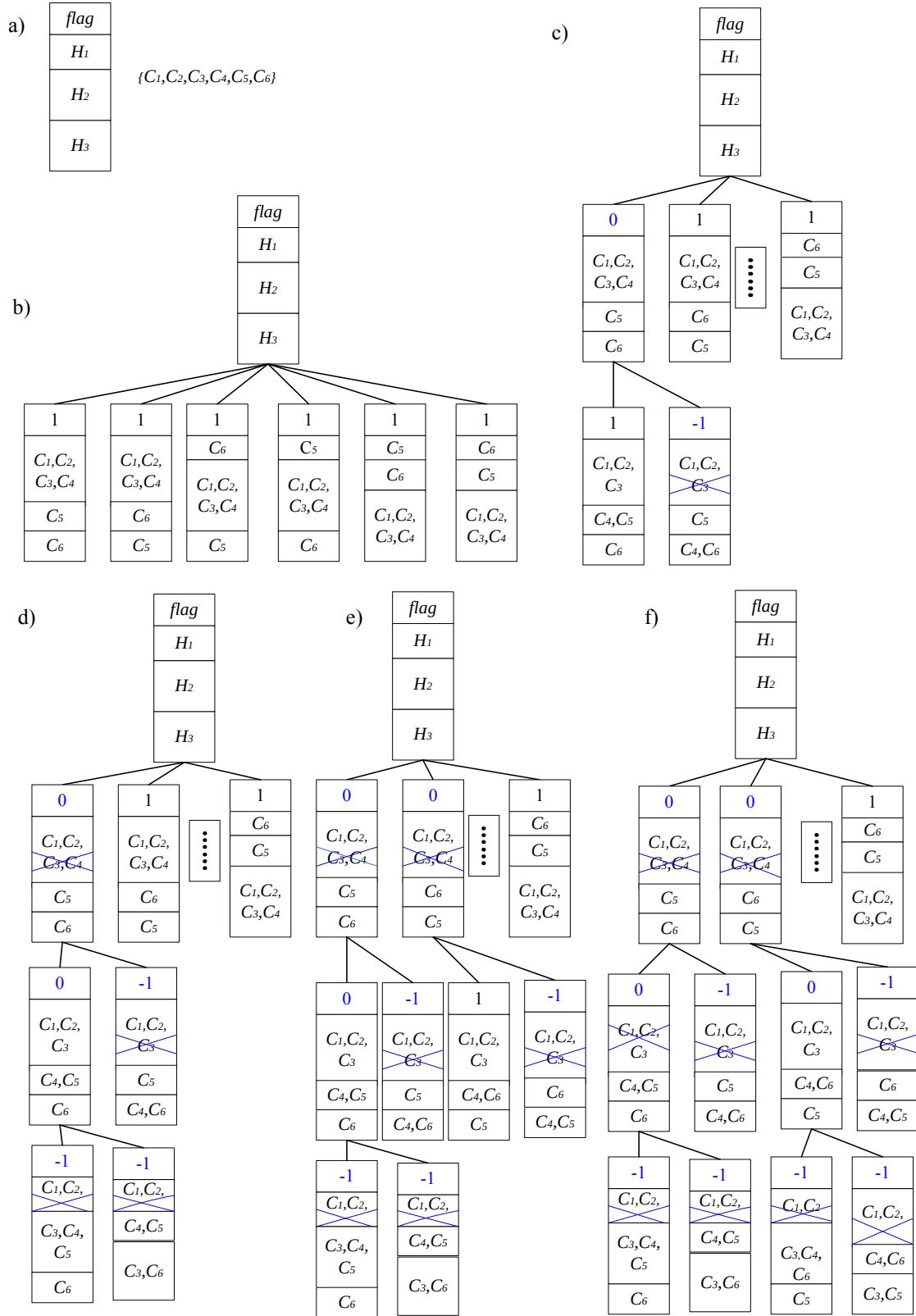
为了更好的解释 m 优先选择算法，考虑一个简单系统。该系统包括 3 个主机节点 $\{H_1, H_2, H_3\}$ 和 6 个软件组件 $\{C_1, C_2, C_3, C_4, C_5, C_6\}$ ，如图 4-6 a) 所示。

$flag$ 用来描述当前部署架构的状态。这 6 个软件组件将被部署到主机节点 H_1, H_2 或者 H_3 。根据软件组件相关度矩阵，每次选择具有与其它软件组件的相关度之和最小的软件组件，这个过程重复 $m-1$ 次 (m 是主机节点数)。 $m-1$ 个软件组件将被部署到其他的 $m-1$ 个主机节点上。在实例系统中，假定按照软件组件与其它软件组件的相关度之和从小到大排序为 C_6, C_5, C_4, C_3, C_2 和 C_1 ，选择 C_5 和 C_6 来初始化部署过程。

图 4-6 b) 为系统初始化的结果。为了阐述方便，设图 4-6 a) 为根节点，它的层数为 0。用 $DA_{i,j}$ 表示第 i 层的第 j 个部署架构，用 $RDA_{i,j}$ 表示部署架构 $DA_{i,j}$ 的系统可靠性。由根节点产生 6 个新的部署架构， $DA_{1,1}, DA_{1,2}, DA_{1,3}, DA_{1,4}, DA_{1,5}$ 和 $DA_{1,6}$ ，这些部署架构的层数为 1。设这 6 个部署架构的系统可靠性从大到小的顺序为： $RDA_{1,1} > RDA_{1,2} > RDA_{1,3} > RDA_{1,4} > RDA_{1,5} > RDA_{1,6}$ 。

根据 m 优先选择算法，选择部署架构 $DA_{1,1}$ 来重部署，如图 4-6 c) 所示。将要被部署的软件组件为 C_4 。因为 $DA_{1,1}$ 已被重部署，所以 $DA_{1,1}$ 的 $flag$ 值变为 0。由 $DA_{1,1}$ 扩展出来的部署架构为 $DA_{2,1}$ 和 $DA_{2,2}$ 。这里假设 $RDA_{2,2} < RDA_{1,1}$ 和 $RDA_{2,1} > RDA_{1,1}$ ，由于新扩展出来的部署架构 $DA_{2,2}$ 的可靠性低于 $DA_{1,1}$ ， $DA_{2,2}$ 的 $flag$ 值设为 -1。由于 $DA_{2,1}$ 的可靠性值高于 $DA_{1,1}$ ， $DA_{2,1}$ 的 $flag$ 值设为 1， $DA_{2,1}$ 可以继续被扩展。由于 $DA_{1,1}$ 已经被扩展过，它 $flag$ 值变为 0。因为目前只有部署架构 $DA_{1,1}$ 的 $flag$ 值为 0，所以 $DA_{1,1}$ 为当前“最优”的部署架构。根据 Prune 方法，减掉部署架构 $DA_{2,2}$ 。余下的未扩展的部署架构的可靠性从大到小的顺序变为： $RDA_{2,1} > RDA_{1,2} > RDA_{1,3} > RDA_{1,4} > RDA_{1,5} > RDA_{1,6}$ 。

在图 4-6 d) 中，选择部署架构 $DA_{2,1}$ 来重部署，选定的软件组件为 C_3 ，扩展 $DA_{2,1}$ 后得到部署架构 $DA_{3,1}$ 和 $DA_{3,2}$ 。由于 $DA_{2,1}$ 已经被扩展，所以 $DA_{2,1}$ 的 $flag$ 值变为 0。这里假设 $RDA_{3,1} < RDA_{2,1}$ 和 $RDA_{3,2} < RDA_{2,1}$ ，则部署架构 $DA_{3,1}$ 的 $flag$ 值为 -1，部署架构 $DA_{3,2}$ 的 $flag$ 值为 -1。根据 Prune 方法，由于 $DA_{3,1}$ 和 $DA_{3,2}$ 的 $flag$ 值都为 -1，减掉部署架构 $DA_{3,1}$ 和 $DA_{3,2}$ ，另外，由于 $DA_{2,1}$ 的 $flag$ 值变为 0， $RDA_{2,1} > RDA_{1,1}$ ，并且 $DA_{1,1}$ 的 $flag$ 值为 0，根据 Prune 方法剪掉部署架构 $DA_{1,1}$ ，当前“最优”部署架构变为 $DA_{2,1}$ 。则余下的未被扩展的部署架构的系统可靠性从大到小的顺序变为



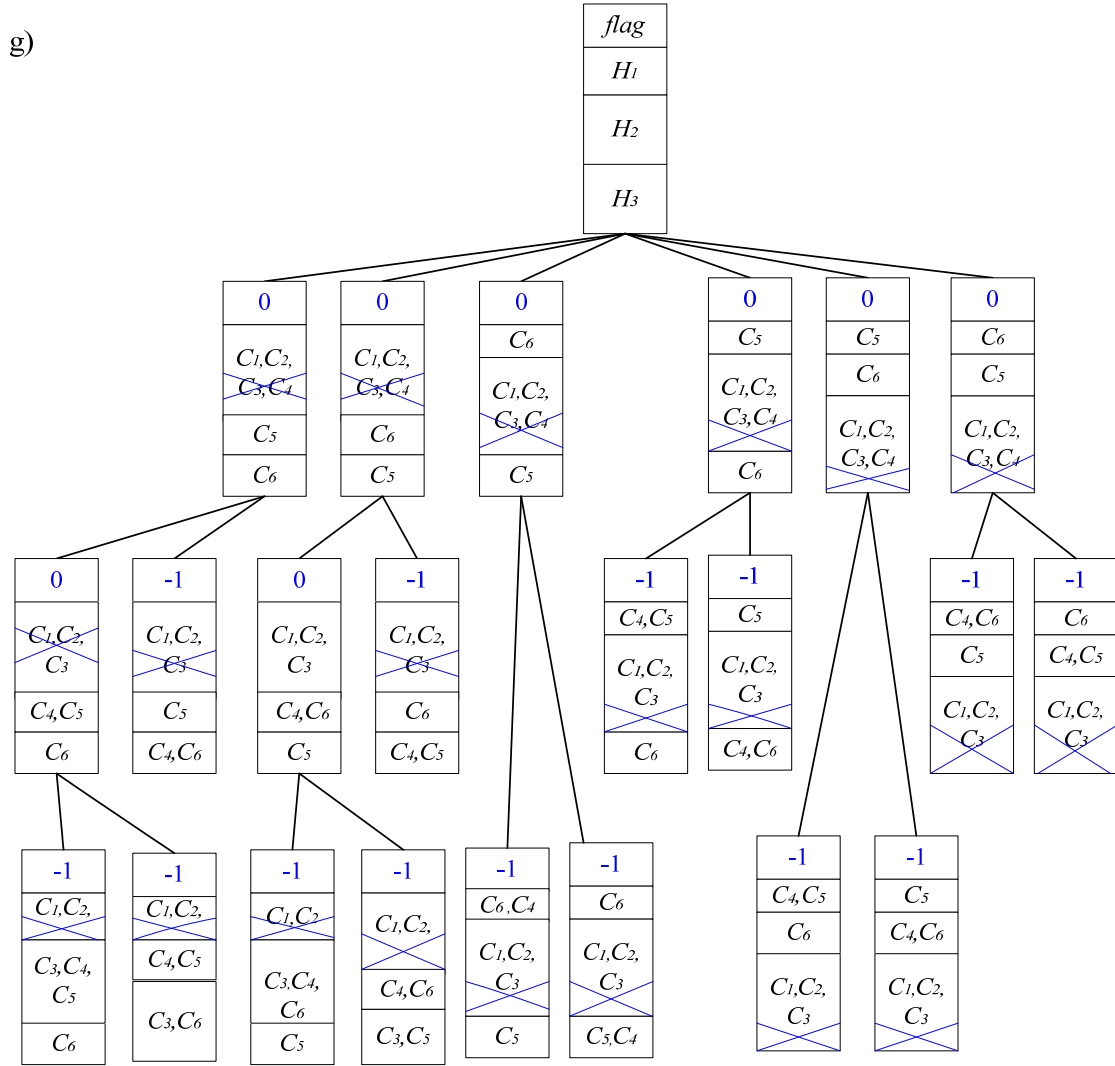


图 4-6 部署架构的扩展图

Fig. 4-6 Graph of extending deployment architectures

a) 根节点

b) 系统初始化

c) DA_{1,1} 的扩展图

a) Root node

b) System initialization

c) Graph of extending DA_{1,1}

d) DA_{2,1} 的扩展图

e) DA_{1,2} 的扩展图

d) Graph of extending DA_{2,1}

e) Graph of extending DA_{1,2}

f) DA_{2,3} 的扩展图

g) 其它部署架构的扩展图

f) Graph of extending DA_{2,3}

g) Graph of extending other deployment architectures

$RDA_{1,2} > RDA_{1,3} > RDA_{1,4} > RDA_{1,5} > RDA_{1,6}$ 。

在图 4-6 e) 中, 选择 $flag$ 为 1 的部署架构 $DA_{1,2}$ 来进行重部署, 选定的软件组件仍为 C_4 。扩展 $DA_{1,2}$ 得到的新部署架构为 $DA_{2,3}$ 和 $DA_{2,4}$, 这里假设 $RDA_{2,3} > RDA_{1,2}$ 和 $RDA_{2,4} < RDA_{1,2}$, 故 $DA_{2,3}$ 的 $flag$ 值为 1, $DA_{2,4}$ 的 $flag$ 值为 -1。由于 $DA_{1,2}$ 被扩展过, $DA_{1,2}$ 的 $flag$ 值变为 0。根据 Prune 方法, 由于 $DA_{2,4}$ 的 $flag$ 值为 -1, 剪掉 $DA_{2,4}$ 。由于 $DA_{1,2}$ 的 $flag$ 值变为 0, 并且 $RDA_{1,2} < RDA_{2,1}$, 根据 Prune 方法, 剪掉 $DA_{1,2}$ 。当前“最优”部署架构仍为 $DA_{2,1}$ 。则余下的未扩展的部署架构的系统可靠性顺序变为 $RDA_{2,3} > RDA_{1,3} > RDA_{1,4} > RDA_{1,5} > RDA_{1,6}$ 。

在图 4-6 f) 中, 选择 $DA_{2,3}$ 来重部署, 选定的软件组件为 C_3 。由 $DA_{2,3}$ 扩展出来的部署架构为 $DA_{3,3}$ 和 $DA_{3,4}$ 。由于 $DA_{2,3}$ 已被扩展, 所以 $DA_{2,3}$ 的 $flag$ 值变为 0。这里假设 $RDA_{3,3} < RDA_{2,3}$ 和 $RDA_{3,4} < RDA_{2,3}$, 故 $DA_{3,3}$ 的 $flag$ 值为 -1, $DA_{3,4}$ 的 $flag$ 值为 -1。根据 Prune 方法, 由于 $DA_{3,3}$ 和 $DA_{3,4}$ 的 $flag$ 值都为 -1, 剪掉 $DA_{3,3}$ 和 $DA_{3,4}$ 。由于 $RDA_{2,3} > RDA_{2,1}$, $DA_{2,3}$ 和 $DA_{2,1}$ 的 $flag$ 值都为 0, 所以根据 Prune 方法剪掉 $DA_{2,1}$ 。当前“最优”部署架构变为 $DA_{2,3}$ 。则余下的未被扩展的部署架构的系统可靠性顺序从大到小变为 $RDA_{1,3} > RDA_{1,4} > RDA_{1,5} > RDA_{1,6}$ 。

由于要选择初始化后的具有较高可靠性的 $2m$ 种部署架构进行扩展, 实例系统包含了 3 个主机节点, 也就是初始化后的 6 种部署架构都要进行扩展。类似地, 余下的 $flag$ 值为 1 的部署架构, 如图 4-6 g) 所示。依次扩展 $DA_{1,3}$ 、 $DA_{1,4}$ 、 $DA_{1,5}$ 和 $DA_{1,6}$, 得到新的部署架构 $DA_{2,5}$ 、 $DA_{2,6}$ 、 $DA_{2,7}$ 、 $DA_{2,8}$ 、 $DA_{2,9}$ 、 $DA_{2,10}$ 、 $DA_{2,11}$ 和 $DA_{2,12}$ 。为了方便, 假设 $RDA_{2,5} < RDA_{1,3}$, $RDA_{2,6} < RDA_{1,3}$, $RDA_{2,7} < RDA_{1,4}$, $RDA_{2,8} < RDA_{1,4}$, $RDA_{2,9} < RDA_{1,5}$, $RDA_{2,10} < RDA_{1,5}$, $RDA_{2,11} < RDA_{1,6}$, $RDA_{2,12} < RDA_{1,6}$ 。也就是新产生的部署架构 $DA_{2,5}$ 、 $DA_{2,6}$ 、 $DA_{2,7}$ 、 $DA_{2,8}$ 、 $DA_{2,9}$ 、 $DA_{2,10}$ 、 $DA_{2,11}$ 和 $DA_{2,12}$ 的 $flag$ 值都为 -1。由于 $DA_{1,3}$ 、 $DA_{1,4}$ 、 $DA_{1,5}$ 和 $DA_{1,6}$ 已经被扩展, 所以这四个部署架构的 $flag$ 值都变为 0。根据 Prune 方法, 由于新产生的部署架构的 $DA_{2,5}$ 、 $DA_{2,6}$ 、 $DA_{2,7}$ 、 $DA_{2,8}$ 、 $DA_{2,9}$ 、 $DA_{2,10}$ 、 $DA_{2,11}$ 和 $DA_{2,12}$ 的 $flag$ 值都为 -1, 减掉部署架 $DA_{2,5}$ 、 $DA_{2,6}$ 、 $DA_{2,7}$ 、 $DA_{2,8}$ 、 $DA_{2,9}$ 、 $DA_{2,10}$ 、 $DA_{2,11}$ 和 $DA_{2,12}$ 。由于 $DA_{1,3}$ 、 $DA_{1,4}$ 、 $DA_{1,5}$ 和 $DA_{1,6}$ 的 $flag$ 值都为 0 的, 并且 $RDA_{1,3} < RDA_{2,3}$, $RDA_{1,4} < RDA_{2,3}$, $RDA_{1,5} < RDA_{2,3}$, $RDA_{1,6} < RDA_{2,3}$, 根据 Prune 方法, 剪掉 $DA_{1,3}$ 、 $DA_{1,4}$ 、 $DA_{1,5}$ 和 $DA_{1,6}$, 则余下的 $flag$ 值变为 0 的 $DA_{2,3}$ 为优化的部署架构。

4.4.4 实验及结果分析

为了评估这四个算法，将在不同主机节点数和软件组件数上运行这四个算法。第一个例子为 3 个主机节点和 8 个软件组件，第二个例子为 4 个主机节点和 11 个软件组件，第三个例子为 10 个主机节点和 100 个软件组件，第四个例子为 15 个主机节点和 200 个软件组件。对于每个例子，随机产生系统的输入，这些输入包括软件组件间的相关度矩阵 CC ， n 个软件组件的失效率， m 个主机节点的失效率，主机节点的失效率为 $[0, 0.0001]$ 间的任意实数，软件组件的失效率为 $[0, 0.01]$ 间的任意实数。实验用 Intel 的 Core2 Quad Q9400 处理器、主频 2.66GHz、3GB DDR2、Windows XP 系统下的 Java SDK 1.5。

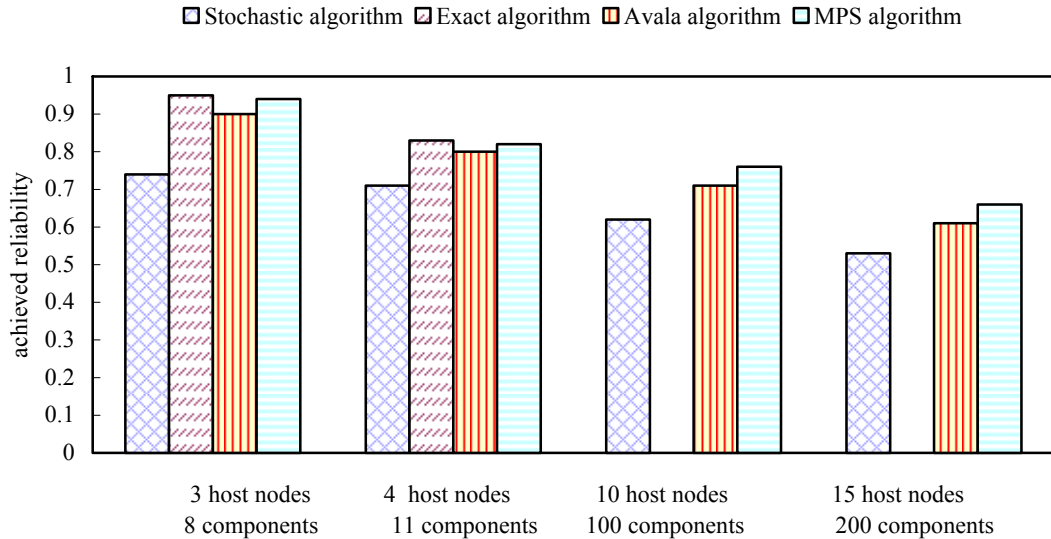


图 4-7 运行四个算法得到的系统可靠性

Fig. 4-7 Achieved system reliability of four algorithms

从图 4-7 可以看出，对于第一个例子和第二个例子，精确算法能够找到优化的部署架构，部署架构的可靠性最高。 m 优先选择算法确定的部署架构的可靠性略低于精确算法，可以知道 m 优先选择算法在这两个例子中找到不是优化的部署架构，而是近似优化的部署架构，但找到部署架构的可靠性要高于 Avala 算法和随机算法。随着主机节点数和软件组件数的增多，对于包含 10 个主机节点和 100 个软件，15 个主机节点和 200 个软件组件的两个例子来说，精确算法求得最优部署架构所需的时间太昂贵，不在可接受的范围，因此，这两个例子未采用精确算法求得优化的部署架构。对于第三个和第四个例子， m 优先选择算法确定的部署架构的可靠性仍高于 Avala 算法和随机算法，因此， m 优先选

择算法不仅能找到高可靠性的部署架构，而且也能处理较大规模的软件组件部署。

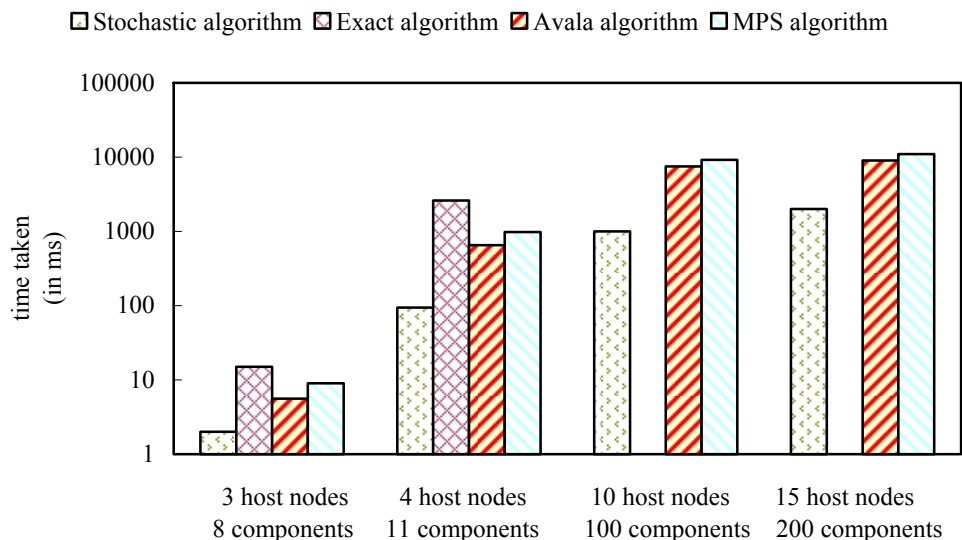


图 4-8 四个算法的执行时间

Fig. 4-8 Taken time of four algorithms

从图 4-8 可以看出，精确算法需要计算时间明显高于其它三种部署算法，随机算法需要计算时间最少，m 优先选择算法需要的计算时间略高于 Avala 算法。综合图 4-7 和 4-8，不同的算法在得到的系统可靠性和执行时间上表现出不同的性能。对于第一个例子和第二个例子，精确算法能找到最优的部署架构，但需要昂贵的执行时间。因此，精确算法不适合软件组件数或主机节点数较多的系统。然而，对于这四个例子，随机算法需要的执行时间最少，但是找到的部署架构的可靠性最低。Avala 算法需要的执行时间要多于随机算法，但是找到的部署架构要高于随机算法。m 优先选择算法的执行时间要多于随机算法（相差因子 $m \log m \log n$ ）和 Avala 算法（相差因子 m ），但是 m 优先选择算法能找到更高的系统可靠性的部署架构。

4.5 本章小结

根据软件组件间的交互关系和交互的重要性，提出软件组件间的相关性和软件组件间的相关度概念。在分析 A*算法优缺点基础上，根据软件组件间的相关度矩阵，设计了 m 优先选择算法来重部署系统。对 m 优先选择算法进行正确性和时间复杂度分析。为了解释 m 优先选择算法，给出 3 个主机节点和 6 个软

件组件的详细执行过程。之后，给出 4 个例子的实验，实验结果表明，从 3 个主机节点和 8 个软件组件，与 4 个主机节点和 11 个软件组件的例子来看，m 优先选择算法找到的部署架构的可靠性和精确算法找到的部署架构的可靠性非常接近，但是需要的计算时间却比精确算法少得多。m 优先选择算法找到的部署架构的可靠性比 Avala 算法找到的部署架构的可靠性高，但是需要的计算时间却比 Avala 算法多。从 10 个主机节点和 100 个软件组件，15 个主机节点和 200 个软件组件的例子来看，m 优先选择算法的执行时间要慢于随机算法(相差因子 $m \log m \log n$)和 Avala 算法(相差因子 m)，但是 m 优先选择算法能得到更高的系统可靠性。并且从 15 个主机节点和 200 个软件组件的例子可以看出，m 优先选择算法适合大规模的组件部署。

第 5 章 组件复制及故障传播的可靠性研究

5.1 引言

分布式、移动和普适环境下的系统可靠性受到网络互联和节点状态及自身结构的影响。大规模的广域网如 Internet 的节点数以亿计，而这些节点大都是不稳定的，Kutzner 等^[121]在 PlanetLab 平台对 Overnet 做了详细评估，结果证明，有将近 50% 的主机节点在线时间超过了 6 小时，有 20% 比例的主机节点在线时间会超过 48 小时，可用的主机节点数量将随时间的增长而趋于不断下降。Plissonneau 等^[122]提出，Internet 上的主机节点平均无故障时间仅约为 1 小时，这意味着在节点数目为 1 万的中等规模网络中，平均仅仅两分钟就有 1 个节点变为不可用。而根据中国互联网信息中心针对 2004 年至 2008 年间的调查报告^[123]可以得知，中国大陆地区范围的主机节点在不同时段通过 Internet 互联的百分比有很大差异，如图 5-1 所示。

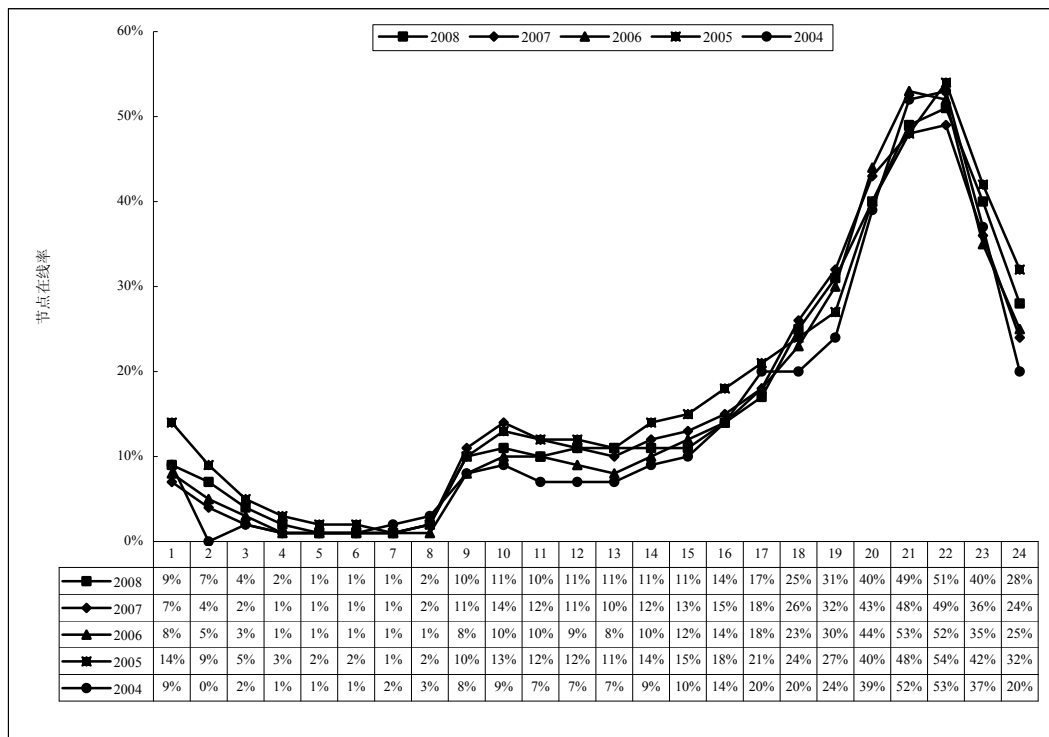


图 5-1 中国大陆 Internet 节点接入率

Fig. 5-1 The Internet node connection rate in China mainland

自早晨 8 点在线节点开始逐渐增加，晚上 8 点至 10 点之间的在线节点数达

到最高峰值，而凌晨 4 点到早 8 点之间的在线节点数为低谷状态。这样频繁、大规模的节点状态变化无疑将会显著影响系统的可靠性等性能。另外，Stephan Edidenbenz 等^[124]提出，造成数据网络动态性的主要原因是节点本身的自主行为，网络节点可以随意打开、关闭主机或随时终止所提供的服务及相关组件。正由于节点可以随时处于网络脱机或是服务组件停止状态，节点所提供的服务自然带有相当程度的不可靠性。尤其是在较大规模的普适、移动和分布式系统中，这种动态性使得相关节点很可能无法正确获得所需的数据或文件内容。

从上面的例子可以看出，在普适、移动和分布式的环境下，经常会有节点或者组件面临失效。如果节点失效，则部署在这个主机节点上的软件组件所提供的服务都将不可用。而如果软件服务是独立的或者软件组件有副本，系统仍可正常工作。由于这些环境下的系统资源是受限的，例如主机节点（如 PDA）的电量有限、主机节点可用内存有限等等，必须有选择地复制软件组件从而获得最大利益。Popescu^{[83]12-13} 根据可靠性优化目标函数 OF，应用贪婪算法来选择被复制的软件组件集合，该算法考虑内存、网络带宽、服务重要性的影响。然而，通过贪婪算法选择出的软件组件集合通常得到的是局部优化解，而不是最优的软件组件集合。针对此方法所存在的问题，本章设计启发式的贪婪复制算法选择部署阶段的将要被复制的软件组件，分析得到的可靠性目标优化值和实际消耗的系统资源，并对复制的后服务可靠性进行分析。此外，当节点或者组件失效时，虽然有组件副本仍可继续提供服务，系统仍可正常工作。然而，在节点或者组件崩溃的过程中，会将故障传播给与其直接进行交互的其它的节点或者组件。因此，本章也对相关主机节点失效和软件组件失效引起的故障传播进行分析。

5.2 复制技术

复制技术是实现容错机制的主要手段^[125]，该技术主要采用空间冗余方式为同一服务提供冗余的计算资源，即使系统中的某些软硬件资源失效时，其冗余的计算资源可以继续提供服务。服务的运行实例被称为服务副本。由于不同领域中设计目标不同，因此复制技术的用途也有所差别^[126-127]。图 5-2 对目前主要复制技术进行分类。

根据提供一致性语义分为同步复制和异步复制。同步复制要求能够保证强副本一致性^[128]。异步复制不能确保强副本一致性，但要保证最终一致性^[129]，也就是保证所有的服务副本有相同的最终状态。异步复制也叫懒复制或者乐观复制。一般而言，与同步复制系统相比，异步复制系统维持状态同步的开销小得多，具有更好的性能。然而，一些金融、工业控制等关键应用必须采用同步

复制技术。

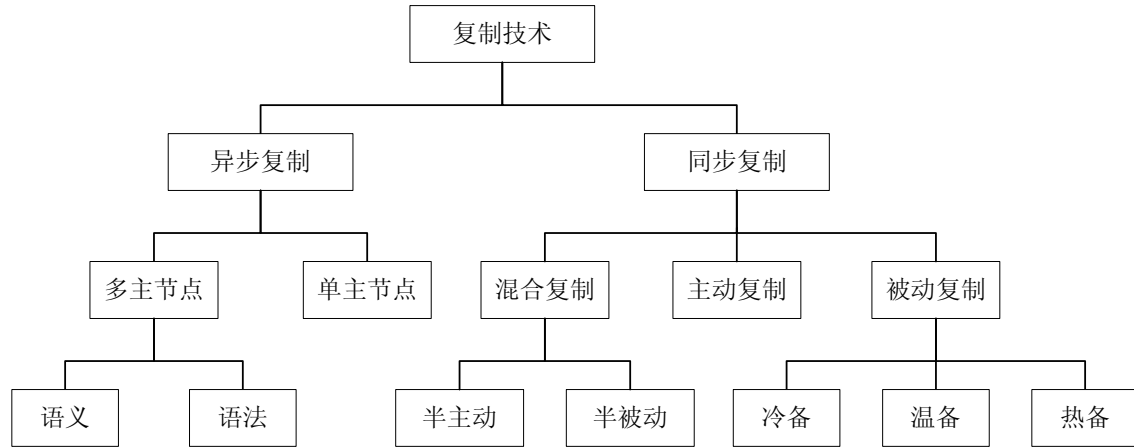


图 5-2 复制技术的分类

Fig.5-2 Classification of replication technology

主动复制^[130-131]，也称为状态机复制 (state machine replication)，这是一种基于故障屏蔽策略的复制方法。被动复制^[132-134]，也称为主从复制(primary-back replication)是基于故障恢复机制的一类复制技术。被动复制技术也被称为热备(hot passive)。此外，被动复制也包括温备 (warm passive)和冷备 (cold passive)两个类型^[135]。除主动与被动复制外，还有介于这两者之间的其它同步复制技术，包括半主动复制(semi-active)^[136]与半被动复制(semi-passive)^[137]。

从资源使用的角度来分析，主动复制占用的较多计算资源和网络资源。与主动复制相比，被动复制可以节省很多的 CPU 资源，而且占用的网络资源情况和服务状态的大小相关。从性能角度来分，一般认为主动复制优于被动复制^[138]。

5.3 启发式的贪婪复制算法

5.3.1 结构描述和目标函数

(1) 结构描述

系统包括主机节点、软件组件、服务、网络物理链接、逻辑链接和服务逻辑链接。用这些实体来建模移动、普适和分布式系统中的软件组件部署，具体包括^{[83]2-3}：

- ① 主机节点集合 $H = \{h_1, h_2, \dots, h_m\}$ ，描述系统中的主机节点；
- ② 软件组件集合 $C = \{c_1, c_2, \dots, c_n\}$ ，描述系统中的软件组件；
- ③ 服务集合 $S = \{s_1, s_2, \dots, s_k\}$ ，描述系统提供或者执行的不同用例，一个服

务由系统中交互的软件组件构成；

④ 网络物理链接集合 $PL = \{pl_1, pl_2, \dots, pl_x\}$ ，连接两个主机节点。部署在这些主机节点上的软件组件间进行交互。网络物理链接有很多属性，如网络带宽等；

⑤ 逻辑链接集合 $LL = \{ll_1, ll_2, \dots, ll_y\}$ ，描述两个软件组件间的交互关系；

⑥ 服务逻辑链接集合 $SLL = \{sll_1, sll_2, \dots, sll_z\}$ ，服务逻辑链接连接两个软件组件，某种程度上提供相同服务。它有很多属性，例如交换数据的大小和平均交互频率等。

矩阵 CS 描述软件组件和服务之间的关系， $cs_{i,j}$ 的值可为 1 或者为 0，如果软件组件 c_i 某种程度上提供服务 s_j ， $cs_{i,j}$ 的值为 1；如果软件组件 c_i 不提供服务 s_j ， $cs_{i,j}$ 的值为 0。

$$CS = \begin{matrix} & \begin{matrix} s_1 & s_2 & \cdots & s_k \end{matrix} \\ \begin{matrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{matrix} & \begin{bmatrix} cs_{1,1} & cs_{1,2} & \cdots & cs_{1,k} \\ cs_{2,1} & cs_{2,2} & \cdots & cs_{2,k} \\ \vdots & \vdots & \vdots & \vdots \\ cs_{n,1} & cs_{n,2} & \cdots & cs_{n,k} \end{bmatrix} \end{matrix}$$

矩阵 FCC 描述软件组件间的交互频率， $fcc_{i,j}$ 表示软件组件 c_i 和 c_j 间的交互频率， $fcc_{i,j}$ 的值为 $[a, b]$ 间的任意整数， a 和 b 的值依赖于具体系统。矩阵 DCC 描述两个软件组件间的交互事件， $dcc_{i,j}$ 描述软件组件 c_i 和 c_j 间交互事件的大小， $dcc_{i,j}$ 的值为 $[e, f]$ 间的任意实数，而 e 和 f 的值依赖于具体系统。

$$FCC = \begin{matrix} & \begin{matrix} c_1 & c_2 & \cdots & c_n \end{matrix} \\ \begin{matrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{matrix} & \begin{bmatrix} fcc_{1,1} & fcc_{1,2} & \cdots & fcc_{1,n} \\ fcc_{2,1} & fcc_{2,2} & \cdots & fcc_{2,n} \\ \vdots & \vdots & \vdots & \vdots \\ fcc_{n,1} & fcc_{n,2} & \cdots & fcc_{n,n} \end{bmatrix} \end{matrix} \quad DCC = \begin{matrix} & \begin{matrix} c_1 & c_2 & \cdots & c_n \end{matrix} \\ \begin{matrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{matrix} & \begin{bmatrix} dcc_{1,1} & dcc_{1,2} & \cdots & dcc_{1,n} \\ dcc_{2,1} & dcc_{2,2} & \cdots & dcc_{2,n} \\ \vdots & \vdots & \vdots & \vdots \\ dcc_{n,1} & dcc_{n,2} & \cdots & dcc_{n,n} \end{bmatrix} \end{matrix}$$

(2) 目标函数

复制软件组件能提高系统可靠性，但是需要消耗额外的系统资源。在选择将被复制的软件组件时，决策者需要考虑用例的重要性、软件组件消耗的资源、软件组件的可靠性、计算机硬件和网络等等。选择好将被复制的软件组件后，决策者需要决定如何将这此软件组件部署到主机节点上^{[83]2}。为了强调软件组件副本选择问题，Malek^[139]将它转换为优化问题，优化问题的目标是找到满足给定约束条件的较好折中方法，相应的目标函数如公式 (5-1) 所示。其中， $|C|$ 表示系统中软件组件的数目， $|S|$ 表示系统中服务的数目， $R(s_j, c_i)$ 表示软件组件 c_i 和

它的组件副本提供的服务 s_j 的可靠性， $Cri(s_j)$ 表示服务 s_j 的重要性。

$$OF = \sum_{c_i=1}^{|C|} \sum_{s_j=1}^{|S|} \eta \times R(s_j, c_i) \times Cri(s_j) \quad (5-1)$$

$$\eta = \begin{cases} 1, & \text{如果 } cs_{i,j} = 1 \\ 0, & \text{如果 } cs_{i,j} = 0 \end{cases} \quad (5-2)$$

每个软件组件副本为原始软件组件的一个附加实例，根据软件组件副本的数目来计算提供服务的可靠性^{[83]4}，见公式（5-3）。其中， $|ReplicaSet(c_i)|$ 表示软件组件 c_i 和它的组件副本总数目， $Replica(R_p, c_i)$ 表示软件组件 c_i 或者它的副本组件提供服务 s_k 。

$$R(s_j, c_i) = 1 - \prod_p^{|ReplicaSet(c_i)|} FailProbability(s_j, Replica(R_p, c_i)) \quad (5-3)$$

$$FailProbability(s_k, c_i) = ph_j + (1 - ph_j) \times pc_i \quad (5-4)$$

在公式(5-4)中，软件组件 c_i 部署在主机节点 h_j 上， ph_j 表示主机节点 h_j 的失效率， pc_i 表示软件组件 c_i 的失效率。这里不考虑网络物理链接对系统可靠性的影响，认为网络物理链接是完全可靠的。

此外，系统所需的峰值带宽 BMS 可由公式（5-5）求得，其中 BM_{ij} 表示主机节点 H_i 向主机节点 H_j 单向通信所需的带宽。

$$BMS = \sum_{1 \leq j \leq m, 1 \leq i \leq m, j \neq i} BM_{ij} \quad (5-5)$$

5.3.2 启发式的贪婪复制算法描述

（1）已有复制算法

第一个启发式策略是在每个主机节点上复制所有的软件组件，从而最大化系统可靠性。也就是系统中的每个主机节点都包含所有的软件组件。第二个启发式策略是复制每个软件组件一次，因此，如果一个软件组件失效，仍有另一个软件组件可用。第三个启发式策略是复制每个软件组件两次^{[83]11-13}。

贪婪算法^{[83]12-13} 是一个迭代算法。每次迭代过程中，找到一个可能“最好”的解决办法。如果搜索空间没有局部最优化，则贪婪算法会找到最优化解。但是，通常情况并非如此，贪婪算法得到的是一个近似最优的解决方案。贪婪算法根据当前可用的内存和网络带宽，每次选择出使目标函数 OF 优化值提高最大的软件组件和其部署的主机节点。当贪婪算法选择出的软件组件需要的带宽大于系统可用带宽，或者需要的内存大于系统可用内存时，算法停止，该算法需

要花费时间为 $O(n^3)$ (n 为软件组件数目)。

(2) 启发式的贪婪复制算法描述

第一个启发式策略复制了所有的软件组件，这对主机节点要求非常高，要有足够大的可用内存部署所有的软件组件。并且，对于任意的软件组件来说，其它的主机节点上都含有这个软件组件的副本，如何保持这些软件组件和它的副本状态的一致性，也是非常难处理的。对于第二个和第三个启发式策略，与第一个复制策略相比，消耗的系统资源减少很多。但是从成本效益的角度分析，被复制的软件组件提供服务的重要性不同和软件组件的可靠性不同，将所有的软件组件都复制一次或者两次是不合理的。因此，充分地利用系统资源，有选择地复制软件组件。

由于贪婪算法每次在满足系统可用资源的情况下，选择使目标函数 OF 值最大化的软件组件。然而，对于某些软件组件，复制这些软件组件虽然能把 OF 值提高最大，但消耗的可用内存和网络带宽也相当多。基于这个原因，本章设计了启发式的贪婪复制算法，估计出复制某个软件组件得到的优化值、需要的带宽和内存，求出优化值和需要的内存的优化比值 OM 、优化值和需要的带宽的优化比值 OB ，将这两个优化比值作为选择软件组件进行复制的主要依据。根据 OM 选择出一组满足约束条件的软件组件集合，根据 OB 选择出一组满足约束条件的软件集合，其中，优化值更高的软件组件集合就是该算法确定的将被复制的软件组件集合，详细算法描述的伪代码如图 5-3 所示，其中， $H(C_i) = H_j$ 表示软件组件 C_i 部署到主机节点 H_j 上。

Algorithm 启发式的贪婪复制算法

Input n software components $C = \{C_1, C_2, \dots, C_n\}$, m host nodes $H = \{H_1, H_2, \dots, H_m\}$;

Output Replicated software components ;

```

1  {for each software component  $C_i \in C$ ,  $H(C_i) = H_j \in H$ ,  $flagband=0, flagmemory=0$ ;
2
3  /* Initialization. Replicate each software component, and deploy the software component
4  on each other host node, and estimate the increased bandwidth, increased memory and
5  increased optimal value. /
6  for each software component  $C_i \in C$ 
7      for each host node  $H_k \in H$ 
8          /* Software component  $C_i$  has been deployed on  $H_j$  before replication */
9          { Replicate  $C_i$  once, and deploy  $C_i$  on host node  $H_k$  ( $H_k \in H, H_k \neq H_j$ );
10             IncreaseOptimumValue( $C_i, H_k$ ); // Estimate increased optimal value;
11             IncreaseBandwidth( $C_i, H_k$ ); // Estimate increased bandwidth;
12             IncreaseMemory( $C_i, H_k$ ); // Estimate increased memory;
```

```

14      /* Estimate increased optimal value of each unit's bandwidth/
15       $OB(C_i, H_k) = \text{IncreaseOptimumValue}(C_i, H_k) / \text{IncreaseBandwidth}(C_i, H_k);$ 
16      /* Estimate increased optimal value of each unit's memory/
17       $OM(C_i, H_k) = \text{IncreaseOptimumValue}(C_i, H_k) / \text{IncreaseMemory}(C_i, H_k);$ 
18  }
19      /* Find the optimal set of software components based on  $OB$  /
20      while(flagband=0)
21      {
22          Choose the software component that has maximum value based on  $OB(C_i, H_k)$  each time;
23          /* Check whether the selected software component satisfies the condition or not/
24          If (required memory of  $C_i \leq \text{system available memory}$  ) &&(required bandwidth of  $C_i$ 
25           $\leq \text{system available bandwidth}$  )
26          Select software component  $C_i$ , and mark the host node  $H_k$ ;
27          else flagband=1; // Mark the process finished;
28      }
29      /* Find the optimal set of software components based on  $OM$  /
30      while(flagmemory=0)
31      {
32          Choose the software component that has the maximum value based on  $OM(C_j, H_p)$  each
33          time;
34          If (required memory of  $C_j \leq \text{system available memory}$  ) &&(required bandwidth of  $C_j$ 
35           $\leq \text{system available bandwidth}$  )
36          Select software component  $C_j$ , and mark the host node  $H_p$ ;
37          else flagmemory =1; // Mark the process finished;
38      }
39      Software components of the set with higher optimal value will be replicated.
40  }
41
42  }
```

图 5-3 启发式的贪婪复制算法的伪代码

Fig. 5-3 Pseudocode of heuristic greedy replication algorithm

5.3.3 启发式的贪婪复制算法的正确性和复杂度分析

(1) 启发式的贪婪复制算法的正确性分析

启发式的贪婪复制算法是正确的，如果它对每一个输入都最终停止，而且产生正确的输出。首先分析对于每个输入，该算法都停止，之后分析对于每个输入，该算法都能产生正确的输出。

① 对于每一个输入算法都能最终停止

设系统有 n 个软件组件 $C = \{C_1, C_2, \dots, C_n\}$ 和 m 个主机节点 $H = \{H_1, H_2, \dots, H_m\}$ 。

对于给定的有限可用系统资源，选择部分软件组件进行复制。对于每个软件组件，假定将其复制其它 $m-1$ 主机节点上后，计算增加的内存、增加的带宽和增加的优化值，这需要常数时间 $O(m)$ 。 n 个软件组件的这样的初始化过程，需要花费时间 $O(mn)$ 。也就是，初始化过程最多执行 $O(mn)$ 步。

对于根据 OB 和 OM 来搜索将被复制的软件组件集合，这两个循环的条件都是当在选择某一个软件组件时，如果这个软件组件需要的内存超过了系统当前可用内存，或者这个软件组件需要的带宽超过了系统当前可用的带宽，则这两个循环将结束。那么设定在软件组件复制前，系统可用内存为有限的数 M_s ，系统可用带宽为有限的数 B_s ，在 n 个软件组件中，假定需要内存最小的软件组件为 $C_a \in C$ ， C_a 需要的内存为 M_a ，那么最多经过 $\lceil M_s / M_a \rceil$ 步，循环将由于系统可用内存分配完而停止；同理，在 n 个软件组件中，假定需要带宽最小的软件组件为 $C_b \in C$ ， C_b 需要的带宽为 B_b ，那么最多经过 $\lceil B_s / B_b \rceil$ 步，循环将由于系统可用带宽分配完而停止。也就是最多经过 $\min\{\lceil M_s / M_a \rceil, \lceil B_s / B_b \rceil\}$ 步，两个循环将停止。

因此，对于每一个输入，最多经过 $O(mn) + \min\{\lceil M_s / M_a \rceil, \lceil B_s / B_b \rceil\}$ 步，该算法将停止。

② 对于每一个输入算法都能产生正确的结果

这里不正确的结果包括两种情况：选择的软件组件复制集合不满足系统可用内存的限制和选择的软件组件复制集合不满足系统可用带宽的限制。

每次选择将被复制的软件组件时，首先判断该软件组件是否满足系统可用内存要求和带宽要求。对于根据 OB 来选择软件组件时，如果软件组件需要的内存超过了系统可用内存，或者软件组件需要的带宽超过了系统可用带宽，则循环下次执行时判断不满足执行条件，循环将退出。对于根据 OM 来选择软件组件时，如果软件组件需要的内存超过了系统可用内存，或者软件组件需要的带宽超过了系统可用带宽，则循环下次执行时，判断不满足执行条件，循环将退出。也就是如果遇到不满足内存和带宽约束条件的软件组件，算法将停止，也就选择出的将被复制的软件组件都满足约束条件。因此，对于每一个输入，算法都能产生出正确的结果。

综上所述，启发式的贪婪复制算法是正确的。

(2) 启发式的贪婪复制算法时间复杂度分析

启发式的贪婪复制算法初始化过程需要花费时间 $O(mn)(m \leq n)$ ，根据 OB 选出一个最大值，也就是选出一个软件组件及其应该部署到的主机节点，需要花费时间 $O(n^2)$ 。如果可复制 kn (k 为大于 0 的常数) 个软件组件，那么花费的

时间为 $O(kn^3)$ ；同理，根据 OM 选择一个最大值，也就是选择一个软件组件及其应该部署到的主机节点，需要花费时间 $O(n^2)$ 。如果可复制 ln (l 为大于 0 的常数) 个软件组件，那么花费的时间为 $O(ln^3)$ 。其中， $O(mn + (k+l)n^3) = O(n^3)$ 。因此，启发式的贪婪复制算法的时间复杂度为 $O(n^3)$ 。

5.3.4 实验及结果分析

系统包括 12 个软件组件和 4 个主机节点。实验输入包括软件组件的大小、软件组件间的交互频率、软件组件间交互事件的大小、软件组件的失效率、主机节点的失效率和服务的重要性。当不考虑系统可用资源时，可利用三个启发式算法复制软件组件。当考虑系统可用的资源时，将采用启发式的贪婪复制算法选择将被复制的软件组件，并与贪婪算法进行比较。

(1) 输入

$Size_k$ 表示软件组件 c_k 的大小，为 $[1, 5]$ 间的任意实数。 $Size_1 = 3.53$ ， $Size_2 = 4.73$ ， $Size_3 = 4.87$ ， $Size_4 = 4.48$ ， $Size_5 = 3.19$ ， $Size_6 = 2.57$ ， $Size_7 = 4.86$ ， $Size_8 = 3.62$ ， $Size_9 = 3.18$ ， $Size_{10} = 3.67$ ， $Size_{11} = 4.93$ ， $Size_{12} = 2.48$ 。

$fcc_{i,j}$ 表示软件组件 c_i 和 c_j 间的交互频率，为 $[0, 7]$ 间的任意整数。 $fcc_{3,4} = 2$ ， $fcc_{4,3} = 3$ ， $fcc_{8,9} = 4$ ， $fcc_{9,8} = 3$ ， $fcc_{6,8} = 3$ ， $fcc_{8,6} = 2$ ， $fcc_{1,10} = 5$ ， $fcc_{10,1} = 5$ ， $fcc_{1,3} = 6$ ， $fcc_{3,1} = 5$ ， $fcc_{2,5} = 1$ ， $fcc_{5,2} = 2$ ， $fcc_{6,7} = 2$ ， $fcc_{7,6} = 1$ ， $fcc_{5,6} = 7$ ， $fcc_{6,5} = 5$ ， $fcc_{4,11} = 2$ ， $fcc_{11,4} = 3$ ， $fcc_{1,12} = 1$ ， $fcc_{12,1} = 1$ 。其它的软件组件间的交互频率为 0。

$dcc_{i,j}$ 表示软件组件 c_i 和 c_j 间的交互事件大小，为 $[1, 8]$ 间的任意实数。 $dcc_{3,4} = 7.55$ ， $dcc_{4,3} = 6.2$ ， $dcc_{8,9} = 1.7$ ， $dcc_{9,8} = 1.2$ ， $dcc_{6,8} = 2.1$ ， $dcc_{8,6} = 2.2$ ， $dcc_{1,10} = 2.37$ ， $dcc_{10,1} = 1.8$ ， $dcc_{1,3} = 2.02$ ， $dcc_{3,1} = 1.6$ ， $dcc_{2,5} = 4.85$ ， $dcc_{5,2} = 3.28$ ， $dcc_{6,7} = 5.1$ ， $dcc_{7,6} = 5.3$ ， $dcc_{5,6} = 1.16$ ， $dcc_{6,5} = 1.2$ ， $dcc_{4,11} = 5.39$ ， $dcc_{11,4} = 6$ ， $dcc_{1,12} = 4$ ， $dcc_{12,1} = 3$ 。其它的软件组件间的交互的事件大小为 0。

pc_i 表示软件组件 c_i 的失效率，为 $[0, 0.3]$ 间的任意实数。 $pc_1 = 0.0350$ ， $pc_2 = 0.2307$ ， $pc_3 = 0.1679$ ， $pc_4 = 0.1542$ ， $pc_5 = 0.0396$ ， $pc_6 = 0.0714$ ， $pc_7 = 0.1631$ ， $pc_8 = 0.2944$ ， $pc_9 = 0.0936$ ， $pc_{10} = 0.1666$ ， $pc_{11} = 0.1954$ ， $pc_{12} = 0.1624$ 。

ph_j 表示主机节点 h_j 的失效率，为 $[0, 0.1]$ 间的任意实数。 $ph_1 = 0.0078$ ， $ph_2 = 0.0462$ ， $ph_3 = 0.0824$ ， $ph_4 = 0.0594$ 。

$Cri(s_i)$ 表示服务 s_i 的重要性，为 $[1, 3]$ 间的任意整数。 $Cri(s_1) = 2$ ， $Cri(s_2) = 3$ ， $Cri(s_3) = 2$ ， $Cri(s_4) = 2$ 。 $cs_{i,j}$ 表示软件组件 c_i 某种程度上提供服务 s_j ， $cs_{i,j}$ 的值为

0 或者为 1。 $cs_{3,1}=1$, $cs_{11,1}=1$, $cs_{4,1}=1$, $cs_{2,2}=1$, $cs_{5,2}=1$, $cs_{6,2}=1$, $cs_{7,2}=1$, $cs_{8,3}=1$, $cs_{9,3}=1$, $cs_{1,4}=1$, $cs_{10,4}=1$, $cs_{12,4}=1$ 。

(2) 资源消耗和得到的优化值的比较

本实验分析不同复制策略的资源消耗和得到的可靠性优化值。这里采用文献[83]给出 12 个软件组件和 4 个主机节点的一种部署架构关系,如图 5-4 所示。软件组件 c_3 、 c_9 和 c_{12} 部署到主机节点 h_1 , 软件组件 c_8 和 c_{10} 部署到主机节点 h_2 , 软件组件 c_4 和 c_6 部署到主机节点 h_3 , 软件组件 c_1 、 c_2 、 c_5 、 c_7 和 c_{11} 部署到主机节点 h_4 。

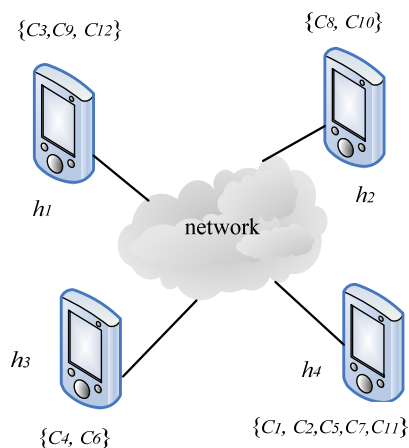


图 5-4 软件组件在主机节点上的部署图

Fig. 5-4 Deployment graph of software components deployed on host nodes

当系统可靠性低于某一规定值时,通过软件组件复制可提高系统可靠性。然而复制软件组件需要额外消耗系统资源,例如网络物理带宽、内存等等。首先,对三种软件组件复制策略所消耗的系统资源和得到的优化值 OF 进行比较,如表 5-1 所示。

表 5-1 3 个启发式策略的比较结果

Table 5-1 Comparison results of three heuristic strategies

算法	需要的带宽	需要的内存	优化值 OF
基本情况	161.17	186.16	22.9694
复制每个软件组件一次	295.26	372.3	27.9147
复制每个软件组件两次	345.16	558.5	27.9966
复制所有软件组件	517.74	744.6	27.9998

在表 5-1 中,对于每个软件组件复制一次的情况,如果软件组件 C_i 部署到

主机节点 H_j 上, 则 C_i 的组件副本将部署到 H_{j+1} , 以此类推。如果 C_i 部署到主机节点 H_4 上, 则 C_i 的组件副本将部署到 H_1 上。对于每个软件组件复制两次的情况, 如果 C_i 部署到主机节点 H_j 上, 则 C_i 的组件副本将部署到 H_{j+1} 和 H_{j+2} 上, 以此类推, 如果 C_i 部署到 H_3 上, 则 C_i 的组件副本将部署到 H_4 和 H_1 上。对于复制所有软件组件的情况, 每个主机节点上都包含所有的软件组件。这里主机节点的拓扑结构为星型结构, 所需的带宽为系统的峰值带宽。

从表 5-1 可以看出, 复制所有软件组件可以提供最高的目标优化值 OF, 但带宽和内存成本是非常昂贵的。然而, 由于在某些系统中, 成本并不是关键的, 它仍是一个有效的策略。复制每个软件组件一次比复制所有软件组件需要的带宽和内存成本低很多。但在不可靠的环境下, 软件组件经常失效, 这个策略是不足够的。每个软件组件复制两次的带宽成本增加不多, 内存增量较大, 仍明显低于复制所有软件组件的方法, 然而可以提供更高的可靠性。此外, 软件组件和它的组件副本都对所有的信息进行处理。

当系统中可用资源非常有限, 只能复制一部分软件组件。为了充分利用系统可用资源和得到更高的优化值, 贪婪算法每次选择使目标优化值提高最大的软件组件来复制。然而, 这个策略通常得到近似最优的软件组件集合, 本章利用启发式的贪婪复制算法选择出将被复制的软件组件。贪婪算法和启发式的贪婪复制算法选择的软件组件所消耗的系统资源和得到优化值的比较如表 5-2 所示。

表 5-2 贪婪算法和启发式的贪婪复制算法的结果比较

Table 5-2 Comparison results of greedy and heuristic greedy replication algorithms

算法	所需的带宽	所需的内存	优化值 OF
基本情况	161.17	186.16	22.9694
贪婪算法(130%)	195.08	238.29	24.4073
启发式的贪婪复制算法(130%)	204.13	241.21	24.6653
贪婪算法 (140%)	219.63	258.7	24.9293
启发式的贪婪复制算法(140%)	193.7	257.99	25.1353

第一种情况为当系统可用资源增加 30%, 第二种情况为系统可用资源增加了 40%。当系统可用资源增加 30%时, 贪婪算法选择复制的软件组件为 c_2 、 c_7 和 c_{12} , 软件组件 c_2 和 c_7 的组件副本部署到主机节点 h_1 上, 软件组件 c_{12} 的组件副本部署到主机节点 h_3 ; 启发式的贪婪复制算法选择复制的软件组件为 c_2 、 c_8 、 c_{10} 和 c_{12} , 其中软件组件 c_2 、 c_8 和 c_{10} 的组件副本部署到主机节点 h_1 上, 软件组件 c_{12} 的组件副本部署到主机节点 h_4 。当系统可用资源增加 40%时, 贪婪算法选择将被

复制的软件组件为 c_2 、 c_7 、 c_{10} 和 c_8 ，这些组件副本均部署到主机节点 h_1 上；启发式的贪婪复制算法选择复制的软件组件为 c_2 、 c_7 、 c_8 、 c_{12} 和 c_6 ，其中软件组件 c_2 、 c_7 、 c_8 和 c_6 的组件副本均部署到主机节点 h_1 上， c_{12} 的组件副本部署到主机节点 h_3 上。

从表 5-2 可以看出，当系统可用资源增加 30% 时，启发式的贪婪复制算法实际消耗的带宽和内存均高于贪婪算法，但启发式的贪婪复制算法在充分利用系统可用资源的情况下，能得到更高的优化值。当系统可用资源增加 40% 时，启发式的贪婪复制算法实际消耗的带宽和内存都低于贪婪算法，并且启发式的贪婪复制算法得到的优化值高于贪婪算法。因此，在系统可用资源有限的情况下，与贪婪算法相比，启发式的贪婪复制算法能充分利用系统可用资源，更有效地选择将被复制的软件组件，得到更高的可靠性优化值。

(3) 复制算法对可靠性影响的比较

本实验分析不同复制策略下，软件组件提供服务的可靠性变化情况。如图 5-5 所示，Basic 表示没有软件组件复制的情况下，每个软件组件在某种程度上提供部分服务的可靠性，RepOnce 表示每个软件组件复制一次的情况下，每个软件组件的某种程度上提供部分服务的可靠性，RepAll 表示在所有的主机节点上复制所有的软件组件的情况下，每个软件组件的某种程度上提供部分服务的可靠性。

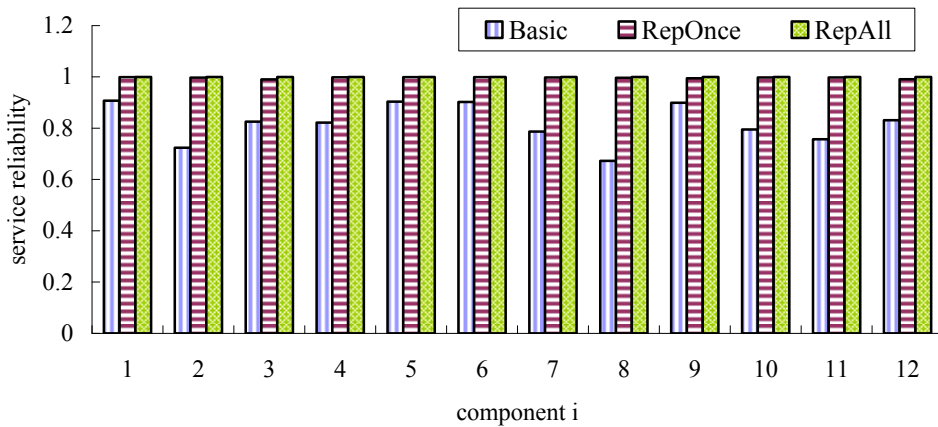


图 5-5 三个启发式策略的可靠性对比图

Fig. 5-5 Component reliabilities of three heuristic strategies

从图 5-5 可以看出，在软件组件复制前，软件组件 c_2 和 c_8 提供的服务可靠性低，是系统中的薄弱点。通过在每个主机节点上复制所有的软件组件和复制每个软件组件一次，每个软件组件提供服务的可靠性都得到很大提高，消除系

统可靠性提高的瓶颈。此外，复制软件组件一次与复制前相比，软件组件提供的服务可靠性都有明显地提高。复制所有的软件组件，与复制软件组件一次相比，软件组件提供服务的可靠性提高不明显，但其消耗的系统资源要明显多。

对于资源受限情况，为了使实验简明，分析系统可用资源增加 30%和系统可用资源增加 40%的这两种情况。当系统可用资源增加 30%时，采用启发式的贪婪复制算法和贪婪算法选择需要被复制的软件组件，软件组件提供的服务可靠性如图 5-6 所示，G30 表示用贪婪算法得到软件组件提供的服务可靠性，A30 表示用启发式的贪婪复制算法得到的软件组件提供的服务可靠性。

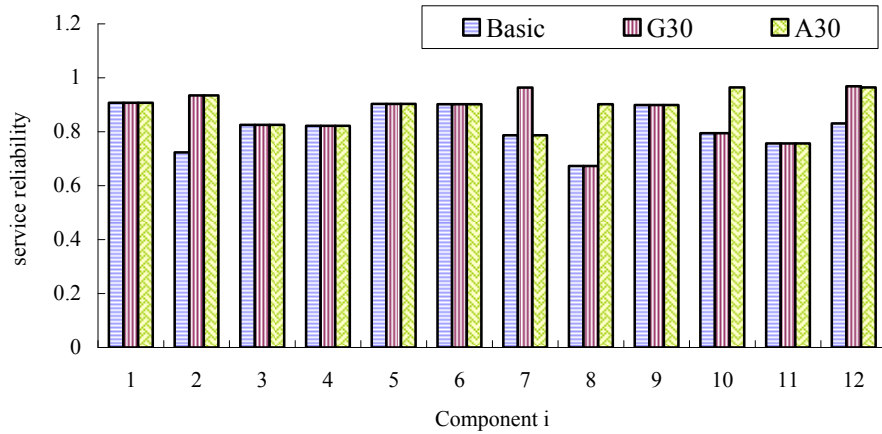


图 5-6 系统资源增加 30%时，组件提供服务的可靠性变化图

Fig. 5-6 Difference in component reliabilities when system resources increased by 30%

从图 5-6 可以看出，当系统可用资源增加 30%，贪婪算法选择复制的软件组件为 c_2 、 c_7 和 c_{12} ，启发式的贪婪复制算法选择复制的软件组件为 c_2 、 c_8 、 c_{10} 和 c_{12} 。这两个复制算法都选择提供服务重要性高的软件组件进行复制。贪婪算法复制软件组件后，消除了复制前系统中的薄弱点 C_2 ，但软件组件 C_8 提供的服务可靠性仍然很低。而且，软件组件复制后， C_8 提供的服务可靠性与其它服务可靠性的差距加大。启发式的贪婪复制算法复制软件组件后， C_2 和 C_8 提供的服务可靠性明显提高，消除复制前系统中的薄弱点 C_2 和 C_8 。

当系统可用资源增加 40%时，采用启发式的贪婪复制算法和贪婪算法选择需要被复制的软件组件，软件组件提供的服务可靠性如图 5-7 所示。G40 表示采用贪婪算法复制软件组件后，软件组件提供的服务可靠性，A40 表示采用启发式的贪婪复制算法选择复制的软件组件后，软件组件提供的服务可靠性。

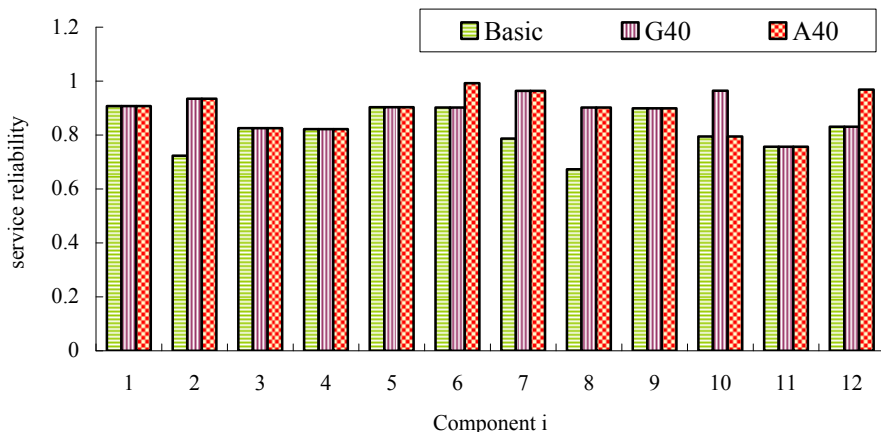


图 5-7 系统资源增加 40%时组件的可靠性变化

Fig. 5-7 Difference in component reliabilities of system resources increased by 40%

从图 5-7 可以看出，当系统可用资源增加 40%时，贪婪算法选择复制的软件组件为 c_2 、 c_7 、 c_8 和 c_{10} ，启发式的贪婪复制算法选择复制的软件组件为 c_2 、 c_7 、 c_8 、 c_6 和 c_{12} 。当系统可用资源受限时，两种算法都选择软件组件可靠性低和其提供服务重要性高的 c_2 和 c_8 来复制，消除了复制前系统中的薄弱点，使系统更稳定地提供服务。与贪婪算法相比，启发式的贪婪复制算法能充分利用有限的可用系统资源，更有效地选择出将被复制的软件组件，提高更多服务的可靠性。

当给定有限的系统可用资源时，为了更清楚地看出系统中服务可靠性的整体变化，给出了服务可靠性的均值和标准差的比较，如表 5-3 所示。

表 5-3 给定不同可用系统资源时服务可靠性的均值和标准差的比较

Table 5-3 Comparison of mean value and standard deviation of service reliability in different available system resources

指标	Basic	G30	A30	G40	A40
平均值	0.8188	0.8626	0.8807	0.8843	0.8892
标准差	0.0765	0.0895	0.0674	0.0629	0.0741

从表 5-3 可以看出，当系统可用资源增加 30%和 40%时，启发式的贪婪复制算法得到的服务可靠性的平均值都高于贪婪算法，与复制前的服务可靠性相比，整个服务的可靠性有了明显的提高。当系统可用资源增加 30%时，启发式的贪婪复制算法得到的服务可靠性的标准差低于复制前，也就是系统中各个服务的可靠性差距变小。贪婪复制算法得到的服务可靠性的标准差高于复制前，

使系统中各个服务的可靠性差距变大。另外，启发式的贪婪复制算法得到的服务标准差低于贪婪算法，表明系统中各个服务的可靠性差别更小，不存在提高整体可靠性的薄弱点。当系统可用资源增加 40% 时，启发式的贪婪复制算法得到的服务标准差高于贪婪算法，再结合图 5-7 可以看出，复制软件组件 C_6 和 C_{12} ，其服务可靠性提高幅度较大，使得服务的可靠性差距变大，而这是一种有益的提高。

5.4 故障传播对服务可靠性影响的分析

可靠性是一个具有不同意义、特点和与不同内容相关的复杂属性，需要从不同角度理解失效行为、给定的使用剖面和恢复机制等等^[140]。对于有软件组件复制的系统，服务失效是引起系统失效的主要原因。如果软件组件失效，其组件副本仍能继续提供服务，这时系统仍继续正常工作。但是，失效的软件组件在崩溃的过程中，会将故障传播给与其进行交互的其它软件组件。故障传播分析可以用来找到系统中最容易暴露出问题的模块，确定在错误没有发生前不同模块间如何相互影响。除了获知故障传播的特征，了解故障可能造成的最大损害也同等重要。值得注意的是，最有可能传播的故障不是必然造成最大损害的。故障传播影响的分析可以作为其它的分析活动的补充^[141]。因此，本节主要分析软件组件失效和主机节点失效所引起的故障传播对服务可靠性的影响。

5.4.1 实验输入

实验的输入包括 4 个主机节点的失效率、8 个软件组件的失效率、随机产生的软件组件在主机节点上的部署架构和软件组件间的交互关系。 ph_i 表示主机节点 H_i 的失效率， ph_i 为 $[0, 0.001]$ 间的任意实数， $ph_1 = 0.001$ ， $ph_2 = 0.0001$ ， $ph_3 = 0.0006$ ， $ph_4 = 0.0009$ 。 pc_i 表示软件组件 C_i 的失效率， pc_i 为 $[0, 0.2]$ 间的任意实数， $pc_1 = 0.07$ ， $pc_2 = 0.11$ ， $pc_3 = 0.003$ ， $pc_4 = 0.008$ ， $pc_5 = 0.11$ ， $pc_6 = 0.18$ ， $pc_7 = 0.15$ ， $pc_8 = 0.09$ 。

系统初始时，这 8 个软件组件都能提供一种不同的服务。随机产生的部署架构和软件组件间的交互关系，如图 5-8 所示。软件组件 C_1 和 C_2 部署到主机节点 H_1 上，软件组件 C_3 和 C_4 部署到主机节点 H_2 上，软件组件 C_5 和 C_6 部署到主机节点 H_3 ，软件组件 C_7 和 C_8 部署到主机节点 H_4 。点线描述主机节点间的网络物理链接，实线描述软件组件间的逻辑链接关系。

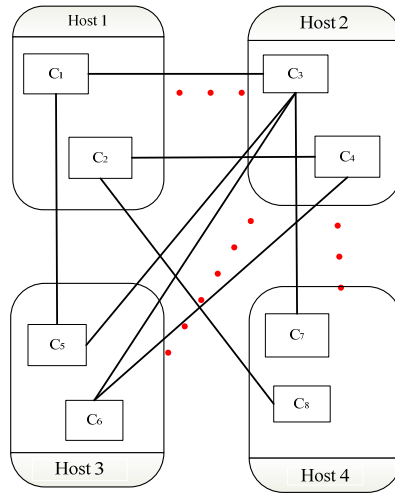


图 5-8 软件组件在主机节点上的部署图

Fig. 5-8 Deployment graph of software components deployed on host nodes

5.4.2 相关软件组件失效引起的故障传播分析

本实验分析软件组件失效后引起的故障传播。软件组件的失效原因可能是内部代码执行引起的，或者是外部运行环境引起。软件组件内部的故障数越多，执行的频率越高，软件组件越容易失效。软件组件通过接口与其它软件组件进行交互，如果某个软件组件失效，那么在这个软件组件崩溃的过程，会通过接口将故障传播给与其进行直接交互的其它软件组件。这里设定，如果某个软件组件失效，与其交互的软件组件的失效概率都将增加 0.01。软件组件失效对系统服务的影响如图 5-9 所示。

在图 5-9 中，Original 表示没有软件组件失效的情况下各服务的可靠性。C1C3F 表示在没有软件组件复制的情况，软件组件 C_1 和 C_3 失效的情况下各服务的可靠性。C1C3FWR 表示复制了软件组件 C_1 和 C_3 的情况下，软件组件 C_1 和 C_3 失效时的各服务的可靠性。C1C3WR 表示在没有软件组件失效的情况下，复制软件组件 C_1 和 C_3 后的各服务的可靠性。

结合图 5-8 和 5-9 可以得到，当没有软件组件复制的时候，软件组件 C_1 和 C_3 失效，即 C1C3F 的情况。此时，软件组件 C_1 和 C_3 将不能提供相应的服务，其提供服务的可靠性变为 0。对于软件组件 C_5 ， C_5 只与软件组件 C_1 和 C_3 进行交互， C_1 和 C_3 的同时失效，使 C_5 不能完成规定的服务， C_5 也失效， C_5 提供的服务可靠性也为 0。同理，软件组件 C_7 只与 C_3 进行交互， C_3 的失效使得 C_7 不能完成规定的服务，软件组件 C_7 提供的服务可靠性也变为 0。对于软件组件 C_6 ，它与 C_3 和 C_4

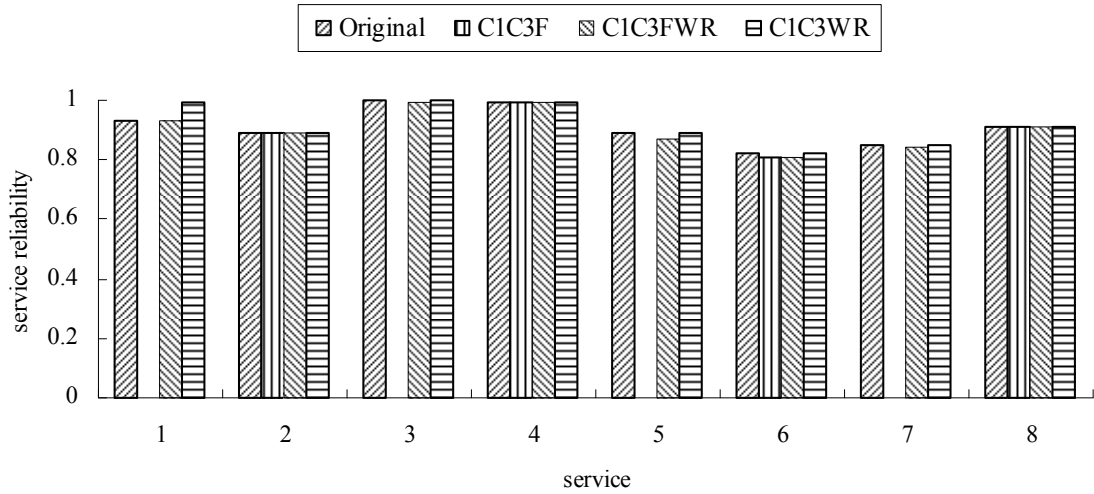


图 5-9 软件组件失效对服务可靠性影响

Fig. 5-9 Failure influence of software components on service reliability

进行交互。 C_3 的失效虽然不能使 C_6 失效，但是会将故障传播给 C_6 ， C_6 提供的服务的失效率增加0.01。对于软件组件 C_4 、 C_2 和 C_8 ，它们都间接地与 C_3 进行交互。软件组件 C_3 的失效使得 C_6 的故障数增加，软件组件 C_6 的失效率增加，但仍可以为 C_4 提供规定的服务，所以软件组件 C_4 提供服务的可靠性没有受到影响。软件组件 C_4 能正常为 C_2 提供服务，软件组件 C_2 仍能正常为 C_8 提供服务，也就是 C_4 、 C_2 和 C_8 都没有受到影响，其提供服务的失效率不变。另外，从故障传播的路径看，只有软件组件 C_5 受到两次故障传播的影响，在这样情况下，软件组件 C_5 属于系统中比较脆弱点。因此，在进行软件组件复制时，也要重点考虑这种多个故障传播路径都经过的薄弱点。

当软件组件 C_1 和 C_3 被复制，将软件组件 C_1 的副本部署到主机节点 H_2 上，软件组件 C_3 的副本部署到主机节点 H_3 上，也就是C1C3FWR描述的情况。在这种情况下，即使 C_1 和 C_3 同时失效，它们的副本仍能提供相应的服务。对于软件组件 C_5 ，与 C_1 和 C_3 交互，也与 C_1 和 C_3 的副本进行交互，当 C_1 和 C_3 失效时，软件组件 C_5 可以继续与它们的副本进行交互，但是这样的失效会将故障传播给软件组件 C_5 。由于软件组件 C_5 受到两个失效软件组件的影响，其提供的服务失效率增加0.02。对于软件组件 C_6 ，受到 C_3 失效的影响，其提供服务的失效率增加0.01。对于软件组件 C_7 ，只与软件组件 C_3 进行交互，尽管 C_3 失效，但是 C_7 可与 C_3 的副本继续进行交互，也就是只 C_7 受到一个软件组件失效的影响，其提供服务的失效率增加0.01。对于软件组件 C_4 、 C_2 和 C_8 都没有受到影响，其提供服务的失效率不变。

当软件组件 C_1 和 C_3 被复制，此时没有软件组件失效，也就 C1C3WR 描述的情况。软件组件 C_1 的副本部署到主机节点 H_2 上，软件组件 C_3 的副本部署到主机节点 H_3 上。只有当软件组件 C_1 和 C_1 的副本同时失效，提供的相应服务才失效。类似，只有当软件组件 C_3 和 C_3 的副本同时失效，提供的相应服务才会失效，因此，软件组件 C_1 和 C_3 提供的服务的可靠性将增加。对于其它的软件组件，由于没有受到交互的软件组件失效的影响，其提供服务的失效率保持不变。

5.4.3 相关主机节点失效引起的故障传播分析

本实验分析主机节点失效引起的故障传播。当主机节点失效时，部署在其上的软件组件将不能为其它软件组件提供服务或者请求服务，也就是软件组件不能完成规定的功能，此时部署在失效主机节点上的软件组件将失效。上一个实验分析，当软件组件失效时，可通过复制来降低软件组件失效对服务可靠性的影响。本实验除了通过复制软件组件来降低失效的影响外，还可以通过软件组件在主机节点上的重部署来降低失效的影响。这里仍假设失效的软件组件会将故障传播给与其直接交互的其它软件组件，使其提供服务的失效率增加 0.01。主机节点失效对各服务的可靠性影响如图 5-10 所示。

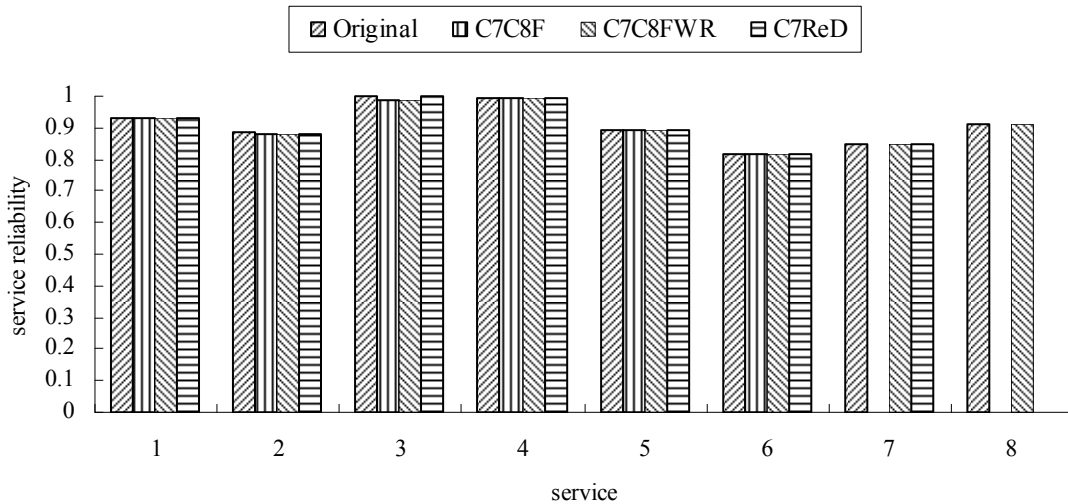


图 5-10 主机节点失效对服务可靠性的影响

Fig. 5-10 Failure influence of host nodes on service reliability

在图 5-10 中，Original 表示没有软件组件失效，没有主机节点失效的情况下的各服务的可靠性。C7C8F 表示在软件组件 C_7 和 C_8 没有组件复制的情况，主机节点 H_4 失效时各服务的可靠性。C7C8FWR 表示在软件组件 C_7 和 C_8 有组件复制

的情况，软件组件 C_7 的副本部署到主机节点 H_3 上，软件组件副本 C_8 部署到主机节点 H_2 上，主机节点 H_4 失效时各服务的可靠性情况。C7ReD 表示在没有软件组件复制的情况，软件组件 C_7 重部署到主机节点 H_1 上，主机节点 H_4 失效时各服务的可靠性情况。

结合图 5-8 和 5-10 可以得到，在 C7C8F 描述的情况下，由于主机节点 H_4 失效，则部署在主机节点 H_4 上的软件组件 C_7 和 C_8 不能为其它软件组件提供服务，也不能向其它软件组件请求服务，不能完成规定的功能，软件组件 C_7 和 C_8 也失效，其提供服务的可靠性变为 0。由于软件组件 C_2 与 C_8 直接交互， C_8 失效将故障传播到 C_2 ，软件组件 C_2 提供服务的失效率增加 0.01。由于软件组件 C_3 与 C_7 直接交互， C_7 的失效将故障传播给 C_3 ， C_3 提供服务的失效率增加 0.01。对于软件组件 C_1 、 C_4 、 C_5 和 C_6 ，没有与软件组件 C_7 和 C_8 直接交互， C_7 和 C_8 的失效没有将故障传播到这些软件组件，这些软件组件提供服务的失效率不变。

在 C7C8FWR 描述的情况下，主机节点 H_4 失效，则部署在主机节点 H_4 上的软件组件 C_7 和 C_8 也失效，但软件组件 C_7 的副本和 C_8 副本仍可以继续提供服务。软件组件 C_3 虽然可以与 C_7 的副本进行交互，但是 C_7 失效的故障仍会传播给 C_3 ， C_3 提供服务的失效率增加 0.01。在 C_8 失效的情况下，软件组件 C_2 可以与 C_8 的副本进行交互，但仍会受到软件组件 C_8 失效的影响， C_2 提供服务的失效率增加 0.01。对于软件组件 C_1 、 C_4 、 C_5 和 C_6 ，没有与软件组件 C_7 和 C_8 直接交互， C_7 和 C_8 的失效没有将故障传播给这些软件组件，这些软件组件提供服务失效率不变。

在 C7ReD 描述的情况下，在主机节点 H_4 失效的情况下，考虑将部署在其上的软件组件进行重部署，选择将软件组件 C_7 重新部署到主机节点 H_1 上。当主机节点 H_4 失效时，部署在其上的软件组件 C_8 将失效，其提供的服务的可靠性将变为 0。由于软件组件 C_7 已部署到主机节点 H_1 上，故软件组件 C_7 没有失效，但是软件组件 C_7 重部署前后的主机节点的可靠性不同，软件组件 C_7 提供服务的可靠性也有变化。如果主机节点 H_1 的可靠性高于 H_4 的可靠性，那么软件组件 C_7 提供服务的可靠性增加。如果主机节点 H_1 的可靠性低于 H_4 的可靠性，那么软件组件 C_7 提供服务的可靠性降低。软件组件 C_2 与 C_8 直接进行交互， C_8 的失效将故障传播到 C_2 ，软件组件 C_2 提供的服务的失效率将增加 0.01。软件组件 C_3 与 C_7 直接进行交互，因为软件组件 C_7 重部署后没有失效，所以 C_3 提供的服务的失效率不变。对于软件组件 C_1 、 C_4 、 C_5 和 C_6 ，没有与软件组件 C_8 直接交互， C_8 的失效没有将故障传播到这些软件组件，这些软件组件提供服务失效率不变。

5.5 本章小结

在分析贪婪算法和 OF 的基础上，本章设计了启发式的贪婪复制算法，该算法初始化软件组件的单位带宽的优化值增加量 OB 和单位内存的优化值增加量 OM ，根据 OB 和 OM 利用贪婪算法分别选择出将被复制的软件组件，两者中得到的总优化值更高的软件组件集合是该算法被选定需要复制的软件组件集合。之后对该算法进行正确性和时间复杂度分析。当增加相同的可用系统资源时，启发式的贪婪复制算法能更充分利用系统可用资源，更有效地选择出将被复制的软件组件集合，从而得到更高优化值。最后，对相关软件组件失效和主机节点失效引起的故障传播进行分析。

第6章 组件时间相关失效率的软件部署可靠性研究

6.1 引言

大多数架构层次的可靠性预测方法^[46,55,69,142-145]都认为软件组件可靠性对系统可靠性有重要影响，并假设系统中的组件可靠性已知，或者假设给定组件提供的服务可靠性已知。第3章和第4章主要针对特定的时间、特定的使用剖面下，通过重部署方法来提高系统可靠性。第5章分析在特定的时间、特定的使用剖面下，通过软件组件复制方法来提高系统可靠性，并对软件组件失效和主机节点失效引起的故障传播进行分析。这三章都是在特定时间下进行可靠性分析，因此给定的软件组件失效率都是常数。在整个系统运行期间，软件组件受内部因素和外部因素的共同影响，内部因素例如软件组件的内部执行代码、异常、服务请求等等，外部因素如波动的网络、可充电电池的电量、主机节点的可靠性、软件组件间的交互频率等等。随着系统运行时间的增加，这些影响因素的值将发生变化，对软件组件的失效影响也将发生变化，也就是软件组件的失效率是一个与时间相关的函数。Xie 和 Wohlin 首先提出可加模型^{[61]188-194}，这里称之为 Xie 模型。Xie 模型假设组件软件系统为由 n 个组件组成的串行系统，所有组件并行开发，独立测试，组件失效均为系统失效，并假设每个组件的失效行为可建模为 NHPP。本章采用 Xie 模型的假设，将部署阶段的软件组件的失效行为建模为 NHPP，分析一段时间内参数 ρ 和软件组件间的交互频率矩阵对可靠性的影响，比较重部署和软件组件复制方法对提高系统可靠性的差别，并分析集成架构风格的可靠性。

6.2 NHPP类软件组件模型

6.2.1 NHPP类模型

NHPP 类模型已经成功应用于软件可靠性工程实践中，成为管理和提高软件可靠性过程的重要工具。在过去 30 年中，人们提出了很多 NHPP 类软件可靠性增长模型，其中经典的模型包括 J-M 模型^[146]、G-O 模型^[147]、Musa 基本模型^[148]、MO 模型^[149]、S-形模型^{[150]475-484}、Yamada 指数模型^{[150]475-484}、Yamada 瑞利模型^[151]等。提高 NHPP 模型可靠性评估和预测的准确度目前仍是软件可靠性工程领域的研究热点之一^[152-156]。下面介绍泊松过程和非齐次泊松过程的定义。

定义 6-1 泊松过程 (Poisson process) ^{[157]56}: 计数过程 $\{N(t), t \geq 0\}$ 称为泊松过程, 如果满足条件:

- ① $N(0)=0$;
- ② 具有独立增量;
- ③ 对于 $s, t \geq 0$,

$$P\{N(t+s)-N(s)=k\}=e^{-\lambda t} \frac{(\lambda t)^k}{k!}, \quad k=0,1,\dots \quad (6-1)$$

条件 (3) 说明泊松过程有平稳增量, 因为 $N(t+s)-N(s)$ 的分布只依赖于 t , 与区间起点 s 无关。(3) 中令 $s=0$, 得

$$P\{N(t)=k\}=e^{-\lambda t} \frac{(\lambda t)^k}{k!}, \quad k=0,1,\dots \quad (6-2)$$

所以

$$m(t)=EN(t)=\lambda t \quad (6-3)$$

式中 $m(t)$ ——在 $[0,t]$ 时间间隔中事件出现次数的均值函数;

λ ——泊松过程的强度。

在实际中, 很多随机现象都满足 ①, ②, 但是是否满足 ③ 似乎不很明确。为此给出另一描述性的定义。

定义 6-2 泊松过程 ^{[157]56-57}: 计数过程 $\{N(t), t \geq 0\}$ 称为泊松过程, 如果满足条件:

- ① $N(0)=0$;
- ② 具有平稳增量与独立增量;
- ③ $P\{[N(t+\Delta t)-N(t)]=1\}=\lambda\Delta t+o(\Delta t)$;
- ④ $P\{[N(t+\Delta t)-N(t)]\geq 2\}=o(\Delta t)$ 。

定义中条件 ③ 表明, 在充分小的时间间隔中事件出现一次的概率与时间间隔的长度成正比, 条件 ④ 表示很小的时间间隔中时间不能出现两次或者两次以上。

定义 6-3 非齐次泊松过程 (NHPP) ^{[157]63}: 计数过程 $\{N(t), t \geq 0\}$ 称为具有强度函数 $\lambda(t)$ ($t \geq 0$) 的非齐次泊松过程, 如果:

- ① $N(0)=0$;
- ② 具体独立增量;
- ③ $P\{[N(t+\Delta t)-N(t)]=1\}=\lambda(t)\Delta t+o(\Delta t)$;
- ④ $P\{[N(t+\Delta t)-N(t)]\geq 2\}=o(\Delta t)$ 。

非齐次泊松过程不再有平稳增量,也就是说概率 $P\{N(t+s)-N(t)=k\}$ 不但依赖于 s , 也与 t 有关。这也反映在不同的 t 有不同的强度 $\lambda(t)$ 。

定理 6-1 NHPP 过程 $\{N(t), t \geq 0\}$ 在时间间隔 $[t_0, t_0+t)$ 内出现 A 事件 n 次的概率为^{[59]64}:

$$\Pr\{[N(t_0+t)-N(t_0)]=n\}=\frac{[m(t_0+t)-m(t_0)]^n}{n!}e^{-[m(t_0+t)-m(t_0)]} \quad (6-4)$$

式中 $m(t)$ ——在时间段 $(0, t)$ 内发生事件 A 次数的均值, $m(t)=\int_0^t \lambda(s)ds$;

$\lambda(s)$ ——在时刻 s , 事件 A 单位时间内发生的次数。

由定理(6-1)可以得到, 在时间间隔 $[t_0, t_0+t)$ 内 NHPP 过程 $\{N(t), t \geq 0\}$ 不发生事件 A 的概率为:

$$\Pr\{[N(t_0+t)-N(t_0)]=0\}=e^{-[m(t_0+t)-m(t_0)]} \quad (6-5)$$

6.2.2 软件组件建模

组件是由一方来定义其规格说明, 然后被另一方实现。组件只能通过接口与其它组件进行交互^[34]。组件的使用者可利用接口的规格说明来获取足够的组件信息。

当没有足够的失效数据时, 精确评估单个软件组件的可靠性是困难的, 因此做一些合理的假设是必要的。这里用 Xie 模型的假设, 将部署阶段的软件组件失效行为建模成 NHPP, 从而得到软件组件的可靠性如公式(6-6)所示。

$$RC_i = e^{-\lambda_i t} \quad (6-6)$$

式中 RC_i ——软件组件 C_i 的可靠性;

λ_i ——软件组件 C_i 的失效率。

Cheung^[158]最早提出基于架构的可靠性建模方法, 他强调这样的事实: 如果很少使用不可靠的部分, 那么系统将更可靠。Yacoub 提出了 ALRRA 方法来分析架构层次的可靠性风险, 该方法基于如下的假设: 组件执行的频率越高, 失效的可能性越大^{[159-160] 529-547}。软件组件的执行频率是导致其失效的一个重要原

因，因此，将 λ_i 设为软件组件执行频率的函数，由公式（6-7）求得。

$$\lambda_i = \lambda_0 + \rho \times fc_i + f_{other} \times \rho_{other} \quad (6-7)$$

式中 λ_0 ——软件组件固有的可能失效的概率；

ρ ——单次执行引起软件组件失效的概率；

f_{other} ——其它引起软件组件失效的因素；

ρ_{other} ——其它因素引起组件失效的概率；

fc_i ——软件组件的执行频率，可由公式（6-8）求得。

$$fc_i = \sum_{\forall C_j, H(C_i) \neq H(C_j)} cc_{ij} \quad (6-8)$$

在公式（6-8）中， $H(C_i)$ 表示软件组件 C_i 部署到的主机节点， $cc_{i,j}$ 描述了软件组件 C_i 和 C_j 间的交互频率。由于移动软件动态性的本质，主机节点的移动可能引起网络失效，从而影响软件组件间的交互频率，类似地，如果一个软件组件从一个主机节点迁移到另一个主机节点，软件组件和其它软件组件间交互行为也可能发生很大改变。这种影响可能超出工程师对软件组件可靠性和系统可靠性的原始预期。初始的软件组件间的交互频率可由系统工程师或者软件架构师给出，在系统运行期间，可以检测到软件组件间运行时的交互^[161]。

6.2.3 实验及结果分析

实验系统包括 12 个原始软件组件和 4 个主机节点。每个原始软件组件提供一种不同的服务。这里考虑软件组件的失效率主要与软件组件的执行频率相关，相关参数的取值为 f_{other} 的值为 0， ρ_{other} 的值为 0， $\lambda_0 = 0.0000001$ 。 ph_i 表示主机节点 H_i 的失效概率，为 $[0, 0.01]$ 间的任意实数， $ph_1 = 0.0045$ ， $ph_2 = 0.0093$ ， $ph_3 = 0.0011$ ， $ph_4 = 0.006$ 。随机产生软件组件间交互频率矩阵 CCS ，矩阵中的每个元素为 $[0,9]$ 间的任意整数。

$$CCS = \begin{bmatrix} 0 & 9 & 1 & 7 & 3 & 4 & 8 & 1 & 2 & 0 & 4 & 3 \\ 9 & 0 & 2 & 5 & 1 & 8 & 5 & 0 & 4 & 6 & 3 & 9 \\ 1 & 2 & 0 & 2 & 1 & 8 & 3 & 2 & 0 & 2 & 8 & 7 \\ 7 & 5 & 2 & 0 & 6 & 6 & 7 & 2 & 0 & 3 & 0 & 4 \\ 3 & 1 & 1 & 6 & 0 & 0 & 5 & 8 & 5 & 7 & 7 & 7 \\ 4 & 8 & 8 & 6 & 0 & 0 & 4 & 7 & 4 & 6 & 9 & 2 \\ 8 & 5 & 3 & 7 & 5 & 4 & 0 & 1 & 5 & 4 & 0 & 4 \\ 1 & 0 & 2 & 2 & 8 & 7 & 1 & 0 & 3 & 5 & 7 & 9 \\ 2 & 4 & 0 & 0 & 5 & 4 & 5 & 3 & 0 & 7 & 4 & 0 \\ 0 & 6 & 2 & 3 & 7 & 6 & 4 & 5 & 7 & 0 & 4 & 2 \\ 4 & 3 & 8 & 0 & 7 & 9 & 0 & 7 & 4 & 4 & 0 & 8 \\ 3 & 9 & 7 & 4 & 7 & 2 & 4 & 9 & 0 & 2 & 8 & 0 \end{bmatrix}$$

前面的章节已经讨论过如何将软件组件有效地部署到主机节点上。这里为

了实验的简明，随机选择一种部署架构，软件组件 C_1 、 C_2 和 C_4 部署到主机节点 H_1 上，软件组件 C_3 和 C_5 部署到主机节点 H_2 上，软件组件 C_7 、 C_8 和 C_{10} 部署到主机节点 H_3 上，软件组件 C_6 、 C_9 、 C_{11} 和 C_{12} 部署到主机节点 H_4 上，称为第一种部署架构。这里，如果软件组件没有组件副本，那么任一软件组件的失效都将导致系统失效；如果软件组件有组件副本，只有软件组件和它的组件副本都失效，才导致系统失效。

(1) 参数 ρ 对可靠性的影响

首先分析参数 ρ 对系统可靠性的影响， ρ 有三种取值，分别为 $\rho=0.00001$ ， $\rho=0.00002$ 和 $\rho=0.00004$ 。软件组件的可靠性用公式 (6-6) 来建模。这里采用第一种部署架构， ρ 对系统可靠性影响如图 6-1 所示。

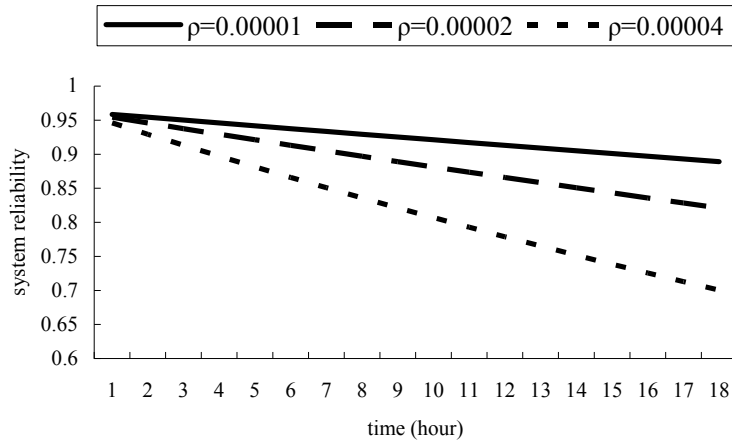


图 6-1 ρ 对系统可靠性的影响

Fig.6-1 Influence of the ρ value on system reliability

在图 6-1 中， $\rho=i$ 表示 ρ 为 i 时的系统可靠性。随着 ρ 值的增加，系统可靠性降低。当 ρ 取值不同时，系统可靠性的差别比较大，如在 $\text{time}=5$ 时， $\rho=0.00001$ 情况下的系统可靠性为 0.9417，而 $\rho=0.00004$ 情况下的系统可靠性为 0.8813。并且随着系统运行时间的增加，系统可靠性的差别在显著增加，因此 ρ 值的确定很重要。 ρ 值可由软件测试阶段，通过得到的失效数据给出 ρ 的估计值，也可以由软件架构师凭借经验给定初始值，并可以在系统运行过程中调整。

之后分析参数 ρ 对软件组件可靠性的影响。 ρ 有三种取值，分别为 $\rho=0.00001$ ， $\rho=0.00002$ ， $\rho=0.00004$ 。软件组件的可靠性用公式 (6-6) 来建模。这里采用第一种部署架构，并且时间 time 取值为 50， ρ 对软件组件可靠性影响

如图 6-2 所示。

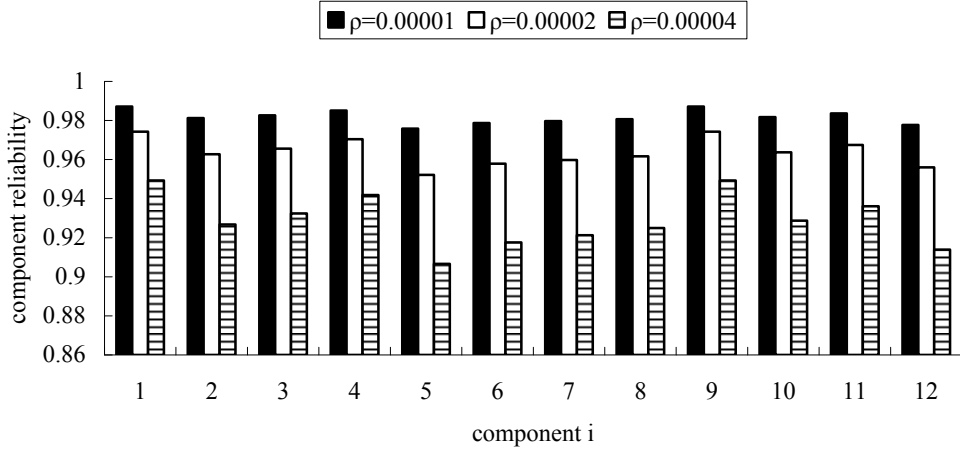


图 6-2 ρ 对软件组件可靠性的影响

Fig.6-2 Influence of the value of ρ on reliability of software components

在图 6-2 中， $\rho=i$ 表示 ρ 为 i 时的各个软件组件的可靠性。从图 6-2 可以看出，随着 ρ 值的增加，各软件组件的可靠性都在下降。对于同一个软件组件，不同 ρ 值下的软件组件可靠性差别较大，可以看出 ρ 值的确定对软件组件的可靠性也有重要影响。另外， ρ 值对软件组件 C_5 的影响较大，而对软件组件 C_1 的影响较小。如果 C_5 是提供关键服务的软件组件，那么需要考虑复制软件组件 C_5 ，使 C_5 提供服务的可靠性提高，从而提高系统可靠性，使系统更稳定。

(2) 软件组件复制对可靠性的影响

对于由多个主机节点构成的系统，一个主机节点的失效不意味着整个系统失效。因为，具有冗余软件组件副本，即使某些软件组件失效，软件组件的副本仍能继续提供服务。组件副本的创建并不是无休止的，需要遵循最大副本数的限制。例如可以将最大数设置为系统中的主机节点数目。这里，软件组件复制有 4 个研究假设：

① 由 M 个主机节点和 N 个软件组件构成的系统，如果软件组件 C_i 被选定为复制的候选组件， C_i 应该被复制 q_i ($q_i \leq M-1$) 次。也就是说， C_i 有 q_i 个组件副本，或者 (q_i+1) 个软件组件提供相同的服务；

② 软件组件和它的副本不能部署在相同的主机节点上；

③ 同一软件组件的任意两个组件副本不能部署在相同的主机节点上；

④ 如果有多个软件组件提供相同的服务，请求组件将接收最快响应的组件所提供的服务。也就是，这里的软件组件复制采用主动复制技术。

根据上面的4个假设，被复制的软件组件 C_i 提供的服务可靠性 SRC_i 可由公式(6-9)求得。在公式(6-9)中，为了不用考虑软件组件副本的放置问题， $q_i = M - 1$ ，软件组件 C_i 有 $M - 1$ 个组件副本，系统所有的主机节点或者包含软件组件 C_i ，或者包含它的组件副本。只有软件组件 C_i 和它的组件副本都失效时，其组件提供的服务才失效，这时 $\lambda_i = \lambda_0$ ， ph_j 表示主机节点 H_j 的失效率。 $H(C_j)$ 表示软件组件 C_j 被部署到的主机节点。

$$SRC_i = 1 - \prod_{\forall j, H_j \in H(C_i)} (1 - (1 - ph_j) \times e^{-\lambda_0 t}) \quad (6-9)$$

根据第5章的研究可知，根据可用系统资源，利用软件组件复制算法来选择将被复制的软件组件，从而提高目标优化值。这里为了使实验简明，随机地选择软件组件 C_3 和 C_5 进行复制，并且不考虑软件组件副本的放置问题。软件组件复制对系统可靠性的影响如图6-3所示。

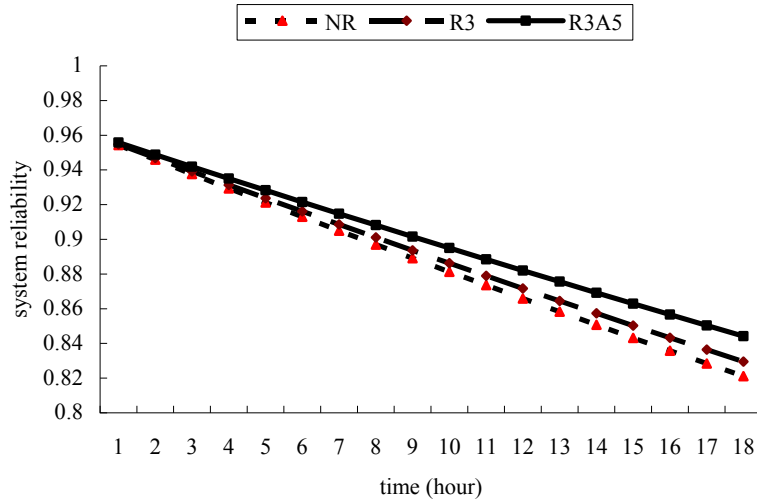


图 6-3 软件组件复制对系统可靠性的影响

Fig.6-3 Influence of software component replica on system reliability

在图6-3中， $\rho = 0.00002$ ，软件组件在主机节点上的部署采用第一种部署架构。NR表示没有软件组件复制时系统可靠性变化情况，R3表示复制软件组件 C_3 后的系统可靠性变化情况，R3A5表示复制软件组件 C_3 和 C_5 后的系统可靠性变化情况。从图6-3可以看出，软件组件复制能提高系统的可靠性，复制多个软件组件得到的可靠性高于复制单个软件组件得到的可靠性。由于每个软件组件

提供服务的重要性不同和软件组件的可靠性不同，确定复制哪些软件组件对系统可靠性有重要影响。此外，确定合理的软件组件副本数目很重要，副本的数目可以根据经验来配置，如果初始数目较多，可能造成系统资源浪费；如果初始数目较少，某些软件组件提供服务的可靠性不高，部分服务可能经常面临失效，系统可靠性得不到有效地保证。

(3) 重部署和组件复制对可靠性提高的比较

由前面的研究知，系统重部署方法和软件组件复制方法可以提高系统可靠性。本实验分析两种方法的优缺点，给出两种方法的适用条件。为了使实验简明，随机产生一种部署架构、随机地选择将被复制的软件组件和随机选择一种较好的部署架构来重部署系统。这里， $\rho = 0.00002$ 。当系统可靠性低于 0.9 时，系统重部署或者进行软件组件复制，如图 6-4 所示。

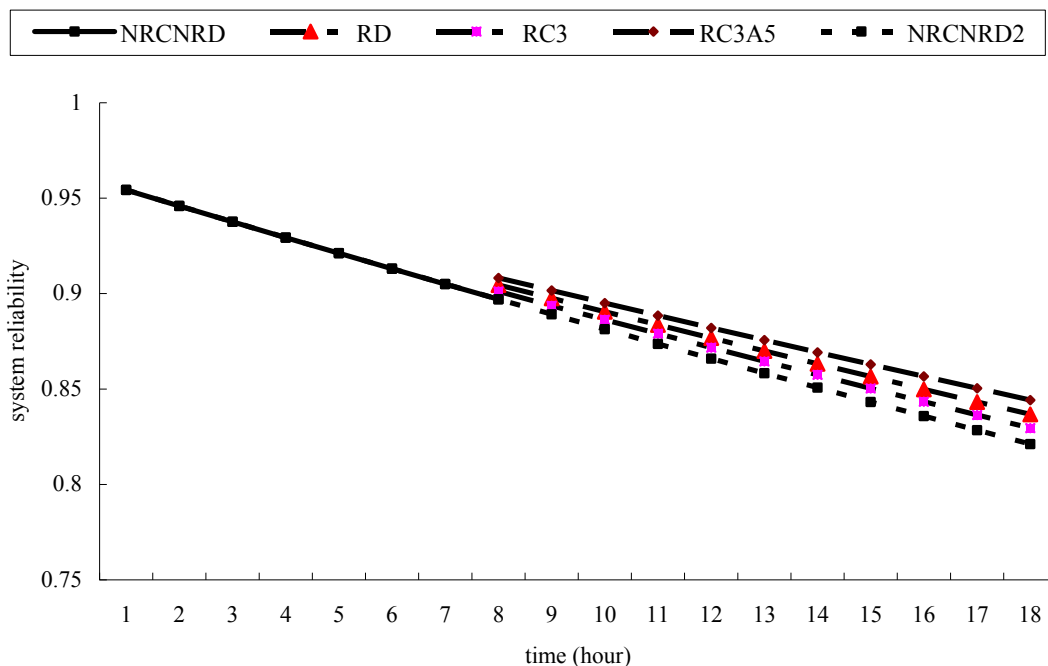


图 6-4 重部署和组件复制对系统可靠性提高的比较

Fig.6-4 Comparison of system reliability improvement by redeployment and component replication

在图 6-4 中，NRCNRD 系统采用第一种部署架构，在时间段[1,8]内的系统可靠性变化情况。RD 表示在时间段[8,18]内的系统重部署后的系统可靠性变化情况。系统重部署后，软件组件 C_1 、 C_2 和 C_9 部署到主机节点 H_1 上，软件组件 C_3 、 C_6 和 C_{11} 部署到主机节点 H_2 上，软件组件 C_5 、 C_8 、 C_{10} 和 C_{12} 部署到主机节点 H_3 上，软件组件 C_4 和 C_7 部署到主机节点 H_4 上。RC3 表示在时间段[8,18]内的复制软件

组件 C_3 的系统可靠性变化情况。RC3A5 表示在时间段[8,18]内的复制软件组件 C_3 和 C_5 的系统可靠性变化情况。NRCNRD2 表示在时间段[8,18]内的没有复制软件组件和重部署的系统可靠性变化情况。

从图 6-4 可以看出，复制软件组件 C_3 和 C_5 的系统可靠性高于系统重部署后的可靠性，系统重部署后的可靠性高于复制软件组件 C_3 后的可靠性。系统重部署过程中，某些软件组件需要从一个主机节点迁移到另外一个主机节点，基本不需要消耗额外的系统资源，但某些软件组件将暂时不能向其它软件组件提供服务或请求服务。复制软件组件需要额外消耗系统资源，但是不需暂停软件组件的提供服务或者请求服务。当系统可靠性低于某一规定值时，如果系统可用资源较多，并且对时间要求较严，可以考虑通过软件组件复制来提高系统可靠性；如果系统可用资源较少，并且允许系统某些服务暂时中断，可以考虑通过重部署来提高系统可靠性。

(4) 交互频率矩阵类型对可靠性的影响

由前面的分析可知，软件组件的可靠性建模依赖于组件间的交互频率矩阵。交互频率矩阵记录了任意两个软件组件间的交互次数，而交互的频繁程度和软件组件提供服务或请求服务的重要性有直接关系。例如，在 CS 的软件架构风格中，服务器组件的接收请求和发送响应信息的次数较多，那么服务器组件的交互频率很高。另外，根据第 5 章的分析可知，通过软件组件复制能提高可靠性。然而，当进行软件组件复制时，又将容错风格引入进来。在第 5 章中，软件组件的复制是采用主动复制，其它的软件组件要与软件组件及其它的组件副本同时进行交互。此时，软件组件间的交互频率矩阵也发生变化，也就是软件架构风格和软件组件间的交互频率矩阵间有着重要的关系，有必要对交互频率矩阵进行深入的研究。将组件间的交互频率的对称矩阵 CCS，交换某些行可得到非对称矩阵 CCA。

$$CCA = \begin{bmatrix} 4 & 8 & 8 & 6 & 0 & 0 & 4 & 7 & 4 & 6 & 9 & 2 \\ 9 & 0 & 2 & 5 & 1 & 8 & 5 & 0 & 4 & 6 & 3 & 9 \\ 3 & 9 & 7 & 4 & 7 & 2 & 4 & 9 & 0 & 2 & 8 & 0 \\ 7 & 5 & 2 & 0 & 6 & 6 & 7 & 2 & 0 & 3 & 0 & 4 \\ 3 & 1 & 1 & 6 & 0 & 0 & 5 & 8 & 5 & 7 & 7 & 7 \\ 0 & 9 & 1 & 7 & 3 & 4 & 8 & 1 & 2 & 0 & 4 & 3 \\ 1 & 0 & 2 & 2 & 8 & 7 & 1 & 0 & 3 & 5 & 7 & 9 \\ 8 & 5 & 3 & 7 & 5 & 4 & 0 & 1 & 5 & 4 & 0 & 4 \\ 2 & 4 & 0 & 0 & 5 & 4 & 5 & 3 & 0 & 7 & 4 & 0 \\ 0 & 6 & 2 & 3 & 7 & 6 & 4 & 5 & 7 & 0 & 4 & 2 \\ 4 & 3 & 8 & 0 & 7 & 9 & 0 & 7 & 4 & 4 & 0 & 8 \\ 3 & 9 & 7 & 4 & 7 & 2 & 4 & 9 & 0 & 2 & 8 & 0 \end{bmatrix}$$

分析不同的类型的软件组件间交互频率矩阵对系统可靠性的影响，如图 6-6

所示。在图 6-5 中, $\rho=0.00002$, 采用的是第一种部署架构, SM 表示软件组件间的交互频率矩阵是对称矩阵时的系统可靠性情况, AM 表示软件组件间的交互频率矩阵是非对称矩阵时的系统可靠性情况。

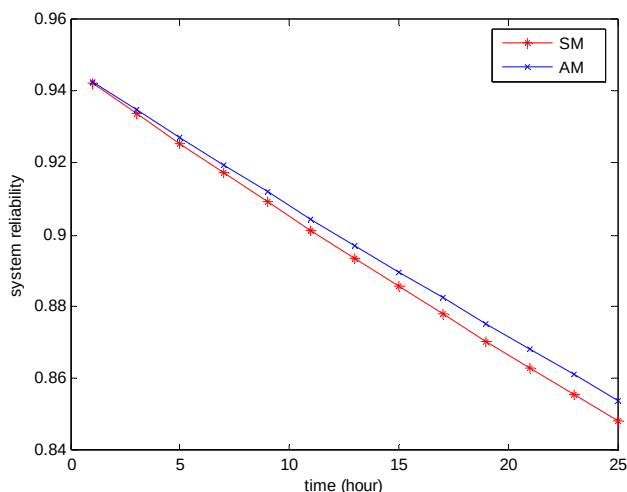


图 6-5 不同的交互频率矩阵对系统可靠性的影响

Fig.6-5 System reliability of different frequency matrix

从图 6-5 可以看出, 虽然两种类型的软件组件间的交互频率矩阵含有相同的元素, 但其对应的系统可靠性有较大差别。另外, 当软件组件的交互频率矩阵交换某些行或者列, 会转换成其它类型的矩阵, 也就是软件组件间的交互关系发生变化, 可能改变原有的软件架构风格。交互频率矩阵是非负的实数矩阵, 实对称矩阵是一类应用非常广泛的矩阵类型。实对称矩阵可分为正定矩阵、负定矩阵和其它情况。不同的软件架构风格与实矩阵的类型和性质间的关系值得进一步研究。

6.3 集成架构风格的可靠性分析

6.3.1 可靠性中常用的概率分布

在可靠性预测和评估方法中, 有许多常用的概率分布, 如正态分布、指数分布、威布尔分布等等。下面介绍几种概率分布的可靠度函数、失效率函数和相关应用。

(1) 正态分布

若随机变量 x 的概率密度见公式 (6-10)

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}, \quad -\infty < x < +\infty \quad (6-10)$$

其中, σ 和 μ 为两个参数, $\sigma > 0$ 为母体标准差, μ 为母体中心倾向 (集中趋势) 尺度, 它可以是均值、众数或中位数, 若 $-\infty < \mu < +\infty$, 则称 X 服从参数为 μ 和 σ^2 的正态分布, 并记作 $X \sim N(\mu, \sigma^2)$ 。其中正态分布的概率密度函数曲线又称为高斯曲线。

标准正态分布的分布函数见公式 (6-11), 其中 $z = (x - \mu) / \sigma$ 。

$$\Phi(z) = \int_{-\infty}^z \frac{1}{\sqrt{2\pi}} e^{-z^2/2} dz \quad (6-11)$$

正态分布的可靠度函数见公式 (6-12):

$$R(t) = \int_t^{+\infty} \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{t-\mu}{\sigma}\right)^2} dt \quad (6-12)$$

正态分布的失效率函数见公式 (6-13)

$$\lambda(t) = \frac{f(t)}{R(t)} = \frac{\Phi\left(\frac{t-\mu}{\sigma}\right)}{\sigma R(t)} \quad (6-13)$$

式中 t —— 随机变量 X 的取值;

μ —— X 的均值;

σ —— X 的方差。

在实际应用中有很多试验或观察数据可用正态分布来拟合, 作为它的近似分布。但正态分布是对称的, 其随机变量的取值是由 $-\infty$ 到 $+\infty$ 的整个数轴, 但许多试验或观察数据并非对称, 即只有正值而无负值, 这种情况在可靠性疲劳实验中更为普遍。因此, 虽然有的文献曾主张用正态分布来作为失效的理论分布, 但另有许多文献认为作为失效的理论分布是不恰当的, 还有人主张用截尾正态分布^{[162]24-27}。

(2) 指数分布

若 X 是一个非负的随机变量, 且有密度函数见公式 (6-14)

$$f(x) = \begin{cases} \lambda e^{-\lambda x}, & x \geq 0, \lambda > 0 \\ 0, & x < 0 \end{cases} \quad (6-14)$$

则称 X 服从参数为 λ 的指数分布, 记为 $e(\lambda)$, 式中 λ 为常数, 是指数分布的失效率。

指数分布的可靠度函数见公式 (6-15), 指数分布的失效率为 $\lambda(t) = \lambda$ 。

$$R(t) = e^{-\lambda t} \quad (6-15)$$

指数分布有个很重要的性质, 即它的无记忆性。它的含义就是, 如果某个

产品的寿命服从指数分布，那么在它经过一段时间 t_0 的工作以后如果仍然正常，则它仍然和新的一样，在 t_0 以后的剩余寿命仍然服从原来的指数分布。无记忆性有时又称为“无后效性”，即在发生前一个故障和发生下一个故障之间，没有任何联系，说明发生的是无后效事件，它们是随机事件，可用指数分布描述 [162]32-34。

(3) 威布尔分布

若 X 是一个非负的随机变量，且有密度函数见公式 (6-16)，

$$f(t) = \frac{m}{\eta} \left(\frac{t}{\eta}\right)^{m-1} e^{-\left(\frac{t}{\eta}\right)^m}, \quad m > 0, \eta > 0 \quad (6-16)$$

式中 m ——形状参数；

η ——尺度参数。

则称 X 服从 2 参数的威布尔分布 [42]12-13。

威布尔分布的可靠度函数见公式 (6-17)，威布尔分布的失效率见公式 (6-18)。

$$R(t) = e^{-\left(\frac{t}{\eta}\right)^m} \quad (6-17)$$

$$\lambda(t) = \frac{m}{\eta} \left(\frac{t}{\eta}\right)^{m-1} \quad (6-18)$$

根据威布尔分布的性质可知，当 $m < 1$ ， $m-1 < 0$ 时，属于早期失效；当 $m = 1$ ， $m-1 = 0$ 时，这时 $\lambda(t) = 1/\eta$ ，属于偶然失效。当 $m > 1$ ， $m-1 > 0$ 这时， $\lambda(t)$ 为升函数，属于耗损失效 [41]33-34。威布尔分布对各种类型的试验数据拟合能力强，例如指数分布只能适应于偶然失效期，而威布尔分布对于浴盆曲线的 3 个失效期都适用。又由于在各个领域中有许多现象近似地符合威布尔分布，因此，它的使用范围很广，是在可靠性工程中广泛应用使用的连续型分布。如果说指数分布常用来描述系统寿命的话，那么威布尔分布则常用来描述零件的寿命 [158]34-35。

6.3.2 集成架构风格的建模

这里通过连接器方法将多个架构风格集成在一个架构结构中。软件组件间通过不同架构风格的连接器来进行交互。设系统包括 n 个软件组件 $C = \{C_1, C_2, \dots, C_n\}$ ， m 个主机节点 $H = \{H_1, H_2, \dots, H_m\}$ ， k 个连接器 $L = \{L_1, L_2, \dots, L_k\}$ 和 s 种架构风格 $A = \{A_1, A_2, \dots, A_s\}$ ，如图 6-6 所示。

n 个软件组件被分为 m 个软件组件子集合， $CS_1, CS_2, \dots, CS_m$ ， CS_i 表示第 i 个软件组件子集合。 $CS_1 = \{C_{11}, C_{12}, \dots, C_{1i}\}$ ， $CS_2 = \{C_{21}, C_{22}, \dots, C_{2j}\}$ ， $\dots$ ， $CS_q = \{C_{q1}, C_{q2}, \dots, C_{qm}\}$ ， $\dots$ ， $CS_m = \{C_{m1}, C_{m2}, \dots, C_{mi}\}$ ，并且 $C = \bigcup_i CS_i$ 。 k 个连接器被分为 m 个连接器的子集合，

$LS_1, LS_2, \dots, LS_m$, LS_i 表示第 i 个连接器子集合。 $LS_1 = \{L_x, \dots, L_y\}$, $LS_2 = \{L_a, \dots, L_b\}$, $\dots, LS_q = \{L_e, \dots, L_f\}$, $\dots$, $LS_m = \{L_i, \dots, L_j\}$, 并且 $L = \bigcup_i LS_i$, 连接器的可靠性研究超出了本文范围, 假定连接器是完全可靠的。

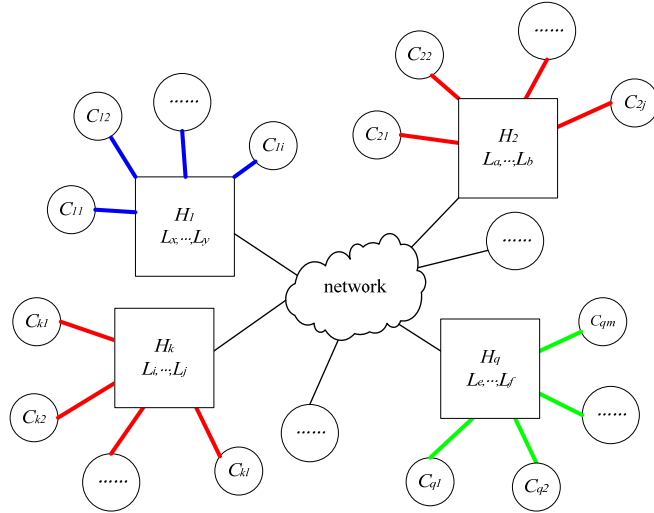


图 6-6 多个风格集成的部署架构图

Fig. 6-6 Deployment architecture graph of system with multiple styles

软件组件部署到主机节点上, 当软件组件执行时, 软件组件的失效原因主要有两种情况: 软件组件部署到的主机节点失效和软件组件内部引起的组件失效; 当软件组件未执行, 如果软件组件部署到的主机节点失效, 那么软件组件则不能提供服务或者不能请求服务, 此时软件组件被认为是失效。这时, 软件组件的失效率近似为主机节点的失效率。软件组件在不同时间单元下的可靠性

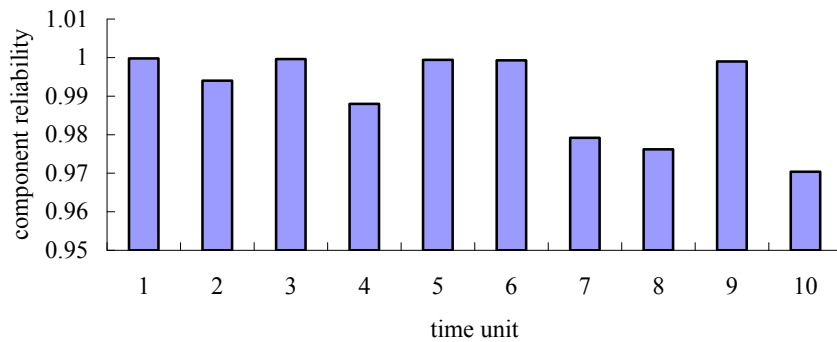


图 6-7 不同时间单元的单软件组件可靠性值

Fig. 6-7 Reliability of individual software component in different time units

的阶段性变化情况如图 6-7 所示。其中，在时间单元 3、5、6 和 9，软件组件处于未执行状态。随着系统运行时间的增加，主机节点的某些部件有损耗，其可靠性略微降低。在时间单元 1、2、4、7、8 和 10，软件组件处于执行状态。

集成架构风格系统既有同构的软件组件，也有异构的软件组件。同构软件组件是传统的完全由同一组织并且使用同一种技术来开发的软件组件。异构的软件组件指包括 COTS (commercial off-the-shelf) 组件、Services 组件等第三方开发的多种异构的软件组件，这些软件组件可能由不同的提供者并且使用不同的技术来开发完成的。因此，所有软件组件的失效行为服从同一分布的假设不合理。对于存在异构软件组件的系统，其软件组件失效行为用不同的函数分布来建模，如公式(6-19)所示。

$$RC_i = \begin{cases} f(t), & \text{组件执行期间} \\ \psi_i, & \text{组件未执行} \end{cases} \quad (6-19)$$

在公式 (6-19) 中， $f(t)$ 表示软件组件的可靠性为随时间变化的某一函数分布，如指数分布、威布尔分布等等。 ψ_i 表示软件组件 C_i 处于未执行状态时的可靠性值，近似为部署到的主机节点的可靠性值。

6.3.3 实验及结果分析

(1) 实验输入

系统包括 CS、PF 和 C2 这三种架构风格、5 个主机节点 $H=\{H_1, H_2, H_3, H_4, H_5\}$ 和 17 个软件组件 $C=\{C_1, C_2, \dots, C_{17}\}$ ，如图 6-8 所示。

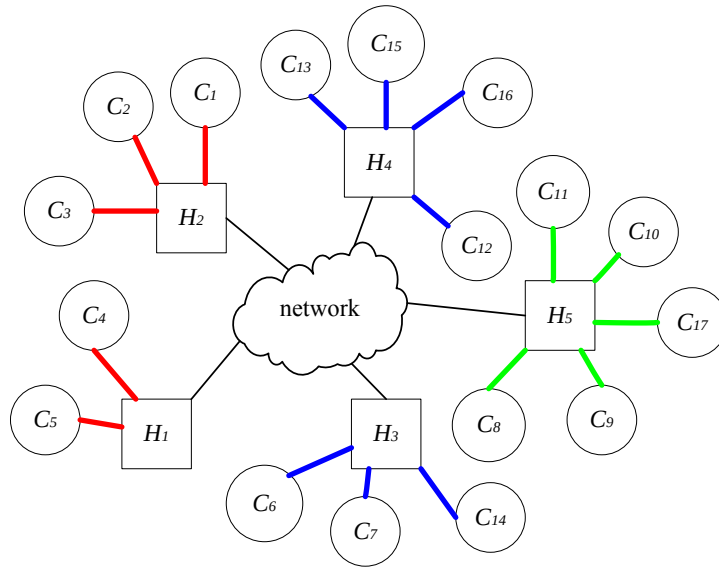


图 6-8 3 个架构风格集成的部署架构图

Fig. 6-8 Deployment architecture graph of three architectural styles

软件组件间通过不同软件架构风格的连接器进行交互。系统输入包括随机产生的软件组件在主机节点上的部署架构，不同架构风格的软件组件类型、软件组件在 10 个时间单元所处的状态和不同类型软件组件的可靠性建模。

随机产生 17 个软件组件在主机节点上的部署架构，软件组件 C_4 和 C_5 部署到主机节点 H_1 上，软件组件 C_1 、 C_2 和 C_3 部署到主机节点 H_2 上，软件组件 C_6 、 C_7 和 C_{14} 部署到主机节点 H_3 上，软件组件 C_{12} 、 C_{13} 、 C_{15} 和 C_{16} 部署到主机节点 H_4 上，软件组件 C_8 、 C_9 、 C_{10} 、 C_{11} 和 C_{17} 部署到主机节点 H_5 上。

17 个软件组件所属的架构风格的软件组件类型，如表 6-1 所示。CS 风格的软件组件有 C_2 、 C_4 、 C_1 、 C_3 、 C_5 ，PF 风格软件组件有 C_7 、 C_{13} 、 C_{15} 、 C_6 、 C_{12} 、 C_{14} 、 C_{16} ，C2 风格的软件组件有 C_8 、 C_9 、 C_{10} 、 C_{11} 、 C_{17} 。

表 6-1 软件组件类型

Table 6-1 Types of software components

software component type	software components
server	C_2, C_4
client	C_1, C_3, C_5
pipe	C_7, C_{13}, C_{15}
filter	$C_6, C_{12}, C_{14}, C_{16}$
C2	$C_8, C_9, C_{10}, C_{11}, C_{17}$

为了分析每个软件组件的在各时间单元的可靠性和系统可靠性，在满足一定约束条件的情况下，指定软件组件处于何种状态，例如，若 client 组件在第 3 个时间单元向 server 组件发送服务请求，若 server 组件未失效，则 server 组件在第 4 个单元将处于执行状态。设定系统中的 17 个软件组件的处于执行状态和未执行状态的时间单元如表 6-2 所示。软件组件处于执行状态，用“√”表示，软件组件处于未执行状态，用“×”表示。

表 6-2 不同时间单元的软件组件的执行状态

Table 6-2 Executed state of software components in different time units

	C_1	C_2	C_3	C_4	C_5	C_6	C_7	C_8	C_9	C_{10}	C_{11}	C_{12}	C_{13}	C_{14}	C_{15}	C_{16}	C_{17}
1	√	√	×	√	√	√	×	√	×	×	√	√	×	×	×	×	√
2	×	√	×	√	×	×	√	×	√	√	×	×	√	×	√	×	×

3	√	×	√	×	√	√	×	√	×	×	√	√	×	√	×	√	√
4	×	√	×	√	×	×	×	×	×	√	×	√	×	×	√	×	×
5	×	×	×	×	√	×	√	×	√	×	√	×	√	×	×	×	√
6	√	√	×	√	×	×	×	√	×	√	×	√	×	×	×	√	×
7	×	√	√	√	×	√	×	×	√	×	√	×	√	√	×	×	√
8	×	√	×	√	×	×	×	√	×	√	×	×	×	×	×	√	×
9	×	×	×	×	√	×	√	×	×	×	√	√	×	×	√	×	√
10	×	√	√	√	×	√	×	×	√	√	×	×	√	×	×	√	×

这里，将 PF 风格的软件组件可靠性用指数分布来建模，如公式（6-20）所示，如果软件组件 C_i 是 pipe 组件，则 $\lambda_i = 0.0004$ ；如果组件 C_i 是 filter 组件，则 $\lambda_i = 0.0002$ 。

$$RC_i = \begin{cases} e^{-\lambda_i \times t}, & \text{组件执行期间} \\ \psi_i, & \text{组件未执行} \end{cases} \quad (6-20)$$

CS 和 C2 架构风格的软件组件可靠性用威布尔分布来建模，见式（6-21）。 β_i 可由公式（6-22）求出。

$$RC_i = \begin{cases} e^{-(t/\eta)^{\beta_i}}, & \text{组件执行期间} \\ \psi_i, & \text{组件未执行} \end{cases} \quad (6-21)$$

$$\beta_i = \beta_0 - \tau_i \quad (6-22)$$

在公式（6-21）和（6-22）中，根据王的研究^[163]，对于包含 20 个软件组件的系统，如果用威布尔分布建模软件组件的可靠性，则 $\eta = 2917$ ， $\beta_0 = 1.29$ 。 τ_i 表示在同一架构风格的不同类型的软件组件对系统可靠性的影响，如果组件 C_i 是 C2 风格的软件组件，则 $\tau_i = 0.0031$ ；如果 C_i 是 server 组件，则 $\tau_i = 0.007225$ ；如果 C_i 是 client 组件，则 $\tau_i = 0.004725$ 。

（2）实验结果分析

随着系统运行时间增加，单个软件组件的可靠性变化情况，如图 6-9 所示。在图 6-9 中， C_7 和 C_{13} 为 pipe 组件，由于这两个软件组件在某些时间单元处于不同的状态，其可靠性差别比较明显。从这 6 个软件组件的可靠性变化情况，很容易看出软件组件在哪些时间单元处于执行状态和未执行状态。随着系统运行时间的增加，软件组件的可靠性总体趋势是在下降。当系统处于未执行状态时，软件组件的可靠性会暂时增加。

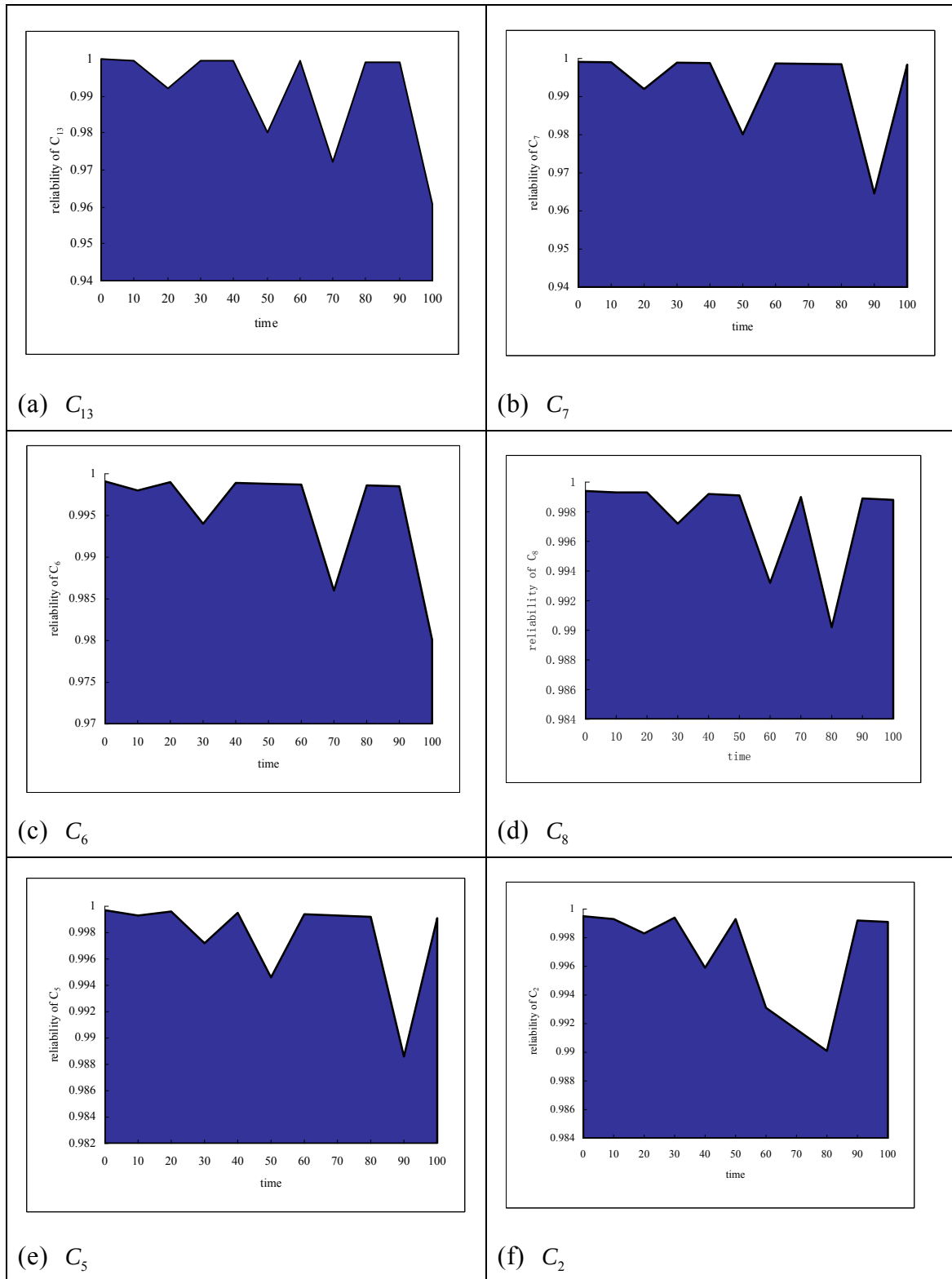


图 6-9 单个组件的可靠性变化图
Fig. 6-9 Reliability of individual software component

在特定的时间下，各个软件组件的可靠性的变化情况如图 6-10 所示。其中， $\text{time}=i$ 表示系统运行时间为 i 时各软件组件的可靠性。

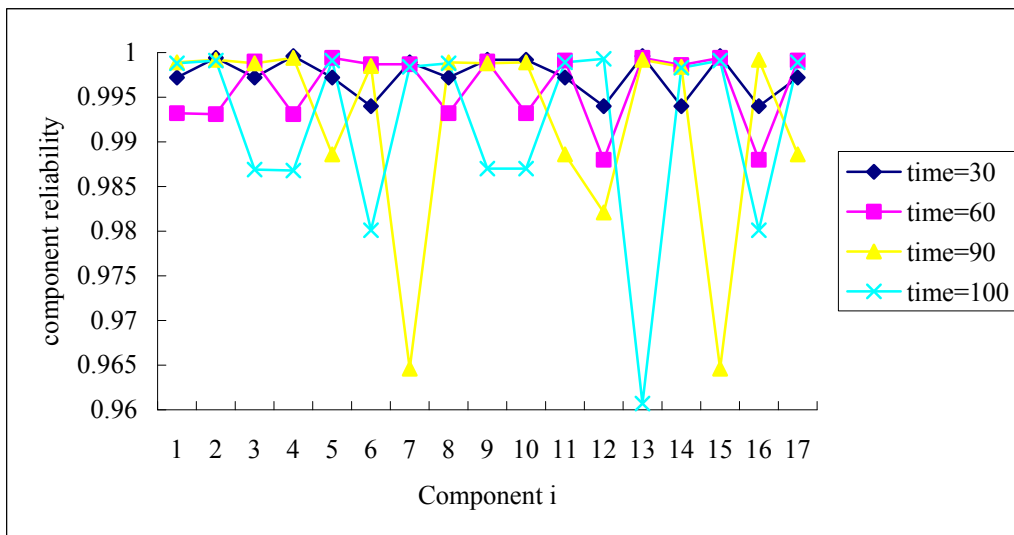


图 6-10 不同时间下各组件的可靠性

Fig. 6-10 Reliability of each software component of different time

从图 6-10 可以看出，随着系统运行时间的增加，软件组件可靠性变化不确定。当系统运行到时间 30 时，各软件组件的可靠性差别并不很明显，系统中没有较弱的点。当系统运行到时间 90 时，软件组件 C_7 和 C_{15} 的可靠性变得很低，成为系统中最弱的两个点。对于软件组件 C_7 ，在时间 60 和 90 的时候，组件的可靠性比较高， C_7 应处于未执行状态，此时组件的可靠性为主机节点 H_3 的可靠性值。但是由于在其它时间段内处于执行状态时情况比较多，导致 C_7 下降明显。对于软件组件 C_{16} ，系统执行到时间 30 时， C_{16} 处于未执行状态，而在时间 60、90 和 100 时， C_{16} 均处于执行状态，其可靠性逐渐下降。

系统可靠性变化情况如图 6-11 所示，随着系统运行时间的增加，系统可靠性总体趋势是下降的。在时间 40、60 和 80 时，系统可靠性有所提高，可能的原因是处于未执行状态的软件组件数目增多，可靠性高的软件组件数量增加，从而使系统可靠性增加。在时间段[60,80]内，系统可靠性波动比较大，尤其在时间 70，系统可靠性很低，可能的原因是处于执行状态的软件组件数目增多，或者某些软件组件已失效。因此，要重点分析系统可靠性波动比较大的时间段内的可靠降低的原因。在时间 70，可以考虑通过软件组件复制或者系统重部署来提高可靠性，从而使系统更稳定地运行。

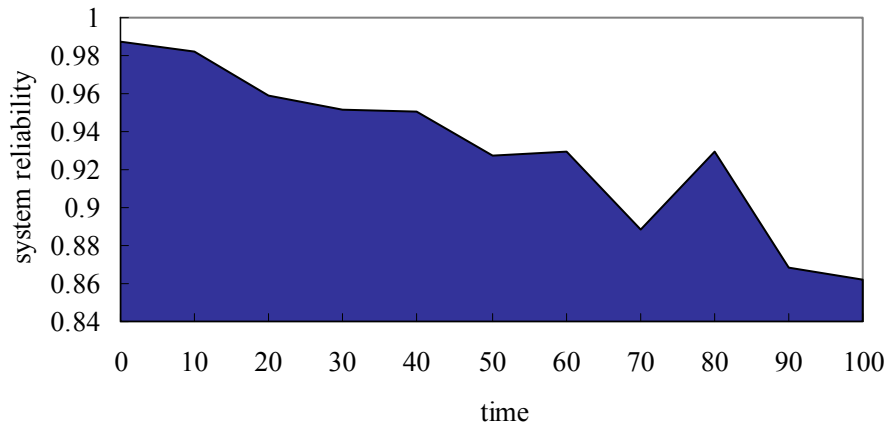


图 6-11 系统可靠性

Fig. 6-11 System reliability

6.4 本章小结

根据 Xie 模型假设，将部署阶段的软件组件的失效行为建模成 NHPP，并根据 ALRRA 方法的假设，将软件组件的失效率设置成与软件组件执行频率相关的函数。分析相关参数对可靠性的影响，给出软件组件复制和系统重部署的差别和适应范围。通过连接器的方式来构建集成架构风格，连接器的可靠性研究超出本文研究范围，设定连接器是完全可靠的，对不同架构风格的软件组件失效行为进行建模，分析集成架构风格的可靠性。实验包括 17 个软件组件、5 个主机节点和 3 种软件架构风格，分析在同一时间下的软件组件可靠性，不同时间下的软件组件的可靠性和系统可靠性变化情况。

结 论

随着互联网的出现，计算机软件所面临的环境发生了变化，由静态、封闭转为动态、开放。为了适应这种变化，软件系统从集中式逐渐演化为分布式。从 20 世纪 90 年代开始，基于组件的软件开发迅速发展并逐步成为主流技术，基于组件的分布式软件得到广泛应用，并渗透到社会生活和人类活动的各个领域，同时人们对基于组件的分布式软件可靠性、可用性等性质给予了更高的期望和要求。

随着分布式软件和网络快速发展，软件部署成为整个软件生命过程中的一个独立阶段。普适、移动和分布式的环境具有高度动态配置、未知的操作剖面 and 波动的执行环境，分布在这样环境下的软件的部署对系统性能有重要影响，并且部署在应急反应系统、导航、民用基础设施等领域的软件要执行关键任务，要满足严格的可靠性要求。为了使部署后的系统满足一定的可靠性要求，部署需要考虑多方面的信息，如待部署主机节点的可靠性、软件组件的可靠性、软件组件的互联性、主机节点的拓扑结构、硬件资源占用（如 CPU、内存）、软件架构风格、软件组件复制等等。

本文主要研究普适、移动和分布式环境的部署阶段的可靠性影响因素和提高系统可靠性的方法。内容包括：从理论上分析软件部署方法和经典的软件架构风格，提出将基于组件的软件部署和 CS、LCS 两种软件架构风格结合起来的方法，该方法分析部署架构和软件架构风格对系统失效的影响。将影响系统部署的因素分为确定性和不确定性的，引入模糊理论来描述这些因素，建立软件组件间模糊关系和主机节点间的模糊关系。根据软件组件间的模糊矩阵和主机节点间的模糊矩阵，设计模糊贪婪部署算法来重部署系统，并对模糊贪婪部署算法进行正确性和复杂度分析。根据软件组件间的交互频率和交互的重要性，引入软件组件间的相关性概念。在软件组件间相关度矩阵的基础上，设计 m 优先选择算法来重部署系统。并对 m 优先选择算法进行正确性和时间复杂性分析，给出包括 3 个主机节点和 6 个软件组件的执行过程。当系统有较多可用资源时，可通过软件组件复制来提高系统可靠性。根据有限的可用系统资源，设计了启发式的贪婪复制算法来选择将被复制的软件组件。如果软件组件有组件副本，那么即使这个软件组件失效，其副本仍可以提供服务，然而，失效的软件组件会将故障传播给与其直接交互的其它软件组件，对主机节点失效和软件组件失效的引起的故障传播进行分析。部署阶段的软件组件的失效行为建模成 NHPP 过程，分析相关参数和不同类型的软件组件间交互频率矩阵对可靠性

的影响,比较系统重部署和软件组件复制方法提高可靠性差别,给出两种方法的适用条件。分析软件组件处于执行和非执行状态下的失效率,用连接器的方式来建模集成架构风格,分析集成架构风格的可靠性。本文主要创新点如下:

(1)提出了一种将基于组件的软件部署和CS、LCS两种架构风格有机结合的方法。根据软件组件在不同主机节点上的部署情况对软件组件失效的影响,提出软件组件的调整前失效概率。根据同一软件架构风格中不同类型软件组件和不同软件架构风格对系统失效的影响,提出软件组件的调整后失效概率和影响因子,并给出影响因子的求解方法。

(2)引入模糊理论将影响部署的因素转化为软件组件间的模糊关系和主机节点间的模糊关系,提出软件组件间的模糊矩阵和主机节点间的模糊矩阵的概念,给出主机节点间模糊矩阵和软件组件间模糊矩阵的求解方法。根据软件组件间的模糊矩阵和主机节点间的模糊矩阵,设计了模糊贪婪部署算法来重部署系统。该算法利用贪婪算法的思想将 n 个软件组件分成 m 个软件组件子集合,然后,利用贪婪算法将 m 个软件组件子集合中的软件组件部署到 m 个主机节点上。对模糊贪婪部署算法进行正确性和时间复杂度分析。

(3)根据软件组件间的交互关系和交互的重要性,提出了软件组件间的相关性概念。在分析A*算法优缺点基础上,根据软件组件间的相关度矩阵,设计了 m 优先选择算法来重部署系统。对 m 优先选择算法进行正确性和时间复杂度分析,并给出3个主机节点和6个软件组件的详细执行过程。

(4)在分析目标优化函数 OF 和贪婪算法的基础上,设计了启发式的贪婪复制算法。根据有限的可用系统资源,该算法利用单位带宽的优化值增量 OB 和单位内存的优化值增量 OM 来选择将被复制的两个软件组件子集合,其中目标优化值高的为该算法选择将被复制的软件组件。对启发式的贪婪复制算法进行正确性和时间复杂度分析。此外,如果软件组件失效,其副本仍可继续提供服务,但是失效的软件组件会将故障传播给与其进行直接交互的其它软件组件。对相关主机节点失效和软件组件失效引起的故障传播进行分析。

(5)根据Xie模型的假设,提出将部署阶段的软件组件的失效行为建模成NHPP。分析复制软件组件和系统重部署对提高可靠性的差别,及两种方法适用条件。通过连接器方式来建模集成架构风格,分析集成架构风格的系统可靠性。

对本文的进一步工作可以在以下几方面展开:

1. 在理论研究的同时,将结合其实际应用进行分析,或者分析不同特定实际应用的案例,将它们抽象成具有不同特征的理论研究模型,更好地将理论与实际应用结合起来。
2. 设计出特定实际应用下的组件部署算法和组件复制算法

参考文献

- [1] 丁博. 软件自适应若干关键技术研究[D]. 长沙: 国防科学技术大学学位论文, 2010: 2-3.
- [2] 满君丰. 开放网络环境下软件行为检测与分析研究[D]. 长沙: 中南大学学位论文, 2010: 1-1.
- [3] 曹旻. 基于组件的分布式软件动态配置模型的研究[D]. 上海: 上海大学学位论文, 2005: 1-5.
- [4] Tassey G. The Economic Impacts of Inadequate Infrastructure for Software Testing[R]. Gaithersburg: National Institute of Standards and Technology, 2002: 1-13.
- [5] 陈松林, 鲁星, 马妙技. 高技术武器系统中软件的现状及应用分析[J]. 航空兵器, 2003, 21(3): 33-35.
- [6] 梅宏, 申峻嵘. 软件体系结构研究进展[J]. 软件学报, 2006, 17(6): 1257-1275.
- [7] Dearle A. Software Deployment, Past, Present and Future[C]. Future of Software Engineering(FOSE'07), Washington: IEEE Computer Society, 2007: 269-284.
- [8] Esfahani N, Malek S. A Modeling Language for Activity-oriented Composition of Service-Oriented Software Systems[C]. Proceedings of 12th International Conference on Model Driven Engineering Languages and Systems (MODELS'09), Denver, Colorado: Springer, 2009: 591-605.
- [9] Malek S, Krishnan H, Srinivasan J. Enhancing Middleware Support for Architecture-based Development through Compositional Weaving of Styles [J]. The Journal of System and Software, 2010, 83: 2513-2527.
- [10] Malek S. A User-centric Approach for Improving a Distributed Software System's Deployment Architecture [D]. USA: Dissertation of University of Southern California, 2007: 1-186
- [11] Bastarrica M C, Shvartsman A A, Demurjian S A. A Binary Integer Programming Model for Optimal Object Distribution[C]. International Conference on Principle of Distributed Systems, Amiens, France: CiteSeerx Press, 1998: 211-226.
- [12] Hunt G, Scott M L. The Coign Automation Distributed Partitioning System[C]. Proceedings of the Third Symposium on Operating System Design and Implementation, New Orleans: USENIX Association, 1999: 187-200.

-
- [13] Mikic-Rakic M, Malek S, Medvidovic N. Improving Availability in Large, Distributed, Component-based Systems via Redeployment[J]. Lecture Notes in Computer Science, 2005, 3798/2005: 83-98.
 - [14] Malek S, Seo C, Ravula S, et al. Reconceptualizing a Family of Heterogeneous Embedded Systems via Explicit Architectural Support[C]. Proceedings of the 29th International Conference on Software Engineering(ICSE'07), Minneapolis, MN: IEEE Computer Society, 2007:591-601.
 - [15] Malek S, Mikic-Rakic M, Medvidovic N. A Style-aware Architectural Middleware for Resource-Constrained, Distributed Systems[J]. IEEE Transactions on Software Engineering, 2005, 31(3):256-272.
 - [16] Medvidovic N, Malek S. Software Deployment Architecture and Quality-of-Service in Pervasive Environments[C]. International Workshop on Engineering of Software Services for Pervasive Environments: in Conjunction with the 6th ESEC/FSE Joint Meeting (ESSPE'07), Dubrovnik, Croatia: ACM Press, 2007: 47-51.
 - [17] Perry D E, Wolf A L. Foundations for the Study of Software Architecture [J]. ACM SIGSOFT Software Engineering Notes, 1992, 17(4):40-52.
 - [18] Fielding R T. 架构风格与基于网络的软件架构设计 [D]. 李锟, 廖志刚, 刘丹, 杨光, 译. Irvine: Dissertation of University of California, 2000: 10-74.
 - [19] Garlan D, Shaw M. Software Architecture: Perspectives on an Emerging Discipline[M]. Prentice Hall, 1996: 1-230.
 - [20] The Boeing Company-Defense and Space Group. STARS Conceptual Framework for Reuse Processes, Lockheed Martin Tactical Defense Systems. STARS Program Technical Report, 1994.
 - [21] 孙昌爱, 金茂忠, 刘超. 软件体系结构研究综述[J]. 软件学报, 2002, 13(7): 1228-1237.
 - [22] Standards Coordinating Committee of the Computer Society of the IEEE, IEEE Standard Glossary of Software Engineering Terminology(IEEE Std 610.12 -1990), IEEE Computer Society, 1990:18-63.
 - [23] 张友生, 李雄. 软件体系结构原理、方法与实践[M]. 北京: 清华大学出版社, 2009: 11-253.
 - [24] Garlan D, Shaw M. An Introduction to Software Architecture [J]. Advances in Software Engineering and Knowledge Engineering, 1993, 1:1-39.
 - [25] Garlan D, Allen R, Ockerbloom J. Architectural Mismatch: Why Reuse Is So Hard [J]. IEEE Software, 1995, 12(6): 17-26.
 - [26] Shaw M, Clements P. A Field Guide to Boxology: Preliminary Classification of Architectural Styles for Software Systems[C]. Proceedings of the 21st Annual

- International Computer Software and Applications Conference (COMPSAC), Washington: IEEE Computer Society, 1997: 6-13.
- [27] 杨芙清, 梅宏. 构件化软件设计与实现[M]. 北京: 清华大学出版社, 2008: 3-297.
- [28] Bass L, Buhman C, Comella D S, et al. Market Assessment of Component-based Software Engineering[M]. Pittsburgh: The Software Engineering Institute, Carnegie Mellon University, 2000:1-43.
- [29] Szyperski C, Gruntz D, Murer S. Component Software: Beyond Object-Oriented Programming [M], Boston: Addison-Wersley, 2004: 1-200.
- [30] Brown A W, Wallnau K. The Current State of CBSE [J]. IEEE Software, 1998, 15(5): 37-46.
- [31] 王志坚, 费玉奎, 姜渊清. 软件构件技术及其应用[M]. 北京: 科学出版社, 2005:1-245.
- [32] 韩松乔. 普适计算中面向移动的软件自适应性研究[D]. 上海: 上海交通大学学位论文, 2008:1-110.
- [33] 王卓. 模型驱动的软构件分类及检索方法研究[D]. 哈尔滨: 哈尔滨工程大学学位论文, 2005:1-110.
- [34] 张弛. 异构组件互操作技术研究[M]. 合肥: 中国科学技术大学出版社, 2008:17-18.
- [35] Software Engineering Standards Subcommittee of the Technical Committee on Software Engineering of the IEEE Computer Society. IEEE Standard Dictionary of Measures to Produce Reliable Software (IEEE Std 982.1-1988), IEEE Computer Society, 1988: 1-40.
- [36] Software Engineering Technical Subcommittee of the IEEE Computer Society. IEEE Guide for the Use of IEEE Standard Dictionary of Measures to Produce Reliable Software (IEEE Std 982.2-1988), IEEE Computer Society, 1988: 1-139.
- [37] 金碧辉. 系统可靠性工程[M]. 北京: 国防工业出版社. 2004: 1-2.
- [38] 蒋仁言, 左明健. 可靠性模型与应用[M]. 北京: 机械工业出版社, 1999: 6-7.
- [39] 陆文, 徐锋, 吕建. 一种开放环境下的软件可靠性评估方法[J]. 计算机学报, 2010, 33(3): 452-462.
- [40] 曹晋华, 程侃. 可靠性数学引论[M]. 北京: 高等教育出版社, 2006: 38-84.
- [41] 程五一, 王贵和, 吕建国. 系统可靠性理论[M]. 北京: 中国建筑工业出版社. 2010:10-151.
- [42] 金星, 洪延姬. 系统可靠性与可用性分析方法[M]. 北京: 国防工业出版社,

-
- 2007: 6-120.
- [43] Malek S, Roshandel R, Kilgore D, et al. Improving the Reliability of Mobile Software Systems through Continuous Analysis and Proactive Reconfiguration[C]. Proceedings of the 31st International Conference on Software Engineering, Vancouver, BC: IEEE Computer Society, 2009:275-278.
 - [44] L. Capra, Emmerich W, Mascolo C. CARISMA: Context-Aware Reflective Middleware System for Mobile Applications[J]. IEEE Transactions on Software Engineering, 2003, 29(10): 929-945.
 - [45] Gokhale S S. Architecture-based Software Reliability Analysis: Overview and Limitations [J], IEEE Transactions on Dependable and Secure Computing, 2007, 4(1): 32-40.
 - [46] Goseva-Popstojanova K, Trivedi K S. Architecture-based Approaches to Software Reliability Prediction [J]. International Journal of Computers & Mathematics with Applications, 2003, 46(7): 1023-1036.
 - [47] A. Immonen, Niemela E. Survey of Reliability and Availability Prediction Methods from the Viewpoint of Software Architecture[J]. Software and Systems Modeling, 2007, 7(1): 49-65.
 - [48] 殷昱煜. 面向服务的软件动态配置技术研究[D]. 浙江: 浙江大学学位论文, 2010: 5-10.
 - [49] Seigrist K. Reliability of Systems with Markov Transfer of Control [J]. IEEE Transactions on Software Engineering, 1988, 14(7): 1049-1053.
 - [50] Seigrist K. Reliability of Systems with Markov Transfer of Control II [J]. IEEE Transactions on Software Engineering, 1988, 14(10): 1478-1480.
 - [51] Laprie J C, Kanoun K. X-Ware Reliability and Availability Modeling [J]. IEEE Transactions on Software Engineering, 1992, 18(2): 130-147.
 - [52] Littlewood B. A Semi-Markov Model for Software Reliability with Failure Costs[C]. Proceedings of the 2nd International Conference on Software Engineering, San Francisco: IEEE Computer Society, 1976:281-300.
 - [53] Gokhale S, Wong W E, Trivedi K S, et al. An Analytic Approach to Architecture-based Software Performance and Reliability Prediction[J]. Performance Evaluation, 2004, 58(4): 391-412.
 - [54] Tomek L A, Muppala J K, Trivedi K S. Modeling Correlation in Software Recovery Blocks [J]. IEEE Transactions on Software Engineering, 1993, 19(11): 1071-1086.
 - [55] Gokhale S, Trivedi K S. Reliability Prediction and Sensitivity Analysis Based on Software Architecture[C]. Proceedings of the 13th International Symposium on Software Reliability Engineering. Annapolis: IEEE Computer Society,

2002:64-75.

- [56] Gokhale S. Quantifying the Variance in Application Reliability[C]. Proceedings of Pacific Rim International Symposium on Dependable Computing, Los Alamitos: IEEE Computer Society, 2004:113-121.
- [57] Yacoub S, Cukic B, Ammar H. A Scenario-Based Analysis for Component-Based Software [J]. IEEE Transactions on Reliability, 2004, 53(4):465-480.
- [58] 毛晓光, 邓勇进. 基于构件软件的可靠性通用模型[J]. 软件学报, 2004, 15(1):27-32.
- [59] Gokhale S, Lyu M R, Trivedi K S. Reliability Simulation of Component-based Software Systems[C]. Proceedings of the 9th International Symposium on Software Reliability Engineering (ISSRE), Paderborn, Germany: IEEE Computer Society, 1998: 192-201.
- [60] Huang R, Lyu M R, Kanoun K. Simulation Techniques for Component-Based Software Reliability Modeling with Project Application[C]. Proceeding of International Symposium on Information Systems and Engineering (ISE 2001), Las Vegas, Nevada, USA: CSREA Press, 2001: 283-289.
- [61] Xie M, Wohlin C. An Additive Reliability Model for the Analysis of Modular Software Failure Data[C]. Proceedings of the Sixth International Symposium on Software Reliability Engineering (ISSRE'95), Toulouse: IEEE Computer Society, 1995: 188-194.
- [62] Everett W. Software Component Reliability Analysis[C]. Proceedings of the Symposium on Application-specific Systems and Software Engineering and Technology, Richardson: IEEE Computer Society, 1999:204-211.
- [63] Laprie J C, Kanoun K. Software Reliability and System Reliability[M]. Handbook of Software Reliability Engineering, New York: McGraw-Hill, 1996: 27-69.
- [64] Wang W L, Hemminger T L, Tang M H. A Moving Average Modeling Approach for Computing Component-Based Software Reliability Growth Trends[J]. INFOCOMP Journal of Computer Science, 2006, 5(3):9-18.
- [65] Wang W L, Hemminger T L, Tang M H. A Moving Average Non-Homogeneous Poisson Process Reliability Growth Model to Account for Software with Repair and System Structures[J]. IEEE Transactions on Reliability, 2007, 56(3): 411-421.
- [66] Pant H, Jeske D R. Software Reliability Predictions for Distributed Software [C]. Paderborn, Germany: IEEE Computer Society, 1998:11-21.
- [67] Luo Y, Ben K. Scenario-based Early Reliability Model for Distributed Software[C]. 2010 Fifth International Conference on Frontier of Computer

-
- Science and Technology, Changchun, Jilin Province: IEEE Computer Society, 2010: 201-207.
- [68] Kumar A, Agrawal D P. A Generalized Algorithm for Evaluating Distributed Programming Reliability[J]. IEEE Transactions on Reliability, 1993, 42(3): 416-426.
- [69] Wang W, Wu Y, Chen M. An Architecture-based Software Reliability Model[C]. Proceedings of Pacific Rim International Symposium on Dependable Computing, Hong Kong: IEEE Computer Society, 1999:143-150.
- [70] Kramer J, Magee J. Dynamic Configuration for Distributed Systems[J]. IEEE Transactions on Software Engineering, 1985, SE-11(4):424-436.
- [71] Wermrlinger M. Towards a Chemical Model for Software Architecture Reconfiguration [J]. IEE Proceedings on Software, 1998, 145(5):130-134.
- [72] Cao J, Cao M, Chan A, et al. Architectural Level Support for Dynamic Reconfiguration and Fault Tolerance in Component-based Distributed Software[C]. Proceedings of the 9th International Conference on Parallel and Distributed Systems(ICPADS'02), Taiwan, ROC: IEEE Computer Society, 2002: 251-256.
- [73] Drummond J. Establishing a Real-time Distributed Benchmark[C]. Proceedings of the 4th International Workshop on Parallel and Distributed Real-time System, Honolulu, HI, USA: IEEE Computer Society, 1996:198-201.
- [74] Kamenoff N I. One Approach for Generalization of Real-time Distributed Systems Benchmarking[C]. Proceedings of the 4th International Workshop on Parallel and Distributed Real-time Systems, Honolulu, Hawaii: IEEE Computer Society, 1996:204-207.
- [75] Sanders W H, Meyer J F. A Unified Approach for Specifying Measures of Performance, Dependability, and Performability[J]. Dependable Computing and Fault-tolerant Systems, 1991, 4:215-237.
- [76] Weideman N H, Kamenoff N I. Hartstone Uniprocessor Benchmark: Definitions and Experiments for Real-time Systems[J]. The Journal of Real-time System, 1992, 4:282-353.
- [77] Jin H, Xie X, Li Y, et al. Athena: A Real-time Performance Evaluation for Distributed Software with Reliability-constrained[C]. Proceedings of the 18th International Parallel and Distributed Processing Symposium (IPDPS'04), Santa Fe, New Mexico: IEEE Computer Society, 2004:165-179.
- [78] 韩宗芬, 李运发, 谢夏, 金海. 一种具有时间约束的分布式软件可靠性评估方法[J]. 计算机研究与发展, 2004, 41(2): 311-316.

-
- [79] Andersson J. A Deployment System for Pervasive Computing[C]. Proceedings of the International Conference on Software Maintenance (ICSM'00), San Jose, California, USA: IEEE Computer Society, 2000: 262-270.
- [80] Vo C C, Torabi T. A Framework for Over the Air Provider-initiated Software Deployment on Mobile Devices[C]. Proceedings of 19th Australian Conference on Software Engineering, Perth, Australia: IEEE Computer Society, 2008: 633-638.
- [81] Hughes P H, Lovstad J S. A Generic Model for Quantifiable Software Deployment[C]. International Conference on Software Engineering Advances (ICSEA 2007), Cap Esterel, France: IEEE Computer Society, 2007: 22-31.
- [82] Mikic-Rakic M, Malek S, Beckman N, et al. A Tailorable Environment for Assessing the Quality of Deployment Architectures in Highly Distributed Setting [C]. Proceedings of the 2nd International Working Conference on Component Deployment (CD2004), Edinburgh, UK: Citeseerx Press, 2004: 1-17.
- [83] Popescu D. Framework for Replica Selection in Fault-tolerant Distributed Systems[R]. Technical Report USC-CSSE-2007-702, 2007, http://csse.usc.edu/csse/TECHRPTS/2007/2007_main_exp.html.
- [84] Malek S. A User-centric Framework for Improving a Distributed Software System's Deployment Architecture[C]. Proceedings of the 14th ACM SIGSOFT Symposium on Foundation of Software Engineering (FSE'06), Portland, Oregon: Citeseer Press, 2006:31-40.
- [85] 张奇峰.大型应用程序的自部署机制研究[D].武汉:华中科技大学学位论文, 2010:1-41.
- [86] Boone B, Verdickt t, Dhoedt B, et al. Design Time Deployment Optimization for Component Based Systems [C]. Proceedings of the 25th Conference on IASTED International Multi-Conference, Anaheim, USA: ACTA Press, 2007: 242-248.
- [87] Goseva-Popstojanova K, Kamavaram S. Software Reliability Estimation under Uncertainty: Generalization of the Method of Moments [C]. Proceedings of the 8th IEEE International Symposium on High Assurance Systems Engineering (HASE), Tampa, Florida: IEEE Computer Society, 2004: 209-218.
- [88] Lan L, Huang G, Ma L Y, et al. Architecture Based Deployment of Large-scale Component Based Systems: The Tool and Principles [J]. LNCS 3489, 2005, 3489/2005: 123-138.
- [89] Seo C, Malek S, Medvidovic N. An Energy Consumption Framework for Distributed Java-based Systems[C]. Proceedings of the twenty-second

-
- IEEE/ACM Conference on Automated Software Engineering (ASE'07), Tokyo, Japan: ACM Press, 2007:421-424.
- [90] Sun Microsystems, Inc. Java Web Start Technology [EB/OL]. <http://java.sun.com/products/javawebstart/>.
- [91] Taconet C, Putrycz E, Bernard G. Context Aware Deployment for Mobile Users[C]. Proceedings of the 27th Annual International Computer Software and Applications Conference(COMPSAC), Texas, USA: IEEE Computer Society, 2003:74-81.
- [92] Ayed D, Taconet C, Bernard G, et al. An Adaptation Methodology for the Deployment of Mobile Component-based Application[C]. In ACS/IEEE International Conference on Pervasive Services, Lyon: IEEE Computer Society, 2006:193-202.
- [93] Fjellheim T. Over-the-air Deployment of Applications in Multi-platform Environments[C]. Proceedings of the Australian Software Engineering Conference(ASWEC'06), Sydney, Australia: IEEE Computer Society, 2006: 159-170.
- [94] Belaramani N M, Wang C L, Lau F C M. Dynamic Component Composition for Functionally Adaptation in Pervasive Environments[C]. Proceedings of the Ninth IEEE Workshop on Future Trends of Distributed Computing Systems(FTDCS'03), San Juan, Puerto Rico, USA : IEEE Computer Society, 2003: 226-232.
- [95] Kwan M. A Distributed Proxy System for Functionality Adaptation in Pervasive Computing Environment[D]. Hong kong: The University of Hong kong, 2002: 3-50.
- [96] 郑迪. 基于上下文感知服务的构件化中间件关键技术研究[D]. 长沙: 国防科学技术大学学位论文, 2008:1-120.
- [97] Ali S, Torabi T, Ali H. A Case for Business Process Deployment for Location Aware Application[J]. International Journal of Computer Science and Network Security, 2006, 6(8):118-127.
- [98] Weinsberg Y, Ben-Shaaul I. A Programming Model and System Support for Disconnected-aware Applications on Resource-constrained Devices[C]. Proceedings of the 24th International Conference on Software Engineering (ICSE'02), Orlando, FL, USA: ACM Press, 2002:374-384.
- [99] 李冰鹏. 基于规范化描述的软件智能部署的研究与实现[D].长沙:国防科学技术大学学位论文, 2008:1-65.
- [100] Sobr I, Tuma P. SOFAnet: Middleware for Software Distribution over Internet[C].Proceedings of the 2005 Symposium on Applications and the

- Internet (SAINT'05), Trento, Italy: IEEE Computer Society, 2005:48-53.
- [101]Mehta M, Drew N, Vardoulas M, et al. Reconfigurable Terminals: an Overview of Architectural Solutions[J]. IEEE Communication Magazine, 2001,39(8): 82-89.
- [102]Majidi E, Alemi M, Rashidi H. Software Architecture: a Survey and Classification[C]. Proceedings of the 2nd International Conference On Communication Software and Networks, Singapore: IEEE Computer Society, 2010: 454-460.
- [103]Berners-Lee T. WWW: Past, Present and Future [J]. IEEE Computer, 1993, 29(10): 69-77.
- [104]Chin R S, Chanson S T. Distributed Object-based Programming Systems [J]. ACM Computing Surveys, 1991, 23(1): 91-124.
- [105]Fuggetta A, Picco G P, Vigna G. Understanding Code Mobility [J]. IEEE Transactions on Software Engineering, 1998, 24(5): 342-361.
- [106]聂南. 基于 XML 的组件接口验证与测试[D]. 武汉: 华中科技大学学位论文, 2008:1-60.
- [107]Zadeh L A. Fuzzy Sets[J]. Information and Control,1965, 8:338-353.
- [108]李洪亮. 关于模糊数变量模糊数值映射的几点讨论[D]. 哈尔滨: 哈尔滨工业大学学位论文, 2009:1-2.
- [109]任雪昆. 非可加测度与模糊 Riemann-Stieltjes 积分[D]. 哈尔滨: 哈尔滨工业大学学位论文, 2008:1-2
- [110]梁保松, 曹殿立. 模糊数学及其应用[M]. 北京: 科学出版社, 2007:1-93.
- [111]李柏年. 模糊数学及其应用[M].合肥:合肥工业大学出版社, 2007:49-50.
- [112]李鸿吉. 模糊数学基础及实用算法[M]. 北京: 科学出版社, 2005:83-88.
- [113] Thomas C, Charles L, Ronald R, et al. Introduction to Algorithms [M]. Beijing: Higher Education Press, 2001: 370-378.
- [114]范周田.模糊矩阵理论与应用[M].北京: 科学技术出版社, 2006:9-10.
- [115]Mikic-Rakic M, Medvidovic N. Support for Disconnected Operation via Architectural Self-reconfiguration[C]. Proceedings of the First International Conference on Autonomic Computing (ICAC), New York, USA: IEEE Computer Society, 2004:114-121.
- [116]Huang C Y, Lin C T. Software Reliability Analysis by Considering Fault Dependency and Debugging Time Lag[J]. IEEE Transactions on Reliability, 2006, 55(3):436-449.
- [117]Chen X. Dependence Management for Dynamic Reconfiguration of Component-based Distributed Systems[C]. Proceedings of 17th IEEE

-
- International Conference on Automated Software Engineering(ASE'02), Edinburgh, Scotland, UK: IEEE Computer Society, 2002: 279-284.
- [118]Judith A L, Wolf A L. Architecture-level Dependence Analysis for Software Systems[J]. International Journal of Software Engineering and Knowledge Engineering, 2001, 11(4): 431-451.
- [119]Binkley A, Schach S. Validation of the Coupling Dependency Metric as a Predictor of Run-time Failures and Maintenance Measures[C]. Proceedings of the 22nd International Conference on Software Engineering, Kyoto, Japan: IEEE Computer Society, 1998:452-455.
- [120]李军国.基于软件体系结构的容错机制动态配置技术研究[D].北京: 北京大学学位论文, 2009:31-32.
- [121]Kutzner K, Fuhrmann T. Measuring Large Overlay Networks-the Overnet Example[J]. Informatik Aktuell, 2005, Teil IV: 193-204.
- [122]Plissonneau L, Costeux J L, Brown P. Analysis of Peer-to-Peer Traffic on ADSL[J]. Lecture Notes in Computer Science, 2005, 3431/2005: 69-82.
- [123]中国互联网络信息中心.2009.中国互联网络发展状况统计报告[R].北京: 中国互联网络信息中心.[2009-01-12].
[HTTP://www.cnnic.net.cn/uploadfiles/doc/2009/1/13/92209.doc](http://www.cnnic.net.cn/uploadfiles/doc/2009/1/13/92209.doc)
- [124]Eidenbenz S, Widmayer P. An Approximation Algorithm for Minimum Convex Cover with Logarithmic Performance Guarantee[J]. SIAM Journal on Computing, 2003, 32(3): 654-670.
- [125]Guerraoui G, Schiper A. Fault-tolerance by Replication in Distributed System[C]. Proceedings of the 1996 Ada-Europe International Conference on Reliable Software Technologies, Montreux, Switzerland: Springer Verlag, 1996: 38-57.
- [126]Wiesmann M, Pedone F, Schiper A, et al. Understanding Replication in databases and Distributed Systems[C]. Proceedings of the 20th International Conference on Distributed Computing Systems, Taipei, Taiwan: IEEE Computer Society, 2000:464-474.
- [127]Coulouris G, Dollimore J, Kindberg T. Distributed Systems: Concepts and Design[M]. Third Edition. New York: Addison Wesley, 2000.
- [128]Object Management Group. The Common Object Request Broker: Architecture and Specification, Version 3.03, March 2004.
- [129]Saito Y, Shapiro M. Optimistic Replication[J]. ACM Computing Survey, 2005, 37(1):42-48.
- [130]Schneider F. Implementing Fault-tolerance Services Using the State Machine Approach[J]. ACM Computing Survey, 1990, 22(4):299-319.

- [131]Defago X, Schiper M, Urban P. Total Order Broadcast and Multicast Protocols: Taxonomy and Survey[J]. ACM Computing Survey, 2004, 6(4):372-421.
- [132]Budhiraja N, Marauillo K, Schneider F B, et al. Optimal Primary-backup Protocols[C]. Proceedings of sixth International Workshop on Distributed Algorithm, Haifa, Israel: Springer-verlag, 1992:362-378.
- [133]Beedubail G, Karmarkar A, Gurijala H, et al. Fault Tolerant Objects in Distributed System Using Hot Replication[C]. Proceedings of the 1996 IEEE Fifteenth Annual International Phoenix Conference on Computers and Communications, Scottsdale, AZ, USA: IEEE Computer Society, 1996:89-95.
- [134]李磊.分布式系统中容错机制性能优化技术研究[D].长沙: 国防科学技术大学学位论文, 2007:12-15.
- [135]Chen I R, Bastani F B. Warm Standby in Hierarchically Structured Process-control Programs[J]. IEEE Transactions on Software Engineering, 1994, 20(8):658-663.
- [136]Powell D, Cheeque M, Drackley D. Fault-tolerance in Delta-4[J]. ACM Operating Systems Review, SIGOPS, 1991, 25(2):122-125.
- [137]Defago X, Schiper A, Sergent N. Semi-passive Replication[C]. Proceedings of the 17th IEEE Symposium on Reliable Distributed Systems, West Lafayette, USA: IEEE Computer Society, 1998:43-50.
- [138]Dumitras T, Srivastava D, Narasimhan P. Architecting and Implementing Versatile Dependability[J]. Architecture Dependable Systems III, 2005, 3549: 212-231.
- [139]S Malek, Medvidovic N, Seo Chiyong, et al. A User Centric Approach for Improving a Distributed Software System's Deployment[R]. USC-CSE-2006-602, 2006.
http://csse.usc.edu/csse/TECHRPTS/2006/2006_main_exp.html
- [140]Krka I, Cheung L, Edwards G, et al. Architecture-based Software Reliability Estimation: Problem Space, Challenges, and Strategies[C]. Proceedings of the DSN Workshop on Architecting Dependable Systems, Anchorage, AK: IEEE Computer Society, 2008:45-53.
- [141]Hiller M, Suri N. EPIC: Profiling the Propagation and Effect of Data Errors in Software[J]. IEEE Transactions on Computers, 2004, 53(5):1-19.
- [142]Corteilessa V, Grassi V. A Modeling Approach to Analyze the Impact of Error Propagation on Reliability of Component-based Systems [J]. Lecture Notes in Computer Science, 2007, 4608/2007: 140-156.
- [143]Yacoub S M, Cukic B, Ammar H H. Scenario-based Reliability Analysis of Component-based Software[C]. Proceedings of the 10th International

-
- Symposium on Software Reliability Engineering, Boca Raton: IEEE Computer Society, 1999: 465-480.
- [144] Brosch F, Zimmerova B. Design-time Reliability Prediction for Software Systems[C]. Proceedings of 3rd CSMR Workshop on Software Quality and Maintenance, Honolulu, Hawaii: Microsoft Academic, 2009:17-23.
- [145] Goseve-Popstojanova K, Hassan A, Guedem A, et al. Architecture Level Risk Analysis Using UML[J]. IEEE Transactions on Software Engineering, 2003, 29(10): 946-960.
- [146] Freiberger W F. Statistical Computer Performance Evaluation[M]. New York: Academic Press, 1972:465-485.
- [147] Goel A L, Okumoto K. Time-Dependent Error-Detection Rate Model for Software and other Performance Measures[J]. IEEE Transactions on Reliability, 1979, 28(3): 206-211.
- [148] Musa J D, Iannino A, Okumoto K. Software Reliability: Measurement, Prediction, Application[M]. New York: McGraw-Hill, 1997:1-541.
- [149] Musa J D, Okumoto K. A Logarithmic Poisson Execution Time Model for Software Reliability Measurement[C]. Proceedings of 7th International Conference on Software Engineering, Orlando: ACM SIGSOFT and IEEE TCSE, 1984:230-238.
- [150] Yamada S, Ohba M, Osaki S. S-Shaped Software Reliability Growth Modeling for Software Error Detection[J]. IEEE Transactions on Reliability, 1983, 32(5):475-484.
- [151] Yamada S, Ohtera H, Narihisa H. Software Reliability Growth Models with Testing Effort[J]. IEEE Transactions on Reliability, 1986, 35(1):19-23.
- [152] Zachariah B, Rattihalli R N. Failure Size Proportional Models and an Analysis of Failure Detection Abilities of Software Testing Strategies[J]. IEEE Transactions on Reliability, 2007, 56(2): 246-253.
- [153] Huang C Y, Lin C T. Analysis of Software Reliability Modeling Considering Testing Compression Factor and Failure-to-Fault Relationship[J]. IEEE Transactions on Computers, 2010, 59(2): 283-288.
- [154] Hwang S, Pham H. Quasi-Renewal Time-Delay Fault-Removal Consideration in Software Reliability Modeling[J]. IEEE Transactions on Systems, Man and Cybernetics, Part A: Systems and Humans, 2009, 39(1): 200-209.
- [155] Huang C Y, Kuo S Y, Lyu M R. An Assessment of Testing-Effort Dependent Software Reliability Growth Models[J]. IEEE Transactions on Reliability, 2007, 56(2): 198-211.
- [156] Huang C Y, Lyu M R, Kuo S Y. A Unified Scheme of Some Non-

- Homogeneous Poisson Process Models for Software Reliability Estimation[J]. IEEE Transactions on Software Engineering, 2003, 29(3): 261–269.
- [157]刘嘉焜, 王公恕. 应用随机过程[M]. 北京: 科学出版社, 2004:56-64.
- [158]宋保维.系统可靠性设计与分析[M]. 西安:西北工业大学出版社, 2008:24-27.
- [159]刘霞, 李明树, 王青, 周津慧. 软件体系结构分析与评价方法评述[J]. 计算机研究与发展, 2005, 42(7):1247-1254.
- [160]Yacoub S M, Ammar H H. A Methodology for Architecture-level Reliability Risk Analysis [J]. IEEE Transactions on Software Engineering, 2002, 28 (6):529-547.
- [161]Roshandel R, Malek S. Refining Reliability Estimation of Mobile Software Systems[C]. Proceedings of the 1st International Workshop on Software Architectures and Mobility, Leipzig, Germany: ACM Press, 2008: 39-41.
- [162]Cheung R C. A User-Oriented Software Reliability Model[J]. IEEE Transactions on Software Engineering, 1980, 6(2):118-125.
- [163]Wang P, JIN T. Complex Systems Reliability Estimation Considering Uncertain Component Lifetime Distributions [C]. Proceeding of the 8th International conference on Reliability, Maintainability and Safety (ICRMS), San Diego, CA, USA: IEEE Computer Society, 2009: 20-24.

攻读博士学位期间发表的论文及其它成果

（一）发表的学术论文

- [1] Xihong Su, Zhibo Wu, Hongwei Liu, Xiaozong Yang, Decheng Zuo. Reliability Analysis of SA Based Software Deployment with Consideration of System Deployment[J]. Intelligent Automation and Soft Computing, 2011,17(6): 749-758.(SCI 收录, IDS 号为 815JP, IF=0.349)
- [2] Xihong Su, Hongwei Liu, Zhibo Wu, Xiaozong Yang, Decheng Zuo. SA Based Software Deployment Reliability Estimation Considering Component Reliability of Exponential Distribution [J]. Journal of Software, 2011, 6(6): 1140-1145.(EI 收录号: 20113414255723)
- [3] Su Xihong, Liu Hongwei, Wu Zhibo, Yang Xiaozong, Zuo Decheng. SA Based Software Deployment Reliability Estimation Considering Component Dependence [J]. Journal of Electronics (China)(电子与信息学报), 2011, 28(1):118-125.
- [4] Su Xihong, Wu Zhibo, Liu Hongwei, Yang Xiaozong, Zuo Decheng. SA Based Software Deployment Reliability Estimation Considering Architectural Style [J]. High Technology Letters(高技术通讯), 2011(4).(录用待发表, EI 源)
- [5] Xihong Su, Hongwei Liu, Zhibo Wu, Xiaozong Yang, Decheng Zuo. A Framework of SA Based Software Deployment Reliability Estimation [J]. Journal of Computational Information Systems, 2010,6 (13): 4409-4416.(EI 收录号: 20110313601288)
- [6] Xihong Su, Hongwei Liu, Zhibo Wu, Xiaozong Yang, Decheng Zuo. SA Based Software Deployment Reliability Estimation Considering Component Reliability[J]. ICIC Express Letters. 2011, 5(7):2287-2291.(EI 收录号: 20112714120092)
- [7] Xihong Su, Zhibo Wu, Hongwei Liu, Decheng Zuo, Xiaozong Yang. A Comprehensive Approach of SA Based Software Deployment Reliability Estimation in Neural Networks[C]. 2010 First International Conference on Pervasive Computing, Signal Processing and Applications, Harbin, 2010, 49-53. (EI 收录号: 20105213520998)
- [8] Xihong Su, Hongwei Liu, Zhibo Wu, Decheng Zuo, Xiaozong Yang. SA

Based Software Deployment Reliability Estimation: Problem Space, Challenges and Strategies[C].2010 International Conference on Educational and Information Technology,Chongqing, 2010,V2-130-V2-135.(EI 收录号: 20105013482595)

- [9] Xihong Su, Zhibo Wu, Hongwei Liu, Xiaozong Yang, Decheng Zuo. SA Based Software Deployment Reliability Estimation Considering Component and Host Node Dependences[C]. Proceedings of the fourth Conference on Dependable Computing, Yichang, 2010, 405-408.
- [10] 苏喜红, 刘宏伟, 吴智博, 杨孝宗, 左德承. 考虑组件复制的 SA 软件部署可靠性研究[J].哈尔滨工业大学学报.(录用待发表, EI 源)
- [11] 苏喜红, 刘宏伟, 吴智博, 杨孝宗, 左德承. 集成架构风格的 SA 软件部署可靠性分析[J].哈尔滨工业大学学报.(录用待发表, EI 源)
- [12] 苏喜红, 吴智博, 杨孝宗, 刘宏伟. 基于多版本较准的软件费用模型研究[J]. 计算机科学,2009, 36(5): 129-132.
- [13] 苏喜红, 吴智博, 杨孝宗, 刘宏伟. 软件可靠性模型综述[J].计算机科学, 2008, 35(10): 9-12.

已投稿:

- [14] Xihong Su, Hongwei Liu, Zhibo Wu, Xiaozong Yang, Decheng Zuo. Reliability Analysis of SA Based Software Deployment Considering Component Reliability of Weibull Distribution [J]. International Journal of Computer Systems Science & Engineering. (SCI 源)
- [15] Xihong Su, Hongwei Liu, Zhibo Wu, Xiaozong Yang, Decheng Zuo. Replica Selection in Fault-tolerant SA Based Software Deployment Model [J]. Journal of Networks.(EI 源)

(二) 参与的科研项目

- [1] 刘宏伟. 高端容错计算机总体技术研究, 国家 863 重大专项. 课题编号: 2008AA01A201.
- [2] 刘宏伟. 考虑测试环境与运行环境差别的软件可靠性增长模型研究, 国家自然科学基金项目. 课题编号: 60503015.
- [3] “十二五” 国防预研项目.

哈尔滨工业大学学位论文原创性声明及使用授权说明

学位论文原创性声明

本人郑重声明：此处所提交的学位论文《分布式系统软件部署策略及其可靠性研究》，是本人在导师指导下，在哈尔滨工业大学攻读学位期间独立进行研究工作所取得的成果。据本人所知，论文中除已注明部分外不包含他人已发表或撰写过的研究成果。对本文的研究工作做出重要贡献的个人和集体，均已在文中以明确方式注明。本声明的法律结果将完全由本人承担。

作者签名：苏喜红

日期：2012 年 1 月 12 日

学位论文使用授权说明

本人完全了解哈尔滨工业大学关于保存、使用学位论文的规定，即：

（1）已获学位的研究生必须按学校规定提交学位论文；（2）学校可以采用影印、缩印或其他复制手段保存研究生上交的学位论文；（3）为教学和科研目的，学校可以将学位论文作为资料在图书馆及校园网上提供目录检索与阅览服务；（4）根据相关要求，向国家图书馆报送学位论文。

保密论文在解密后遵守此规定。

本人保证遵守上述规定。

作者签名：苏喜红

日期：2012 年 1 月 12 日

导师签名：杨孝奎

日期：2012 年 1 月 12 日

致 谢

首先衷心感谢我的导师杨孝宗教授在我学习期间对我的关心和指导。多年来，导师高尚的人格，严谨求实的态度，敏锐的思维，渊博的知识，一丝不苟敬业精神使我受益匪浅，在此对导师表示深深的敬意和衷心的感谢。

感谢张宏莉教授，作为我的责任专家，张老师在百忙之中对本论文做了详细审阅，提出了许多宝贵意见，在此向她表示最诚挚的谢意。

感谢刘宏伟教授，他渊博的知识、严谨的治学态度、敏锐的思维和勤恳的工作作风深深感染了我，始终激励着我不断地把工作做得更好。本论文从选题直至完成自始至终都得到了刘老师的精心指导和建议，深深感谢刘老师在我研究工作全过程中的诸多关怀和帮助。

感谢吴智博教授，吴老师对我的多篇拙作提出了许多点石成金的宝贵意见。吴老师给我的指导、帮助和教诲极大地完善了自己，提高了自己，在此向吴老师表示诚挚的谢意。

感谢实验室老师左德承、温东新、张展、罗丹彦、舒燕君、周海鹰、苗百利、刘辉等人对我的帮助。感谢师兄杨华、莫毓昌、郭建立、马闯、周连科、冯懿、朴鑫、王天宇，师姐赵靖、向琳、宋艳艳、侯春燕，师弟薛利兴、邓文平、景维鹏、王洪伟、钱钊、崔忠强、祁鑫、张大年、苏金光、张亚勇、梁佼、朱林(Julien Mousset)、王力、周健、李洋等，师妹张楠、李博、高妍、高青娥、余晓燕、韩建亚、刘璧怡、颜芬娜(Treesia Ivanna Gandawidjaja)、徐文芳以及 FTCL 实验室的全体成员给予我的帮助，我将永远记得与他们共同工作时的情景。

感谢我的家人在我漫长的求学生涯中给予我的关心和帮助。他们在我攻读博士期间给予了我极大的关怀与支持。特别感谢我的丈夫刘涛对我的理解、支持和爱。

最后，感谢曾经给我教诲的所有师长，感谢所有曾经关心过我，帮助过我的人！

个人简历

苏喜红，女，1981年8月16日出生于吉林省长春市。

2000年9月考入大庆石油学院（现改名为东北石油大学）计算机科学与技术学院，2004年7月本科毕业并获得工学学士学位。

2005年9月——2007年7月，在哈尔滨工业大学计算机科学与技术学院计算机系统结构学科学习并获得工学硕士学位。

2007年9月——2011年12月，在哈尔滨工业大学计算机科学与技术学院计算机系统结构学科攻读博士学位。

获奖情况：

2011年，获得哈尔滨工业大学校级三好学生；